Swift Başucu Rehberi

Bu rehber, *The Swift Programming Language* kitabının Language Guide bölümlerini kitaptaki sırayla özetler. Amacı, Swift tarafında bir mülakata ya da günlük işe hazırlanırken tek kaynaktan hızlıca tekrar yapabilmektir. Her bölümde temel kurallar, gerçek projelerde karşılaşılan kullanım şekilleri, tercihlerin gerekçeleri ve sık yapılan hatalar yer alır. Her bölüm, mülakat soruları ve örnek cevaplarla biter. Markdown önizlemesinde cevaplar kapalı gelir. Önce cevabı kendin ver, sonra açıp karşılaştır.

Kod örnekleri Swift 6 ile derlenmiştir. Her örnekte `import Foundation` satırının bulunduğu varsayılır.

Renkli kutuların anlamı:

> **Not**
>
> Ek bilgi ve bağlam.

> **Tercih**
>
> Tercih: hangi durumda hangi yolun seçileceği.

> **Önemli**
>
> Mülakatta mutlaka bilinmesi gereken kural.

> **Dikkat**
>
> Sık yapılan hata ya da tuzak.

> **Tehlike**
>
> Çalışma anında crash ya da ciddi bug'a yol açan durum.

İçindekiler

1. The Basics
2. Basic Operators
3. Strings and Characters
4. Collection Types
5. Control Flow
6. Functions
7. Closures

8. Enumerations
9. Structures and Classes
10. Properties
11. Methods
12. Subscripts
13. Inheritance
14. Initialization
15. Deinitialization
16. Optional Chaining
17. Error Handling
18. Concurrency
19. Macros
20. Type Casting
21. Nested Types
22. Extensions
23. Protocols
24. Generics
25. Opaque and Boxed Protocol Types
26. Automatic Reference Counting
27. Memory Safety
28. Access Control
29. Advanced Operators

1. The Basics

Swift, type safe bir dildir. Her değerin tipi derleme anında bellidir ve tipler arasında örtük dönüşüm yapılmaz. Bu bölüm dilin temel yapı taşlarını kapsar: constant ve variable tanımı, sayısal tipler, tuple, optional ve hata yakalamaya yardımcı olan assertion'lar.

Constants ve variables

`let` ile tanımlanan değer bir kez atanır, `var` ile tanımlanan değer sonradan değiştirilebilir. Varsayılan tercih her zaman `let` olmalıdır. Değerin değişmesi gerektiği ortaya çıktığında `var`'a geçilir.

```
let maxRetryCount = 3
var retryCount = 0

let defaultTimeout: TimeInterval = 30
var authToken: String?
```

`let` ile tanımlanan bir değerin ilk ataması tanımdan sonra da yapılabilir. Derleyici, her yolda tam olarak bir atama yapıldığını kontrol eder.

```
let isProduction = false

let apiHost: String
if isProduction {
    apiHost = "api.example.com"
} else {
    apiHost = "staging.api.example.com"
}
```

> **Önemli**
>
> `let` tercih edilmesinin asıl sebebi niyeti açık etmektir. Kodu okuyan kişi değerin değişmeyeceğini bilir, derleyici de yanlışlıkla yapılan atamayı yakalar. Value type'larda (struct, enum) `let` instance'ın tamamını değişmez kılar. Reference type'larda (class) ise yalnızca referansı sabitler, nesnenin içi değişebilir.

Type annotation ve type inference

Swift tipi çoğu zaman atanan değerden çıkarır. Tip açıkça yazılması gereken üç durum vardır: ilk değer verilmediğinde, literal'ın çıkarılan tipi istenen tip olmadığında ve okunabilirlik gerektiğinde.

```swift
let pageSize = 20              // Int
let progress = 0.75            // Double, Float değil
let opacity: Float = 0.75      // Float isteniyorsa açıkça yazılır
let userIDs: [UUID] = []       // boş koleksiyonda tip çıkarılamaz
```

Sayısal tipler

Tam sayılar için `Int` kullanılır. Değer negatif olamayacak olsa bile `UInt` tercih edilmez, çünkü API'lerin neredeyse tamamı `Int` bekler ve sürekli dönüşüm yapmak gerekir. `UInt` sadece bit işlemleri ve C kütüphaneleriyle çalışırken anlamlıdır. Ondalıklı sayılarda varsayılan `Double`'dır.

Swift tipler arasında örtük dönüşüm yapmaz. `Int` ile `Double` toplanacaksa biri açıkça dönüştürülür.

```swift
let totalItems = 7
let pageSize = 3

let fullPages = totalItems / pageSize                    // 2, kesirli kısım atılır
let exactPages = Double(totalItems) / Double(pageSize)   // 2.333...
let pageCount = (totalItems + pageSize - 1) / pageSize   // 3, yukarı yuvarlanmış sayfa sayısı
```

> **Dikkat**
>
> `Double(totalItems / pageSize)` yazmak `2.333` değil `2.0` verir. Parantez içindeki bölme önce `Int` olarak yapılır, kesirli kısım atılır, dönüşüm ondan sonra gelir. Dönüşüm bölmeden önce yapılmalıdır.

Para gibi kesin olması gereken değerlerde `Double` kullanılmaz. `Double` ikili tabanda saklandığı için `0.1` gibi değerleri tam temsil edemez. Bunun yerine `Decimal` kullanılır.

```swift
let a = 0.1 + 0.2
print(a == 0.3)                          // false
print(a)                                 // 0.30000000000000004

let price = Decimal(string: "19.99")!
let total = price * 3
print(total)                             // 59.97
```

Büyük sayıları okumak için literal'larda alt çizgi kullanılabilir: `let maxUploadSize = 10_000_000`.

> **Tehlike**
>
> Tam sayı taşması (overflow) Swift'te sessizce gerçekleşmez. Derleyici sabit değerlerdeki taşmayı (`Int.max + 1`) derleme anında yakalar. Çalışma anında hesaplanan bir değer taşarsa program crash olur. Bilinçli olarak taşma istenen durumlar için `&+` gibi overflow operatörleri vardır (bkz. Advanced Operators).

Type aliases

`typealias` mevcut bir tipe ikinci bir isim verir. Yeni bir tip oluşturmaz, sadece okunabilirliği artırır.

```
typealias UserID = UUID
typealias CompletionHandler = (Result<Data, Error>) -> Void

func fetchAvatar(for id: UserID, completion: @escaping CompletionHandler) {
    // ...
}
```

Booleans

Swift'te `Bool` dışında hiçbir tip koşul olarak kullanılamaz. C'deki gibi `if 1` ya da `if count` yazılmaz, koşul açıkça yazılır: `if count > 0`. Bool değişken isimleri bir soru gibi okunmalıdır: `isLoading`, `hasMorePages`, `canSubmit`, `shouldRetry`.

Tuples

Tuple, birden fazla değeri geçici olarak bir arada taşır. En yaygın kullanımı bir fonksiyondan birden fazla değer döndürmektir. Elemanlara isim verilmesi okunabilirliği ciddi şekilde artırır.

```
func parseStatusLine(_ line: String) -> (code: Int, reason: String)? {
    let parts = line.split(separator: " ", maxSplits: 2)
    guard parts.count == 3, let code = Int(parts[1]) else { return nil }
    return (code, String(parts[2]))
}

if let status = parseStatusLine("HTTP/1.1 404 Not Found") {
    print(status.code)      // 404
    print(status.reason)    // Not Found
}

let (code, _) = parseStatusLine("HTTP/1.1 200 OK") ?? (0, "")
print(code)                 // 200
```

> **Tercih**
>
> Tuple geçici sonuçlar içindir. Değer birden fazla katmandan geçecekse, bir koleksiyonda saklanacaksa ya da bir protokole (`Codable` , `Equatable` , `Identifiable`) uyması gerekecekse struct'a çevrilmelidir. Tuple'lar protokollere uyamaz ve metot içeremez.

Optionals

Optional, bir değerin olabileceğini ya da hiç olmayabileceğini ifade eder. `String?` tipi ya bir `String` taşır ya da `nil` 'dir. Swift'te `nil` , sadece optional tiplere atanabilir. Bu sayede "bu değer yok olabilir mi?" sorusunun cevabı tipin kendisinde yazılıdır.

> **Önemli**
>
> Optional aslında standart kütüphanede tanımlı iki case'li bir enum'dur: `case none` ve `case some(Wrapped)` . `nil` yazmak `.none` yazmakla aynıdır. `if let` , `??` ve optional chaining bu enum üzerine kurulmuş söz dizimi kolaylıklarıdır.

Optional'ın içindeki değere erişmenin birkaç yolu vardır.

if let. Değer sadece bir dalın içinde lazımsa kullanılır. Swift 5.7'den itibaren aynı isimle açarken kısa yazım geçerlidir: `if let token` .

```
func makeRequest(path: String, token: String?) -> URLRequest? {
    guard let url = URL(string: "https://api.example.com" + path) else { return nil }

    var request = URLRequest(url: url)
    if let token {
        request.setValue("Bearer \(token)", forHTTPHeaderField: "Authorization")
    }
    return request
}
```

guard let. Fonksiyonun devam edebilmesi için değerin şart olduğu durumlarda kullanılır. Değer yoksa erken çıkılır, açılan değer fonksiyonun geri kalanında kullanılabilir. `else` bloğu kapsamdan mutlaka çıkmalıdır (`return` , `throw` , `break` , `continue` ya da `fatalError`).

```
struct SignUpForm {
    var email: String?
    var password: String?
    var acceptedTerms: Bool
}

enum SignUpError: Error {
    case missingEmail, weakPassword, termsNotAccepted
}

func validate(_ form: SignUpForm) throws -> (email: String, password: String) {
    guard let email = form.email, !email.isEmpty else { throw SignUpError.missingEmail }
    guard let password = form.password, password.count >= 8 else { throw
SignUpError.weakPassword }
    guard form.acceptedTerms else { throw SignUpError.termsNotAccepted }
    return (email, password)
}
```

Bu örnekte koşullar birbirinin içine girmeden alt alta sıralanır. Her `guard` bir ön koşulu kontrol eder, mutlu yol (happy path) fonksiyonun en solunda ve en altında kalır.

Tercih

`if let` mi `guard let` mi? Açılan değer fonksiyonun geri kalanında lazımsa ve değer yokken devam etmenin anlamı yoksa `guard let`. Değer sadece bir dalda lazımsa ve yokluğu normal bir durumsa `if let`. İç içe birden fazla `if let` yazmaya başladıysan (pyramid of doom) `guard`'a geçmenin zamanı gelmiştir.

Dikkat

`if let` ve `guard let` sadece optional değerleri açar. `Bool` ya da optional olmayan bir değerle kullanılamaz. Bool kontrolü için düz koşul yazılır.

```
func submit(acceptedTerms: Bool) {
    // guard let acceptedTerms else { return }
    // error: initializer for conditional binding must have Optional type, not
'Bool'
    guard acceptedTerms else { return }
    print("Submitting")
}
```

Birden fazla binding ve koşul. Virgülle ayrılan her parça sırayla değerlendirilir ve biri başarısız olursa sonrakilere bakılmaz. Her binding kendi `let`'ini ister.

```
let userInfo: [String: Any] = ["id": 42, "name": "Ada"]

if let id = userInfo["id"] as? Int, let name = userInfo["name"] as? String, id > 0 {
    print("\(id): \(name)")
}
```

Nil coalescing (`??`). Değer yoksa varsayılan bir değer kullanılır. Sağ taraf sadece sol taraf `nil` olduğunda değerlendirilir.

```
let nickname: String? = nil
let fullName = "Ada Lovelace"
let displayName = nickname ?? fullName
```

Force unwrap (`!`). Değer `nil` ise program crash olur.

> **Tehlike**
>
> Force unwrap sadece `nil` olmasının programcı hatası anlamına geldiği yerlerde kabul edilebilir. Kodun içine sabit yazılmış geçerli bir URL ya da uygulama paketine eklenmiş bir dosya bu duruma örnektir. Ağdan, kullanıcıdan ya da diskten gelen veride asla kullanılmaz. Crash mesajını anlamlı kılmak için `guard let ... else { preconditionFailure("...") }` daha iyi bir seçenektir.

Implicitly unwrapped optional (`String!`). Optional'dır ama her kullanımda otomatik olarak açılır. Değer `nil` iken erişilirse crash olur. Pratikte UIKit'teki `@IBOutlet` gibi, nesne oluşturulduktan hemen sonra atanan ve ondan sonra hiç `nil` olmayan değerler için kullanılır. Yeni kodda tercih edilmez.

> **Dikkat**
>
> Optional bir değeri string interpolation içine koymak `Optional("Ada")` gibi bir çıktı üretir ve derleyici uyarı verir. Önce açılmalı ya da `??` ile varsayılan verilmelidir.

Error handling

Optional bir değerin yokluğunu ifade eder ama nedenini söylemez. Bir işlemin neden başarısız olduğu önemliyse fonksiyon `throws` olarak işaretlenir ve hata fırlatır. Konunun ayrıntısı Error Handling bölümündedir.

Assertions ve preconditions

Bu fonksiyonlar, kodun doğru çalışması için mutlaka sağlanması gereken koşulları kontrol eder. Koşul sağlanmazsa program durur.

Fonksiyon	Debug build	Release build	Kullanım
`assert`	Çalışır	Çalışmaz	Geliştirme sırasında iç tutarlılık kontrolü
`precondition`	Çalışır	Çalışır	Fonksiyonu çağıranın uyması gereken sözleşme
`fatalError`	Çalışır	Çalışır	Ulaşılmaması gereken kod, `init?(coder:)` gibi zorunlu ama desteklenmeyen yollar

```
func page<T>(_ items: [T], number: Int, size: Int) -> ArraySlice<T> {
    precondition(number >= 0, "Page number must be non-negative")
    precondition(size > 0, "Page size must be positive")

    let start = number * size
    guard start < items.count else { return [] }
    return items[start..<min(start + size, items.count)]
}
```

> **Not**
>
> Kullanıcının yanlış veri girmesi bir programcı hatası değildir. Bu tür durumlar `precondition` ile değil, optional ya da hata fırlatma ile ele alınır. Assertion'lar sadece "bu asla olmamalı" denen durumlar içindir.

Mülakat soruları

S. let ile var arasındaki fark ne, varsayılan olarak hangisini kullanırsın ve neden?

C. `let` bir kez atanır, `var` değiştirilebilir. Varsayılan `let`'tir çünkü niyeti açık eder. Okuyan kişi değerin değişmeyeceğini bilir ve derleyici yanlışlıkla yapılan atamayı yakalar. Struct'larda `let` instance'ın tamamını değişmez yapar. Class'ta ise sadece referansı sabitler, nesnenin property'leri değişmeye devam edebilir.

S. Optional nedir, Swift'te nasıl tanımlanmıştır?

C. Bir değerin olabileceğini ya da olmayabileceğini ifade eden tiptir. Standart kütüphanede `none` ve `some(Wrapped)` case'lerine sahip generic bir enum olarak tanımlıdır. `nil` aslında `.none`'dır. Bu tasarım sayesinde bir değerin yok olabileceği bilgisi tip sisteminde yer alır ve derleyici, değer açılmadan kullanılmasını engeller.

S. if let ile guard let arasındaki fark ne, hangisini ne zaman seçersin?

C. `if let` ile açılan değer sadece o bloğun içinde geçerlidir. `guard let` ile açılan değer ise fonksiyonun geri kalanında kullanılabilir ve `else` bloğu kapsamdan çıkmak zorundadır. Fonksiyonun devam etmesi için değer şartsa `guard let` kullanırım. Böylece ön koşullar başta toplanır, iç içe bloklar oluşmaz ve asıl iş akışı girintisiz kalır. Değer sadece bir dalda lazımsa ve yokluğu normal bir durumsa `if let` kullanırım.

S. Force unwrap kullanmak ne zaman kabul edilebilir?

C. `nil` olmasının programcı hatası anlamına geldiği durumlarda. Kod içinde sabit yazılmış geçerli bir URL ya da uygulama paketindeki bir dosya buna örnektir. Ağdan, diskten ya da kullanıcıdan gelen veride kabul edilemez. Böyle durumlarda bile `guard let` ile `preconditionFailure` kullanmak crash mesajını daha anlamlı yapar.

S. assert, precondition ve fatalError arasındaki fark ne?

C. `assert` sadece debug build'de çalışır, geliştirme sırasında iç tutarlılığı kontrol etmek için kullanılır. `precondition` release build'de de çalışır ve fonksiyonu çağıranın uyması gereken sözleşmeyi korur. `fatalError` her koşulda programı durdurur ve ulaşılmaması gereken kod yolları için kullanılır.

S. Para tutarlarını neden Double ile tutmayız?

C. `Double` ikili tabanda saklandığı için `0.1` gibi ondalık değerleri tam temsil edemez. Küçük yuvarlama hataları toplama ve çarpma işlemleri boyunca birikir. Örneğin `0.1 + 0.2 == 0.3` ifadesi `false` döner. Para için `Decimal` ya da en küçük birim cinsinden `Int` (kuruş, cent) kullanılır.

S. Tuple ne zaman, struct ne zaman kullanılır?

C. Tuple, bir fonksiyondan birden fazla değer döndürmek gibi geçici ve yerel durumlar için uygundur. Değer birden fazla katmandan geçiyorsa, saklanıyorsa, davranış (metot) gerekiyorsa ya da bir protokole uyması gerekiyorsa struct kullanılır. Tuple'lar protokollere uyamaz, bu yüzden `Codable` ya da `Equatable` olamaz.

2. Basic Operators

Swift'in operatörleri C'ye benzer ama hataya yol açan davranışları kaldırılmıştır. Atama işlemi değer döndürmez, tam sayı taşması crash ile sonuçlanır, aralık operatörleri ise dilin bir parçasıdır.

Assignment

Atama operatörü (`=`) değer döndürmez. Bu sayede `==` yerine yanlışlıkla `=` yazılan bir `if` koşulu derlenmez. Bileşik atama operatörleri (`+=` , `-=` , `*=`) de değer döndürmez, dönüş tipleri `Void` 'dir.

```
var badgeCount = 3
badgeCount += 1
print(badgeCount)          // 4
```

> **Dikkat**
>
> `print(badgeCount += 1)` yazmak sayıyı değil `()` yazdırır. Bileşik atama bir değer üretmez. Önce işlem yapılır, sonra değişken ayrıca yazdırılır.

Arithmetic ve remainder

`+` , `-` , `*` , `/` operatörleri aynı tipteki iki değer arasında çalışır. Tam sayı bölmesi kesirli kısmı atar. `%` kalan operatörüdür ve sonucun işareti soldaki sayının işaretini alır: `-7 % 3` sonucu `-1` 'dir. `Double` için `%` yoktur, onun yerine `truncatingRemainder(dividingBy:)` kullanılır.

```
let elapsedSeconds = 3_725
let hours = elapsedSeconds / 3600
let minutes = (elapsedSeconds % 3600) / 60
let seconds = elapsedSeconds % 60
print("\(hours)h \(minutes)m \(seconds)s")     // 1h 2m 5s

let rowIndex = 14
let isEvenRow = rowIndex.isMultiple(of: 2)     // % 2 == 0 yerine daha okunaklı
```

Comparison

`==` , `!=` , `<` , `>` , `<=` , `>=` her zaman `Bool` döndürür. Kendi tiplerin için `==` kullanabilmek için tipin `Equatable` 'a, `<` kullanabilmek için `Comparable` 'a uyması gerekir. Class instance'larında aynı nesne olup olmadığını kontrol etmek için `===` kullanılır (bkz. Structures and Classes).

Tuple'lar, yedi elemandan azsa ve elemanları karşılaştırılabilirse soldan sağa doğru karşılaştırılır. Bu özellik çok kriterli sıralamada işe yarar.

```
struct Release {
    let major: Int
    let minor: Int
    let patch: Int
}

let releases = [
    Release(major: 2, minor: 1, patch: 0),
    Release(major: 1, minor: 9, patch: 3),
    Release(major: 2, minor: 0, patch: 7)
]

let sorted = releases.sorted { ($0.major, $0.minor, $0.patch) < ($1.major, $1.minor,
$1.patch) }
print(sorted.map { "\($0.major).\($0.minor).\($0.patch)" })    // ["1.9.3", "2.0.7",
"2.1.0"]
```

Ternary conditional

`koşul ? a : b` kısa iki seçenekli ifadeler içindir. İç içe ternary yazılmaz, okunabilirlik hızla düşer.

```
let unreadCount = 1
let label = "\(unreadCount) " + (unreadCount == 1 ? "message" : "messages")
```

> **Dikkat**
>
> Ternary'nin önceliği `+` ve `==` operatörlerinden düşüktür. Yukarıdaki parantezler kaldırılırsa ifade `("\(unreadCount) " + unreadCount) == 1 ? ...` gibi yorumlanır ve derlenmez. Ternary başka bir ifadenin parçasıysa her zaman paranteze alınır.

Nil-coalescing

`a ?? b`, `a` doluysa içindeki değeri, `nil` ise `b`'yi döndürür. `b` sadece gerektiğinde değerlendirilir, bu yüzden sağ tarafta pahalı bir işlem olması sorun yaratmaz. Zincirleme kullanılabilir: `cachedName ?? remoteName ?? "Guest"`.

Range operators

Operatör	Anlamı	Örnek
`a...b`	Closed range, `b` dahil	`1...5` → 1, 2, 3, 4, 5
`a..<b`	Half-open range, `b` hariç	`0..<items.count`
`a...` / `...b` / `..<b`	One-sided range	`items[2...]`, `items[..<3]`

Dizilerin index'leri sıfırdan başlayıp `count - 1`'de bittiği için half-open range doğal seçimdir. Ancak index üzerinde dönmek gerekiyorsa `0..<items.count` yerine `items.indices` kullanmak daha güvenlidir.

```
let notifications = ["Build passed", "PR approved", "Deploy started", "Deploy finished"]

for index in notifications.indices {
    print("\(index + 1). \(notifications[index])")
}

let latestTwo = notifications.suffix(2)
let firstTwo = notifications.prefix(2)
```

> **Tehlike**
>
> Bir diziyi range ile kesmek `ArraySlice` döndürür ve bu slice orijinal dizinin index'lerini korur. `let tail = notifications[2...]` dedikten sonra `tail[0]` yazmak crash olur, çünkü `tail`'in ilk index'i `2`'dir. Slice üzerinde `first`, `startIndex` ya da `Array(tail)` dönüşümü kullanılır.

> **Tehlike**
>
> Alt sınırı üst sınırdan büyük bir range oluşturmak (`5...3`) çalışma anında crash olur. Sınırlar hesaplanıyorsa önce kontrol edilmelidir.

Logical operators

`!`, `&&`, `||` operatörleri `Bool` üzerinde çalışır ve short-circuit davranışı gösterir. `&&` ifadesinde sol taraf `false` ise sağ taraf hiç değerlendirilmez. Bu özellik güvenli kontrol yazmak için kullanılır.

```
let recentSearches: [String] = []

if !recentSearches.isEmpty && recentSearches[0].count > 2 {
    print("Suggest: \(recentSearches[0])")
}
```

Burada dizi boşsa `recentSearches[0]` ifadesine hiç ulaşılmaz. Sıra ters yazılsaydı boş dizide crash olurdu. Birden fazla `&&` ve `||` bir arada kullanılıyorsa, öncelik kurallarını bilmek yerine parantez kullanmak niyeti açıkça gösterir.

Mülakat soruları

S. Swift'te atama operatörü neden değer döndürmez?

C. C'de `if (x = y)` gibi, `==` yerine yanlışlıkla `=` yazılan koşullar derlenir ve zor bulunan hatalara yol açar. Swift atamayı `Void` döndüren bir işlem yaparak bu hatayı derleme anında engeller.

S. Int taşması olduğunda Swift ne yapar, C'den farkı ne?

C. C'de işaretli tam sayı taşması tanımsız davranıştır ve program yanlış değerle çalışmaya devam edebilir. Swift'te taşma çalışma anında yakalanır ve program durur. Taşmanın bilinçli olarak istendiği durumlar (hash hesaplama gibi) için `&+`, `&-`, `&*` overflow operatörleri kullanılır.

S. Short-circuit evaluation nedir, pratikte nerede işine yarar?

C. `&&` ifadesinde sol taraf `false` ise, `||` ifadesinde sol taraf `true` ise sağ taraf hiç değerlendirilmez. Güvenli kontrol yazmak için kullanılır. `!items.isEmpty && items[0] == x` ifadesinde dizi boşsa index erişimine hiç ulaşılmaz. Ayrıca pahalı kontrolleri sağa koyarak gereksiz hesaplamadan kaçınılabilir.

S. Bir diziden aldığın slice'ın ilk elemanına neden [0] ile erişemezsin?

C. `ArraySlice`, orijinal dizinin belleğini paylaşır ve onun index'lerini korur. `array[3...]` ile alınan slice'ın ilk index'i `3`'tür, `0` değil. `slice[0]` crash olur. `slice.first`, `slice[slice.startIndex]` kullanılır ya da slice `Array(slice)` ile yeni bir diziye çevrilir.

3. Strings and Characters

Swift'te `String` bir value type'tır ve Unicode'u tam olarak destekler. Bu iki özellik, string'lerle çalışırken karşılaşılan alışılmadık kuralların (Int index olmaması, `count`'un maliyetli olması) sebebidir.

Literals

Tek satırlık string'ler çift tırnakla yazılır. Çok satırlı metinler için üç tırnak kullanılır. Kapanış tırnağının girintisi, satırlardan ne kadar boşluk silineceğini belirler.

```
let query = """
    SELECT id, title
    FROM articles
    WHERE published = 1
    """
```

Ters bölü ve tırnak işaretlerinin çok olduğu metinlerde raw string kullanılır. `#"..."#` içinde kaçış karakterleri işlenmez. Interpolation için `\#(...)` yazılır.

```
let pattern = #"^\d{3}-\d{4}$"#
let field = "email"
let json = #"{"field": "\#(field)", "required": true}"#
print(json)     // {"field": "email", "required": true}
```

Value semantics

String bir value type'tır. Bir değişkene atandığında ya da bir fonksiyona verildiğinde kopyalanır. Kopyalama işlemi copy on write sayesinde ucuzdur: gerçek kopya sadece iki taraftan biri değiştirildiğinde yapılır.

Interpolation ve CustomStringConvertible

Bir tipin interpolation içinde nasıl görüneceği `CustomStringConvertible` protokolü ile belirlenir. Loglama ve hata mesajlarında oldukça kullanışlıdır.

```swift
struct HTTPError: Error, CustomStringConvertible {
    let statusCode: Int
    let path: String

    var description: String {
        "HTTP \(statusCode) at \(path)"
    }
}

let error = HTTPError(statusCode: 503, path: "/v1/orders")
print("Request failed: \(error)")      // Request failed: HTTP 503 at /v1/orders
```

Unicode ve Character

Swift'te bir `Character`, ekranda tek karakter olarak görünen birimdir (extended grapheme cluster). Bu birim birden fazla Unicode scalar'dan oluşabilir. Bu yüzden aynı metin için farklı "uzunluklar" vardır.

```swift
let reaction = "👍🏽"
print(reaction.count)                   // 1, kullanıcının gördüğü karakter sayısı
print(reaction.unicodeScalars.count)    // 2, el + ten rengi
print(reaction.utf16.count)             // 4, NSString ve UIKit'in kullandığı birim
print(reaction.utf8.count)              // 8, byte cinsinden boyut
```

> **Tercih**
>
> Hangi uzunluk ne zaman kullanılır? Kullanıcıya gösterilen karakter limiti için `count`. `UITextField` delegate metotları ve `NSRange` ile çalışırken `utf16.count`, çünkü Foundation tarafı UTF-16 ile çalışır. Ağ üzerinden gönderilen veri ya da veritabanı alan boyutu için `utf8.count`.

Index ve Substring

String'e `Int` ile erişilemez. Karakterler farklı byte uzunluklarında olabildiği için n'inci karaktere ulaşmak baştan saymayı gerektirir. Swift bu maliyeti gizlemek yerine `String.Index` tipi ile görünür kılar.

```swift
let trackingCode = "TR-2026-88412"

let firstChar = trackingCode[trackingCode.startIndex]                    // "T"
let fifth = trackingCode[trackingCode.index(trackingCode.startIndex, offsetBy: 4)]  // "0"

if let dash = trackingCode.firstIndex(of: "-") {
    let country = trackingCode[..<dash]                // "TR"
    print(country)
}
```

Pratikte doğrudan index ile uğraşmak yerine üst seviye API'ler tercih edilir: `prefix(_:)`, `suffix(_:)`, `dropFirst()`, `split(separator:)`, `hasPrefix(_:)`, `contains(_:)`.

String'den range ile alınan parça `Substring` tipindedir. `Substring`, orijinal string'in belleğini paylaşır ve kopyalama yapmaz. Bu sayede ayrıştırma (parsing) işlemleri hızlı çalışır.

```swift
struct LogEntry {
    let level: String
    let message: String
}

func parse(_ line: String) -> LogEntry? {
    let parts = line.split(separator: "|", maxSplits: 1)
    guard parts.count == 2 else { return nil }
    return LogEntry(
        level: String(parts[0]).trimmingCharacters(in: .whitespaces),
        message: String(parts[1]).trimmingCharacters(in: .whitespaces)
    )
}

print(parse("ERROR | Token expired")?.message ?? "-")    // Token expired
```

> **Önemli**
>
> `Substring` uzun süre saklanmamalıdır. Küçük bir parça bile tutulduğu sürece orijinal string'in tamamı bellekte kalır. Bir property'ye atanacak ya da başka bir katmana verilecekse `String(...)` ile yeni bir string'e çevrilir. Yukarıdaki `LogEntry` bu yüzden `String` saklar.

Karşılaştırma ve arama

`==` iki string'i Unicode açısından eşdeğer olup olmadıklarına göre karşılaştırır. Örneğin "é" harfinin tek scalar ile yazılmış hali ile "e" ve aksan işaretinden oluşan hali eşit sayılır. Büyük küçük harf duyarlılığı ve dil kuralları için Foundation API'leri kullanılır.

```
let titles = ["Zebra", "apple", "Äpfel", "item 10", "item 2"]

let sortedForUser = titles.sorted { $0.localizedStandardCompare($1) == .orderedAscending
}
print(sortedForUser)    // ["Äpfel", "apple", "item 2", "item 10", "Zebra"]

let matches = titles.filter { $0.localizedStandardContains("APP") }
print(matches)          // ["apple"]
```

> **Tercih**
>
> Kullanıcıya gösterilen listeleri sıralarken `<` yerine `localizedStandardCompare` kullanılır. Bu fonksiyon büyük küçük harfi, aksanları ve sayıları Finder'daki gibi doğal sırayla ele alır ("item 2", "item 10"dan önce gelir). Arama kutularında `localizedStandardContains` aynı rolü üstlenir.

Sık kullanılan işlemler

```
let input = "  Ada@Example.com \n"
let email = input.trimmingCharacters(in: .whitespacesAndNewlines).lowercased()
print(email)                                    // ada@example.com

let tags = "swift, ios ,  swiftui".split(separator: ",").map { $0.trimmingCharacters(in:
.whitespaces) }
print(tags)                                     // ["swift", "ios", "swiftui"]

let slug = "Release Notes 2.1".lowercased().replacingOccurrences(of: " ", with: "-")
print(slug)                                     // release-notes-2.1
```

> **Tercih**
>
> Boşluk kontrolü için `count == 0` yerine `isEmpty` kullanılır. `isEmpty` sabit sürede çalışır, `count` ise tüm karakterleri saymak zorundadır.

Sayı ve tarih biçimlendirme

Kullanıcıya gösterilen sayıları `String(format:)` ile biçimlendirmek hem C tarzı format hatalarına açıktır hem de kullanıcının bölge ayarlarını hesaba katmaz. Modern Foundation'da `formatted()` kullanılır.

```swift
let amount = Decimal(string: "1249.5")!
print(amount.formatted(.currency(code: "TRY").locale(Locale(identifier: "tr_TR"))))   // ₺1.249,50

let ratio = 0.873
print(ratio.formatted(.percent.precision(.fractionLength(1))))
```

Mülakat soruları

S. Swift'te bir String'e neden Int ile index veremeyiz?

C. Swift'te bir karakter, birden fazla Unicode scalar'dan oluşabilir ve bellekte farklı uzunluklarda yer kaplar. Bu yüzden n'inci karaktere ulaşmak baştan saymayı gerektirir, yani O(n) bir işlemdir. Int subscript bu maliyeti O(1) gibi gösterirdi. Swift bunun yerine `String.Index` tipini kullanarak maliyeti görünür kılar.

S. Substring neden var, ne zaman String'e çevirirsin?

C. Substring orijinal string'in belleğini paylaşır ve kopyalama yapmaz, bu yüzden parsing gibi işlemler hızlıdır. Ancak tutulduğu sürece orijinal string'in tamamını bellekte tutar. Kısa süreli işlemlerde Substring ile çalışırım. Bir property'ye atanacak, saklanacak ya da başka bir katmana verilecekse `String(...)` ile çeviririm.

S. "👍🏽".count kaç döner, unicodeScalars.count kaç döner, neden?

C. `count` 1 döner, çünkü kullanıcı tek bir karakter görür. `unicodeScalars.count` 2 döner, çünkü bu emoji bir el scalar'ı ile bir ten rengi scalar'ından oluşur. Swift'te `Character` tek scalar değil, ekranda tek karakter olarak görünen bir birimdir (extended grapheme cluster).

S. String bir value type. Büyük bir string'i fonksiyona vermek pahalı mı?

C. Değil. String copy on write kullanır. Kopyalama sırasında içerik kopyalanmaz, iki değişken aynı depolama alanını paylaşır. Gerçek kopya ancak taraflardan biri değiştirildiğinde yapılır.

S. isEmpty ile count == 0 arasında fark var mı?

C. Sonuç aynıdır ama maliyet farklıdır. `isEmpty` sabit sürede çalışır. String'de `count` ise tüm karakterleri saymak zorunda olduğu için O(n)'dir. Bu yüzden boşluk kontrolünde her zaman `isEmpty` kullanılır.

4. Collection Types

Swift'in üç temel koleksiyonu vardır: `Array`, `Set` ve `Dictionary`. Üçü de generic'tir, sadece tek tipte eleman tutar ve value type'tır. Kopyalama copy on write ile ucuz hale getirilmiştir.

Hangisini seçmeli

İhtiyaç	Koleksiyon	Neden
Sıra önemli, tekrar olabilir	`Array`	Index ile O(1) erişim, sıra korunur
Her eleman bir kez, hızlı "var mı?" kontrolü	`Set`	`contains` O(1), elemanlar `Hashable` olmalı
Bir anahtar ile değere erişim	`Dictionary`	Anahtar ile O(1) erişim, anahtarlar `Hashable` olmalı

Array

```
var recentSearches: [String] = []
recentSearches.append("swift concurrency")
recentSearches.insert("actors", at: 0)
recentSearches.append(contentsOf: ["sendable", "async let"])

print(recentSearches.count)              // 4
print(recentSearches.first ?? "none")    // actors

recentSearches.removeAll { $0.hasPrefix("async") }
```

Sık kullanılan API'ler:

API	Açıklama
`first`, `last`	Optional property, dizi boşsa `nil`. Parantez almaz
`isEmpty`, `count`	Property, parantez almaz
`contains(_:)`, `contains(where:)`	Eleman var mı
`firstIndex(of:)`, `firstIndex(where:)`	Eşleşen ilk index, optional
`removeAll(where:)`	Koşula uyanları siler
`enumerated()`	`(offset, element)` çiftleri
`sorted()`, `sorted(by:)`	Sıralanmış yeni dizi döndürür
`sort()`, `sort(by:)`	Diziyi yerinde sıralar

> **Dikkat**
>
> `count`, `first`, `last` ve `isEmpty` metot değil property'dir. `items.count()` yazmak derlenmez.

> **Tehlike**
>
> `items[0]` dizi boşsa crash olur. Var olmayabilecek elemanlara `first`, `last` ya da `indices.contains(index)` kontrolü ile erişilir.

`append` ortalamada O(1)'dir. Dizinin başına ya da ortasına ekleme ve silme ise O(n)'dir, çünkü sonraki elemanların kaydırılması gerekir. Eleman sayısı önceden biliniyorsa `reserveCapacity(_:)` ile tekrar tekrar bellek ayırmanın önüne geçilebilir.

Set

Set, sırasız ve tekrarsız bir koleksiyondur. Elemanların `Hashable` olması gerekir. Asıl gücü `contains` işleminin sabit sürede çalışmasıdır.

```
let blockedUserIDs: Set<Int> = [12, 98, 455]
let commentAuthorIDs = [7, 12, 33, 455, 801]

let visibleAuthors = commentAuthorIDs.filter { !blockedUserIDs.contains($0) }
print(visibleAuthors)        // [7, 33, 801]
```

> **Tercih**
>
> Bir döngü içinde bir dizi üzerinde `contains` çağrılıyorsa toplam maliyet O(n × m) olur. Aranan koleksiyonu önce `Set`'e çevirmek maliyeti O(n + m)'ye düşürür. Binlerce elemanlı listelerde bu fark kullanıcı tarafından hissedilir.

Set işlemleri iki kümeyi karşılaştırmak için kullanılır. Aşağıdaki örnekte bir kullanıcının sahip olduğu yetkiler ile bir ekranın gerektirdiği yetkiler karşılaştırılıyor.

```
let granted: Set = ["read", "comment", "upload"]
let required: Set = ["read", "upload", "delete"]

let all = granted.union(required)                  // read, comment, upload, delete
let common = granted.intersection(required)        // read, upload
let missing = required.subtracting(granted)        // delete, eksik olan yetki
let exclusive = granted.symmetricDifference(required)  // comment, delete
let canOpen = required.isSubset(of: granted)       // false
```

İşlem	Sonuç
`a.union(b)`	İki kümedeki tüm elemanlar
`a.intersection(b)`	Sadece iki kümede de olanlar
`a.subtracting(b)`	`a`'da olup `b`'de olmayanlar
`a.symmetricDifference(b)`	Sadece birinde olanlar, ortak olanlar hariç
`a.isSubset(of: b)`	`a`'nın tüm elemanları `b`'de var mı

> **Not**
>
> Set'te elemanların sırası garanti edilmez ve programın her çalıştırılmasında değişebilir. Ekranda gösterilecekse önce `sorted()` ile diziye çevrilir.

Dictionary

Dictionary, anahtar ile değer eşleştirir. Anahtarın `Hashable` olması gerekir. Subscript ile okuma her zaman optional döner, çünkü anahtarın sözlükte olup olmadığı derleme anında bilinemez.

```
var headers: [String: String] = [
    "Content-Type": "application/json",
    "Accept-Language": "tr-TR"
]

headers["Authorization"] = "Bearer abc123"     // ekleme ya da güncelleme
headers["Accept-Language"] = nil               // anahtarı siler

if let contentType = headers["Content-Type"] {
    print(contentType)
}

for (key, value) in headers.sorted(by: { $0.key < $1.key }) {
    print("\(key): \(value)")
}
```

Varsayılan değerli subscript, sayma ve gruplama işlerini sadeleştirir.

```swift
let events = ["login", "purchase", "login", "logout", "login"]

var counts: [String: Int] = [:]
for event in events {
    counts[event, default: 0] += 1
}
print(counts["login"] ?? 0)      // 3
```

Bir diziden sözlük oluşturmak için iki hazır initializer vardır.

```swift
struct Order {
    let id: Int
    let customerID: Int
    let total: Decimal
}

let orders = [
    Order(id: 1, customerID: 10, total: 120),
    Order(id: 2, customerID: 11, total: 80),
    Order(id: 3, customerID: 10, total: 45)
]

let ordersByCustomer = Dictionary(grouping: orders, by: \.customerID)
print(ordersByCustomer[10]?.count ?? 0)          // 2

let ordersByID = Dictionary(uniqueKeysWithValues: orders.map { ($0.id, $0) })
print(ordersByID[3]?.total ?? 0)                 // 45
```

Tehlike

`Dictionary(uniqueKeysWithValues:)` aynı anahtar iki kez gelirse crash olur. Anahtarların benzersiz olduğundan emin değilsen `Dictionary(_:uniquingKeysWith:)` kullanılır ve çakışmada hangi değerin kalacağı belirtilir: `Dictionary(pairs, uniquingKeysWith: { _, latest in latest })`.

Not

Dictionary'de de sıra garanti edilmez. Sabit bir sırayla göstermek gerekiyorsa anahtarlar ya da çiftler sıralanır.

Value semantics

Üç koleksiyon da value type'tır. Bir diziyi başka bir değişkene atamak ya da fonksiyona vermek bağımsız bir kopya oluşturur. Kopya üzerinde yapılan değişiklik orijinali etkilemez.

```
var draft = ["Intro", "Setup"]
let published = draft
draft.append("Advanced")

print(published.count)    // 2
print(draft.count)        // 3
```

Bu kopyalama copy on write sayesinde ucuzdur. `published` ve `draft` ilk başta aynı belleği paylaşır. `draft` değiştirildiği anda gerçek kopya yapılır.

> **Önemli**
>
> `let` ile tanımlanan bir koleksiyon tamamen değişmezdir. Eleman eklenemez, silinemez, değiştirilemez. Bu durum sadece Swift'in kendi koleksiyonları için geçerlidir. Class tipindeki elemanları tutan `let` bir dizide, dizinin kendisi değişmez ama içindeki nesnelerin property'leri değiştirilebilir.

Mülakat soruları

S. Array, Set ve Dictionary arasında nasıl seçim yaparsın?

C. Sıra önemliyse ya da aynı eleman birden fazla kez olabiliyorsa `Array`. Elemanlar benzersizse ve sık sık "bu eleman var mı?" diye soruluyorsa `Set`, çünkü `contains` O(1)'dir. Değere bir anahtar üzerinden erişiliyorsa `Dictionary`. Örneğin engellenen kullanıcı ID'leri `Set`, ID'ye göre önbelleğe alınmış kullanıcılar `Dictionary`, ekranda gösterilen sıralı liste `Array` olur.

S. Hashable ne demek, Set ve Dictionary neden buna ihtiyaç duyar?

C. `Hashable` bir değerin tam sayı bir hash değeri üretebildiği anlamına gelir. Set ve Dictionary elemanları bu hash değerine göre yerleştirir. Böylece arama, tüm elemanları dolaşmak yerine doğrudan ilgili konuma bakarak sabit sürede yapılır. Eşit iki değer aynı hash'i üretmek zorundadır. Bu yüzden `Hashable`, `Equatable`'ı da gerektirir. Struct ve enum'larda tüm property'ler `Hashable` ise uyum otomatik sentezlenir.

S. Copy on write nedir?

C. Value type koleksiyonlar kopyalandığında içerik hemen kopyalanmaz. İki değişken aynı depolama alanını paylaşır. Taraflardan biri değiştirilmek istendiğinde depolama alanının başka biri tarafından paylaşılıp paylaşılmadığı kontrol edilir. Paylaşılıyorsa gerçek kopya o anda yapılır. Bu sayede value semantics korunur ve gereksiz kopyalama maliyeti ödenmez.

S. Dictionary subscript'i neden optional döner?

C. Verilen anahtarın sözlükte olup olmadığı derleme anında bilinemez. Swift, anahtar yokken crash olmak ya da anlamsız bir varsayılan değer döndürmek yerine bu belirsizliği tipe yansıtır. Varsayılan değer isteniyorsa `dict[key, default: value]` ya da `dict[key] ?? value` kullanılır.

S. array.first ile array[0] arasındaki fark ne?

C. `first` optional döner ve dizi boşsa `nil` olur. `array[0]` ise dizi boşsa crash olur. Dizinin boş olmadığı garanti değilse `first` kullanılır.

S. Bir listeyi filtrelerken içerde başka bir dizide contains çağırıyorsun. Liste büyüyünce yavaşlıyor. Ne yaparsın?

C. Dizide `contains` O(n)'dir. Dış döngüyle birlikte toplam maliyet O(n × m) olur. İçerde aranan diziyi bir kez `Set`'e çeviririm. Böylece her `contains` O(1) olur ve toplam maliyet O(n + m)'ye iner.

5. Control Flow

Swift'te akış kontrolü C'ye benzer ama daha güvenlidir. `switch` tüm durumları kapsamak zorundadır, case'ler arasında örtük geçiş (implicit fallthrough) yoktur. `if` ve `switch` değer üreten ifadeler olarak da kullanılabilir.

for-in

`for-in` diziler, range'ler, sözlükler ve `Sequence` protokolüne uyan her tip üzerinde çalışır. Döngüye `where` ile filtre eklenebilir.

```
struct Member {
    let name: String
    let isActive: Bool
}

let members = [Member(name: "Ada", isActive: true), Member(name: "Linus", isActive:
false)]

for member in members where member.isActive {
    print("Sending invite to \(member.name)")
}
```

Belirli adımlarla ilerlemek için `stride` kullanılır. Aşağıdaki örnek bir listeyi sabit boyutlu parçalar halinde işler. Toplu yükleme ya da API'nin tek seferde kabul ettiği kayıt sınırı gibi durumlarda bu kalıp sık görülür.

```
let photoIDs = Array(1...10)
let batchSize = 4

for start in stride(from: 0, to: photoIDs.count, by: batchSize) {
    let end = min(start + batchSize, photoIDs.count)
    print("Uploading \(Array(photoIDs[start..<end]))")
}
// Uploading [1, 2, 3, 4]
// Uploading [5, 6, 7, 8]
// Uploading [9, 10]
```

`stride(from:to:by:)` bitiş değerini dahil etmez, `stride(from:through:by:)` dahil eder.

İki diziyi eleman eleman eşleştirmek için `zip`, sıra numarası ile birlikte dolaşmak için `enumerated()` kullanılır.

```swift
let fields = ["email", "password"]
let messages = ["Invalid format", "Too short"]

for (field, message) in zip(fields, messages) {
    print("\(field): \(message)")
}

for (offset, field) in fields.enumerated() {
    print("\(offset + 1). \(field)")
}
```

> **Dikkat**
>
> `enumerated()` index değil sıra numarası (offset) üretir ve her zaman sıfırdan başlar. Bir `ArraySlice` üzerinde bu sayı ile subscript yapmak yanlış elemana ya da crash'e götürür. Gerçek index lazımsa `zip(slice.indices, slice)` kullanılır.

Optional elemanlardan oluşan bir dizide sadece dolu olanları dolaşmak için `for case let` kalıbı kullanılır.

```swift
let avatarURLs: [String?] = ["a.png", nil, "c.png"]

for case let url? in avatarURLs {
    print("Prefetching \(url)")
}
```

while ve repeat-while

Kaç kez döneceği önceden bilinmeyen döngülerde `while` kullanılır. Koşul her turun başında kontrol edilir.

```swift
var attempt = 0
var delay = 0.5
let maxAttempts = 4

while attempt < maxAttempts {
    attempt += 1
    print("Attempt \(attempt), next delay \(delay)s")
    delay *= 2
}
```

`repeat-while` gövdeyi en az bir kez çalıştırır, koşulu turun sonunda kontrol eder. Cursor tabanlı sayfalamada ilk sayfa her zaman istendiği için doğal bir seçimdir.

```
func fetchPage(after cursor: Int?) -> (items: [String], nextCursor: Int?) {
    switch cursor {
    case nil: return (["a", "b"], 2)
    case 2?: return (["c", "d"], 4)
    default: return (["e"], nil)
    }
}

var cursor: Int? = nil
var allItems: [String] = []

repeat {
    let page = fetchPage(after: cursor)
    allItems += page.items
    cursor = page.nextCursor
} while cursor != nil

print(allItems)    // ["a", "b", "c", "d", "e"]
```

if

Swift 5.9'dan itibaren `if` ve `switch` bir değer üreten ifade (expression) olarak kullanılabilir. Her dal tek bir ifade içermeli ve tüm dallar aynı tipte değer üretmelidir. Bu kullanım, sadece atama yapmak için `var` tanımlayıp sonra dallarda doldurmanın önüne geçer.

```
let itemCount = 3
let subtitle = if itemCount == 0 { "Your cart is empty" } else { "\(itemCount) items in
cart" }
print(subtitle)
```

switch

`switch`, bir değeri sırayla case'lerle karşılaştırır ve eşleşen ilk case'i çalıştırır. Swift'teki `switch` C'dekinden çok daha güçlüdür:

- Tüm olası değerler kapsanmalıdır (exhaustive). Kapsanmayan durum varsa kod derlenmez.
- Bir case bittiğinde bir sonrakine geçilmez, `break` yazmaya gerek yoktur.
- Bir case'te birden fazla değer virgülle yazılabilir.
- Range, tuple, enum case'i, tip kontrolü gibi pattern'ler kullanılabilir.

```swift
func category(for statusCode: Int) -> String {
    switch statusCode {
    case 200..<300: "Success"
    case 401, 403: "Authorization"
    case 429: "Rate limited"
    case 400..<500: "Client error"
    case 500...: "Server error"
    default: "Unexpected"
    }
}

print(category(for: 403))    // Authorization
print(category(for: 404))    // Client error
```

Case'ler yukarıdan aşağıya kontrol edildiği için özel durumlar genel durumlardan önce yazılır. `403` case'i `400..<500`'den sonra yazılsaydı hiçbir zaman çalışmazdı.

Tuple ile switch. Birden fazla değere aynı anda bakmak gerektiğinde değerler bir tuple'da toplanır. `_` herhangi bir değerle eşleşir.

```swift
enum Plan {
    case free, pro, team
}

func uploadLimitInMB(isVerified: Bool, plan: Plan) -> Int {
    switch (isVerified, plan) {
    case (false, _): 10
    case (true, .free): 100
    case (true, .pro), (true, .team): 1_000
    }
}
```

Bu fonksiyonda `default` yoktur ve buna gerek de yoktur. `Bool` ve `Plan` değerlerinin tüm kombinasyonları kapsanmıştır.

Value binding ve where. Eşleşen değerin bir kısmı `let` ile isimlendirilip case içinde kullanılabilir. `where` ek bir koşul ekler.

```
let response: (statusCode: Int, retryAfter: Int?) = (429, 30)

switch response {
case (429, let seconds?):
    print("Rate limited, retry in \(seconds)s")
case (let code, _) where code >= 500:
    print("Server error \(code), retry with backoff")
case (let code, _):
    print("Finished with \(code)")
}
```

> **Tercih**
>
> Kendi enum'ların üzerinde `switch` yazarken `default` kullanmaktan kaçınılır. Tüm case'ler açıkça yazılırsa, enum'a yeni bir case eklendiğinde derleyici güncellenmesi gereken her `switch`'i hata olarak gösterir. `default` yazıldığında yeni case sessizce oraya düşer.

`fallthrough` C'deki davranışı bilinçli olarak geri getirir ama gerçek projelerde nadiren gerekir. Aynı işi yapan değerler genellikle tek case'te virgülle birleştirilir.

Control transfer

İfade	Etkisi
`continue`	Döngünün bu turunu bitirir, sonraki tura geçer
`break`	Döngüden ya da `switch`'ten çıkar
`fallthrough`	`switch`'te bir sonraki case'e geçer
`return`	Fonksiyondan çıkar
`throw`	Hata fırlatarak kapsamdan çıkar

İç içe döngülerde `break` sadece en içteki döngüden çıkar. Dıştaki döngüden çıkmak için döngüye etiket verilir.

```
let weeklySlots = [["09:00", "10:00"], ["11:00", "13:00"], ["13:00", "15:00"]]
var firstMatch: (day: Int, slot: String)?

search: for (day, slots) in weeklySlots.enumerated() {
    for slot in slots where slot == "13:00" {
        firstMatch = (day, slot)
        break search
    }
}

print(firstMatch?.day ?? -1)    // 1
```

Early exit

`guard`, bir koşul sağlanmazsa kapsamdan erken çıkmak için kullanılır. Fonksiyonlarda nasıl kullanıldığı The Basics bölümünde anlatıldı. Döngü içinde `guard` ile `continue` birlikte kullanılarak geçersiz elemanlar atlanır.

```
let rawPrices = ["12.5", "abc", "", "40"]
var prices: [Double] = []

for raw in rawPrices {
    guard let price = Double(raw) else { continue }
    prices.append(price)
}
print(prices)    // [12.5, 40.0]
```

defer

`defer` bloğu, içinde bulunduğu kapsamdan nasıl çıkılırsa çıkılsın (`return`, `throw`, `break` ya da kapsamın sonu) çıkış anında çalışır. Temizlik işlemlerini, işi başlatan kodun hemen yanına yazmayı sağlar. Böylece fonksiyona sonradan yeni bir `return` eklense bile temizlik atlanmaz.

```swift
enum FeedError: Error {
    case offline
}

final class FeedLoader {
    private(set) var isLoading = false
    var isOnline = false

    func load() throws -> [String] {
        isLoading = true
        defer { isLoading = false }

        guard isOnline else { throw FeedError.offline }
        return ["Post 1", "Post 2"]
    }
}

let loader = FeedLoader()
do {
    _ = try loader.load()
} catch {
    print(error, loader.isLoading)    // offline false
}
```

Aynı kapsamda birden fazla `defer` varsa ters sırayla çalışırlar: en son yazılan ilk çalışır. Bu sıra, kaynakları açıldıkları sıranın tersine kapatmak için uygundur.

> **Not**
>
> Gerçek projelerdeki tipik `defer` kullanımları: loading durumunu sıfırlamak, bir kilidi bırakmak (`lock.unlock()`), açılan dosyayı kapatmak, `UIApplication.endBackgroundTask` çağırmak, test sonunda geçici dosyayı silmek.

Availability

Bir API sadece belirli işletim sistemi sürümlerinde varsa, kullanmadan önce sürüm kontrolü yapılır. Derleyici bu kontrolü zorunlu tutar.

```swift
func configureAppearance() {
    if #available(iOS 17, macOS 14, *) {
        print("Use the new API")
    } else {
        print("Fall back to the old API")
    }
}

@available(iOS 17, macOS 14, *)
func makeModernLayout() {
    print("Only callable on iOS 17 and later")
}
```

`*` işareti, listede adı geçmeyen platformlarda minimum deployment target'ın geçerli olduğunu belirtir. Ters kontrol için `if #unavailable(iOS 17)` yazılır.

Mülakat soruları

S. Swift'teki switch, C'deki switch'ten nasıl farklıdır?

C. Swift'te `switch` tüm durumları kapsamak zorundadır, aksi halde derlenmez. Case'ler arasında örtük geçiş yoktur, bu yüzden `break` unutmaktan kaynaklanan hatalar oluşmaz. Sadece tam sayılarla değil her tiple çalışır ve range, tuple, enum associated value, tip kontrolü gibi pattern'leri destekler. `let` ile değer bağlanabilir, `where` ile ek koşul eklenebilir.

S. Kendi enum'unun switch'inde neden default kullanmaktan kaçınırsın?

C. Tüm case'leri açıkça yazarsam, enum'a yeni bir case eklendiğinde derleyici o enum'u kullanan her `switch`'i hata olarak gösterir ve hiçbir yer gözden kaçmaz. `default` yazarsam yeni case sessizce varsayılan dala düşer ve hata ancak çalışma anında fark edilir.

S. defer ne zaman çalışır? Birden fazla defer olursa sıra nedir?

C. `defer` bloğu, içinde tanımlandığı kapsamdan çıkılırken çalışır. Çıkışın `return`, `throw` ya da kapsamın sonu olması fark etmez. Birden fazla `defer` varsa ters sırayla çalışırlar, en son tanımlanan ilk çalışır. Tipik kullanımı temizliktir: loading durumunu sıfırlamak, kilidi bırakmak, dosyayı kapatmak.

S. while ile repeat-while arasındaki fark ne, gerçek bir kullanım örneği verir misin?

C. `while` koşulu turun başında kontrol eder, koşul baştan sağlanmazsa gövde hiç çalışmaz. `repeat-while` koşulu sonda kontrol eder, gövde en az bir kez çalışır. Cursor tabanlı sayfalama iyi bir örnektir. İlk sayfa her zaman istenir, sonraki sayfalar ise cevaptaki cursor dolu olduğu sürece istenir.

S. İç içe iki döngüde, içteki döngüden dıştakini nasıl sonlandırırsın?

C. Dıştaki döngüye bir etiket veririm (`search: for ...`) ve içeride `break search` yazarım. Etiketsiz `break` sadece en içteki döngüden çıkar.

6. Functions

Fonksiyonlar Swift'te first class değerlerdir. Bir değişkene atanabilir, parametre olarak verilebilir ve başka bir fonksiyondan döndürülebilir. Swift'in fonksiyon söz dizimi, çağrıldığı yerde bir İngilizce cümle gibi okunacak şekilde tasarlanmıştır.

Argument label ve parameter name

Her parametrenin iki adı vardır. Argument label, fonksiyon çağrılırken yazılır. Parameter name ise fonksiyonun gövdesinde kullanılır. Tek bir ad yazılırsa ikisi aynı olur. Label'ı kaldırmak için `_` kullanılır.

```
struct Account {
    var balance: Decimal
}

func transfer(_ amount: Decimal, from source: inout Account, to destination: inout Account) {
    source.balance -= amount
    destination.balance += amount
}

var checking = Account(balance: 500)
var savings = Account(balance: 100)
transfer(150, from: &checking, to: &savings)
print(checking.balance, savings.balance)    // 350 250
```

Çağrı noktası "transfer 150 from checking to savings" diye okunur. Gövdede ise `source` ve `destination` gibi anlamı daha açık isimler kullanılır.

> **Tercih**
>
> Swift API Design Guidelines'ın temel kuralları:
>
> - Çağrı noktasında okunabilirlik, tanımın kısalığından önemlidir.
> - İlk argüman fonksiyon adıyla birlikte doğal bir ifade oluşturuyorsa label kaldırılır: `print(_:)`, `append(_:)`, `contains(_:)`.
> - Argümanın rolünü bir edat anlatıyorsa label olarak edat kullanılır: `remove(at:)`, `move(from:to:)`, `fetchUser(withID:)`.
> - Yan etkisi olan fonksiyonlar fiil (`sort()`), yeni değer döndürenler isim ya da sıfat fiil (`sorted()`) olarak adlandırılır.

Default parameter values

Varsayılan değerler, aynı fonksiyonun birçok overload'unu yazma ihtiyacını ortadan kaldırır. Varsayılan değerli parametreler genellikle listenin sonuna yazılır.

```
enum SortOrder {
    case newest, mostPopular
}

func fetchArticles(page: Int = 1, pageSize: Int = 20, sort: SortOrder = .newest) ->
String {
    "page=\(page)&size=\(pageSize)&sort=\(sort)"
}

print(fetchArticles())                          // page=1&size=20&sort=newest
print(fetchArticles(page: 3))                   // page=3&size=20&sort=newest
print(fetchArticles(sort: .mostPopular))        // page=1&size=20&sort=mostPopular
```

Variadic parameters

Variadic parametre, aynı tipte sıfır ya da daha fazla değer kabul eder. Fonksiyonun içinde bir dizi olarak görünür.

```
enum LogLevel: String {
    case debug, info, error
}

func log(_ messages: String..., level: LogLevel = .info) {
    print("[\(level.rawValue.uppercased())] " + messages.joined(separator: " "))
}

log("User", "42", "signed in")                  // [INFO] User 42 signed in
log("Token refresh failed", level: .error)      // [ERROR] Token refresh failed
```

In-out parameters

Parametreler varsayılan olarak sabittir, fonksiyon içinde değiştirilemez. Bir parametrenin değişikliğinin çağıran tarafa yansıması isteniyorsa `inout` kullanılır ve çağrılırken başına `&` konur. Yukarıdaki `transfer` fonksiyonu buna örnektir.

`inout` referans geçirme değildir, copy in copy out olarak çalışır. Değer fonksiyona kopyalanır, fonksiyon bitince değiştirilmiş hali orijinal değişkene geri yazılır. Bu yüzden `inout` argüman olarak sadece `var` verilebilir. `let` sabitler ve literal'lar verilemez.

> **Tercih**
>
> `inout` yerine çoğu zaman yeni değer döndürmek daha okunaklıdır. `inout`, standart kütüphanedeki `swap(_:_:)` gibi bir değeri yerinde değiştiren yardımcı fonksiyonlarda ya da büyük bir struct'ı kopyalamadan güncellemek gerektiğinde tercih edilir.

Return values

Gövdesi tek bir ifadeden oluşan fonksiyonlarda `return` yazılmayabilir (implicit return). Birden fazla değer döndürmek için tuple, sonuç olmayabiliyorsa optional dönüş tipi kullanılır.

```
func priceRange(of prices: [Decimal]) -> (min: Decimal, max: Decimal)? {
    guard let low = prices.min(), let high = prices.max() else { return nil }
    return (low, high)
}

if let range = priceRange(of: [49.9, 19.9, 120]) {
    print("\(range.min) – \(range.max)")    // 19.9 – 120
}
```

Dönüş değeri çoğu zaman kullanılmayan fonksiyonlar `@discardableResult` ile işaretlenir. Böylece sonuç kullanılmadığında derleyici uyarı vermez.

```
final class Cache {
    private var storage: [String: Data] = [:]

    @discardableResult
    func removeValue(forKey key: String) -> Data? {
        storage.removeValue(forKey: key)
    }
}

let cache = Cache()
cache.removeValue(forKey: "avatar")
```

Hiçbir zaman normal şekilde geri dönmeyen fonksiyonların dönüş tipi `Never`'dır. `fatalError` bu şekilde tanımlanmıştır. Derleyici `Never` döndüren bir çağrıdan sonra kodun devam etmeyeceğini bilir.

Function types

Her fonksiyonun parametre ve dönüş tiplerinden oluşan bir tipi vardır. `(String) -> String?` "String alıp optional String döndüren fonksiyon" demektir. Fonksiyonlar bu tip üzerinden değişkene atanabilir, parametre olarak verilebilir ve döndürülebilir.

Aşağıdaki örnekte form doğrulama kuralları birer fonksiyon olarak tanımlanıyor. Kurallar bir dizide toplanıyor ve sırayla uygulanıyor.

```
typealias Validator = (String) -> String?

func notEmpty(_ value: String) -> String? {
    value.isEmpty ? "This field is required" : nil
}

func minLength(_ length: Int) -> Validator {
    { value in
        value.count >= length ? nil : "Must be at least \(length) characters"
    }
}

let passwordRules: [Validator] = [notEmpty, minLength(8)]
let errors = passwordRules.compactMap { rule in rule("abc") }
print(errors)    // ["Must be at least 8 characters"]
```

`minLength(8)` bir fonksiyon döndüren bir fonksiyondur. `notEmpty` ise parantezsiz yazılarak fonksiyonun kendisi olarak diziye eklenir.

> **Dikkat**
>
> Fonksiyonu bir değer olarak vermekle çağırmanın sonucunu vermek farklı şeylerdir. `notEmpty` fonksiyonun kendisidir ve tipi `(String) -> String?`'dir. `notEmpty("abc")` ise çağrının sonucudur ve tipi `String?`'dir. `[notEmpty()]` yazmak derlenmez, çünkü fonksiyon argüman olmadan çağrılamaz.

Nested functions

Bir fonksiyon başka bir fonksiyonun içinde tanımlanabilir. İç fonksiyon dışarıdan görünmez ve dış fonksiyonun değişkenlerine erişebilir. Sadece tek bir fonksiyonda kullanılan yardımcı mantığı dışarı sızdırmadan gruplamak için kullanılır.

```swift
func formattedDuration(seconds: Int) -> String {
    func pad(_ value: Int) -> String {
        value < 10 ? "0\(value)" : "\(value)"
    }
    return "\(pad(seconds / 60)):\(pad(seconds % 60))"
}

print(formattedDuration(seconds: 125))    // 02:05
```

Mülakat soruları

S. Argument label ile parameter name arasındaki fark ne, neden ikisi ayrı?

C. Argument label fonksiyon çağrılırken, parameter name fonksiyonun gövdesinde kullanılır. Ayrı olmaları çağrı noktasının bir cümle gibi okunmasını sağlar. Örneğin `move(from: a, to: b)` çağrıda doğal okunur, gövdede ise `source` ve `destination` gibi daha açık isimler kullanılabilir. İlk argüman fonksiyon adıyla zaten anlaşılıyorsa label `_` ile kaldırılır.

S. inout nasıl çalışır? Referans ile geçirmek mi?

C. Hayır, copy in copy out şeklinde çalışır. Değer fonksiyona kopyalanır, fonksiyon bittiğinde değiştirilmiş hali orijinal değişkene yazılır. Bu yüzden sadece `var` verilebilir ve çağrıda `&` yazmak zorunludur. Derleyici bazı durumlarda bunu referans ile geçirme şeklinde optimize edebilir ama anlam olarak copy in copy out'tur. Aynı değişkene aynı anda iki `inout` erişim derleyici tarafından engellenir (bkz. Memory Safety).

S. Default parameter value mu, overload mu?

C. Aynı işi yapan ama bazı parametreleri isteğe bağlı olan fonksiyonlar için varsayılan değer tercih edilir. Tek bir implementasyon olur ve tüm kombinasyonlar desteklenir. Overload, parametre tipleri ya da davranış gerçekten farklıysa anlamlıdır.

S. Swift'te fonksiyonların first class olması ne demek?

C. Fonksiyonlar diğer değerler gibi davranır. Bir değişkene atanabilir, bir koleksiyonda saklanabilir, başka bir fonksiyona parametre olarak verilebilir ve bir fonksiyondan döndürülebilir. `map`, `filter`, `sorted(by:)` gibi API'ler ve completion handler'lar bu özelliğe dayanır.

S. @discardableResult ne işe yarar?

C. Değer döndüren bir fonksiyonun sonucu kullanılmadığında derleyici uyarı verir. Sonucu çoğu zaman kullanılmayan fonksiyonlar `@discardableResult` ile işaretlenerek bu uyarı kaldırılır. Örnekler: silinen değeri de döndüren bir `remove` fonksiyonu ya da zincirleme kullanım için `self` döndüren bir builder metodu.

7. Closures

Closure, bir değişkene atanabilen ya da parametre olarak verilebilen bir kod bloğudur. Fonksiyonlar aslında closure'ın özel bir halidir. Closure'ları güçlü yapan özellik, tanımlandıkları yerdeki değişkenleri yakalayıp (capture) daha sonra çalıştırıldıklarında da kullanabilmeleridir.

Swift'te closure'ın üç biçimi vardır:

- Global fonksiyonlar: isimleri vardır, hiçbir değer yakalamazlar.
- Nested fonksiyonlar: isimleri vardır, dış fonksiyonun değerlerini yakalayabilirler.
- Closure expression'lar: isimsizdir, çevrelerindeki değerleri yakalayabilirler.

Closure expression söz dizimi

Closure'ın tam yazımı süslü parantez içinde parametreler, dönüş tipi ve `in` anahtar kelimesinden sonra gövdeden oluşur. Swift, bağlamdan çıkarabildiği her şeyi yazmamaya izin verir.

```
let usernames = ["zeynep", "ali", "berk"]

let a = usernames.sorted(by: { (lhs: String, rhs: String) -> Bool in
    return lhs < rhs
})
let b = usernames.sorted(by: { lhs, rhs in lhs < rhs })    // tipler ve return çıkarıldı
let c = usernames.sorted { $0 < $1 }                       // shorthand argument names, trailing closure
let d = usernames.sorted(by: <)                            // operatörün kendisi bir fonksiyondur

print(d)    // ["ali", "berk", "zeynep"]
```

`$0`, `$1` kısa yazımları closure tek satır ve anlamı açık olduğunda kullanılır. Gövde uzadığında parametrelere isim vermek okunabilirliği korur.

Trailing closures

Bir fonksiyonun son parametresi closure ise, closure parantezlerin dışına yazılabilir. Closure tek parametre ise parantezler tamamen kaldırılabilir. Birden fazla closure parametresi varsa ilki label'sız, sonrakiler label ile yazılır. SwiftUI'daki `Button { } label: { }` kullanımı bu şekildedir.

```
func performRequest(
    _ path: String,
    onSuccess: (String) -> Void,
    onFailure: (Int) -> Void
) {
    path.hasPrefix("/v1") ? onSuccess("{}") : onFailure(404)
}

performRequest("/v1/me") { body in
    print("Body: \(body)")
} onFailure: { code in
    print("Failed with \(code)")
}
```

Closure'ı çağırmak

Closure bir değerdir. Tanımlamak onu çalıştırmaz. Çalıştırmak için sonuna parantez eklenir, tıpkı bir fonksiyon gibi.

```
let refresh: () -> Void = {
    print("Refreshing feed")
}

let action = refresh    // closure'ın kendisi, hiçbir şey yazdırılmaz
action()                // Refreshing feed
```

> **Dikkat**
>
> `{ }` closure'ı paketler, `()` çalıştırır. Bir property'de saklanan `transform` closure'ını çalıştırmak için `transform(value)` yazılır. Sadece `transform` yazmak closure'ı bir değer olarak verir. Optional bir closure `onChange?(value)` şeklinde çağrılır.

Capturing values

Closure, gövdesinde kullandığı dış değişkenleri yakalar. Yakalama, değişkenin o anki değerini kopyalamak değil, değişkenin kendisine bir referans tutmaktır. Değişken sonradan değişirse closure yeni değeri görür. Closure'ın yakaladığı değişken, tanımlandığı kapsam bitse bile closure yaşadığı sürece yaşar.

```swift
func makeIDGenerator(prefix: String) -> () -> String {
    var next = 0
    return {
        next += 1
        return "\(prefix)-\(next)"
    }
}

let nextOrderID = makeIDGenerator(prefix: "ORD")
print(nextOrderID())    // ORD-1
print(nextOrderID())    // ORD-2

let nextInvoiceID = makeIDGenerator(prefix: "INV")
print(nextInvoiceID())  // INV-1
```

`makeIDGenerator` fonksiyonu bittiğinde `next` değişkeni yok olmaz, çünkü döndürülen closure onu yakalamıştır. Her çağrı kendi `next` değişkenine sahip yeni bir closure üretir.

Değişkenin o anki değerini sabitlemek için capture list kullanılır. Capture list'teki değer, closure oluşturulduğu anda kopyalanır.

```swift
func makeLoggers() -> (live: () -> Void, snapshot: () -> Void) {
    var environment = "staging"
    let live = { print("live: \(environment)") }
    let snapshot = { [environment] in print("snapshot: \(environment)") }
    environment = "production"
    return (live, snapshot)
}

let loggers = makeLoggers()
loggers.live()        // live: production
loggers.snapshot()    // snapshot: staging
```

> **Önemli**
>
> Closure'lar reference type'tır. Bir closure'ı başka bir değişkene atamak yeni bir kopya oluşturmaz, iki değişken aynı closure'ı ve aynı yakalanmış değişkenleri gösterir. Bu özellik, bir class'ın kendi property'sinde sakladığı closure içinde `self`'i yakaladığında retain cycle oluşmasının sebebidir.

Escaping closures

Bir fonksiyona verilen closure, fonksiyon döndükten sonra çağrılacaksa ya da bir yerde saklanacaksa `@escaping` olarak işaretlenmelidir. Completion handler'lar, bir property'de saklanan callback'ler ve asenkron çalışan işler bu duruma örnektir. `@escaping` bir attribute'tur ve parametrenin tipine yazılır.

Closure parametreleri varsayılan olarak non-escaping'dir. Derleyici, non-escaping bir closure'ın fonksiyon bitmeden çalışıp yok olacağını bilir. Bu sayede daha iyi optimizasyon yapar ve closure içinde `self` yazmayı zorunlu tutmaz.

```
struct User {
    let id: Int
    let name: String
}

enum ServiceError: Error {
    case notFound
}

final class UserService {
    private var pendingRequests: [(Result<User, ServiceError>) -> Void] = []

    func fetchUser(id: Int, completion: @escaping (Result<User, ServiceError>) -> Void) {
        pendingRequests.append(completion)    // saklandığı için @escaping zorunlu
    }

    func deliver(_ result: Result<User, ServiceError>) {
        pendingRequests.forEach { $0(result) }
        pendingRequests.removeAll()
    }
}

final class ProfileViewModel {
    private let service: UserService
    private(set) var title = "Loading"

    init(service: UserService) {
        self.service = service
    }

    func load() {
        service.fetchUser(id: 42) { [weak self] result in
            guard let self else { return }
            switch result {
```

```
            case .success(let user): self.title = user.name
            case .failure: self.title = "Something went wrong"
            }
        }
    }
}

let service = UserService()
let viewModel = ProfileViewModel(service: service)
viewModel.load()
service.deliver(.success(User(id: 42, name: "Ada")))
print(viewModel.title)    // Ada
```

> **Önemli**
>
> Escaping closure içinde `self`'in açıkça yazılması zorunludur. Swift bunu bilinçli olarak ister: `self` yazdığın anda closure'ın `self`'i güçlü şekilde tuttuğunu görürsün. Closure `self`'in bir property'sinde saklanıyorsa ya da `self`'ten uzun yaşayabiliyorsa `[weak self]` kullanılır. Ayrıntılar Automatic Reference Counting bölümünde.

> **Not**
>
> Struct ve enum gibi value type'larda escaping closure içinde `self` değiştirilemez. Closure, fonksiyon bittikten sonra çalışabileceği için o anda değiştirilecek bir `self` kopyası artık olmayabilir.

Autoclosures

`@autoclosure`, parametre olarak verilen bir ifadeyi otomatik olarak closure'a sarar. Böylece ifade, çağrıldığı anda değil, fonksiyon gerçekten ihtiyaç duyduğunda değerlendirilir. `??`, `assert` ve `&&` operatörlerinin sağ tarafı bu şekilde çalışır.

```
let isVerboseLoggingEnabled = false

func debugLog(_ message: @autoclosure () -> String) {
    guard isVerboseLoggingEnabled else { return }
    print(message())
}

let payload = Array(repeating: "item", count: 10_000)
debugLog("Payload: \(payload.joined(separator: ","))")    // loglama kapalıyken string hiç oluşturulmaz
```

Higher order functions

Koleksiyonlarla çalışırken en sık kullanılan closure tabanlı API'ler:

Fonksiyon	Ne yapar	Dönüş
`map`	Her elemanı dönüştürür	Aynı uzunlukta yeni dizi
`compactMap`	Dönüştürür, `nil` sonuçları atar	Optional olmayan yeni dizi
`flatMap`	Her eleman için dizi üretir, sonuçları tek dizide birleştirir	Düzleştirilmiş dizi
`filter`	Koşula uyanları tutar	Yeni dizi
`reduce`	Tüm elemanları tek bir değerde toplar	Tek değer
`first(where:)`	Koşula uyan ilk eleman	Optional
`contains(where:)`	Koşula uyan eleman var mı	`Bool`
`allSatisfy`	Tüm elemanlar koşula uyuyor mu	`Bool`
`sorted(by:)`	Sıralanmış yeni dizi	Yeni dizi

```
struct LineItem {
    let sku: String
    let quantity: Int
    let unitPrice: Decimal
}

let cart = [
    LineItem(sku: "KB-01", quantity: 1, unitPrice: 1_200),
    LineItem(sku: "MS-07", quantity: 2, unitPrice: 350),
    LineItem(sku: "CB-11", quantity: 0, unitPrice: 90)
]

let skus = cart.map(\.sku)
let purchasable = cart.filter { $0.quantity > 0 }
let total = purchasable.reduce(0) { sum, item in sum + item.unitPrice *
Decimal(item.quantity) }

print(skus)     // ["KB-01", "MS-07", "CB-11"]
print(total)    // 1900

let couponInputs = ["10", "abc", "25"]
let discounts = couponInputs.compactMap { Int($0) }
print(discounts)    // [10, 25]
```

`reduce(into:)`, sonucun bir koleksiyon olduğu durumlarda her adımda yeni kopya oluşturmadığı için daha verimlidir.

```
let quantitiesBySKU = cart.reduce(into: [String: Int]()) { result, item in
    result[item.sku] = item.quantity
}
print(quantitiesBySKU["MS-07"] ?? 0)    // 2
```

> **Dikkat**
>
> `reduce` closure'ında ilk parametre (`$0`) o ana kadar biriken sonuçtur, ikinci parametre (`$1`) sıradaki elemandır. `$0`'ı eleman sanmak sık yapılan bir hatadır. `reduce` closure'larında parametrelere isim vermek (`sum, item`) bu karışıklığı önler.

> **Tercih**
>
> `forEach` yerine çoğu zaman `for-in` tercih edilir. `forEach` içinde `break` ve `continue` kullanılamaz, `return` ise sadece o turu bitirir. `forEach`, kısa ve yan etki üreten zincirlerin sonunda anlamlıdır.

Mülakat soruları

S. Closure nedir, fonksiyondan farkı ne?

C. Closure, bir değer olarak taşınabilen bir kod bloğudur ve tanımlandığı ortamdaki değişkenleri yakalayabilir. Fonksiyonlar aslında isimli closure'lardır. Global fonksiyonlar hiçbir değer yakalamaz. Nested fonksiyonlar ve closure expression'lar ise çevrelerindeki değişkenleri yakalar ve bu değişkenler, tanımlandıkları kapsam bitse bile closure yaşadığı sürece yaşar.

S. @escaping ne demek, closure'lar neden varsayılan olarak non-escaping?

C. `@escaping`, closure'ın fonksiyon döndükten sonra çağrılabileceğini ya da bir yerde saklanabileceğini belirten bir attribute'tur. Completion handler'lar en tipik örnektir. Varsayılan non-escaping'dir çünkü derleyici, closure'ın fonksiyon içinde çalışıp biteceğini bildiğinde daha iyi optimizasyon yapabilir. Ayrıca non-escaping closure'larda retain cycle riski olmadığı için `self` açıkça yazılmak zorunda değildir.

S. Escaping closure içinde neden self yazmak zorundayız?

C. Escaping closure, `self`'i güçlü şekilde yakalar ve fonksiyon bittikten sonra da yaşayabilir. `self` de o closure'ı saklıyorsa retain cycle oluşur. Swift, `self`'i açıkça yazdırarak geliştiricinin bu yakalamanın farkında olmasını sağlar. Gerekirse `[weak self]` ile güçlü yakalama engellenir.

S. Closure'lar value type mı reference type mı? Bunun pratik sonucu ne?

C. Reference type'tır. Bir closure'ı başka bir değişkene atamak aynı closure'ı ve aynı yakalanmış değişkenleri paylaşmak anlamına gelir. Pratik sonucu şudur: bir class kendi property'sinde bir closure saklar ve o closure `self`'i güçlü şekilde yakalarsa ikisi birbirini tutar ve hiçbiri bellekten silinmez.

S. Capture list ne işe yarar?

C. İki işe yarar. Birincisi, bir değişkenin closure oluşturulduğu andaki değerini kopyalar. Sonradan değişken değişse bile closure eski değeri kullanır. İkincisi, reference type'ların nasıl yakalanacağını belirler. `[weak self]` ve `[unowned self]` retain cycle'ı kırmak için kullanılır.

S. map, compactMap ve flatMap arasındaki fark ne?

C. `map` her elemanı dönüştürür ve aynı uzunlukta bir dizi döndürür. `compactMap` dönüştürme sonucu optional olduğunda `nil` olanları atar ve sadece dolu değerleri döndürür, örneğin `["1", "x"].compactMap { Int($0) }` sonucu `[1]` olur. `flatMap` her eleman için bir dizi üretir ve bunları tek bir diziye birleştirir.

S. @autoclosure nedir, nerede kullanılır?

C. Parametre olarak verilen ifadeyi otomatik olarak bir closure'a sarar. Böylece ifade, fonksiyon gerçekten ihtiyaç duyduğunda değerlendirilir. `??` operatörünün sağ tarafı, `assert`'in mesajı ve `&&`'nin sağ tarafı bu şekilde çalışır. Kendi kodumda pahalı log mesajlarını sadece loglama açıkken oluşturmak için kullanabilirim.

8. Enumerations

Enum, birbiriyle ilişkili bir grup değeri tek bir tip altında toplar. Swift'te enum'lar C'dekinden çok daha güçlüdür: her case'e farklı tipte veri (associated value) iliştirilebilir, metot ve computed property tanımlanabilir, protokollere uyulabilir. Bir değerin sadece belirli durumlardan biri olabildiği her yerde ilk akla gelen araçtır.

Temel kullanım ve CaseIterable

```
enum Environment: CaseIterable {
    case development, staging, production

    var baseURL: String {
        switch self {
        case .development: "http://localhost:8080"
        case .staging: "https://staging.api.example.com"
        case .production: "https://api.example.com"
        }
    }
}

let current = Environment.staging
print(current.baseURL)
print(Environment.allCases.count)    // 3
```

`CaseIterable`, tüm case'leri içeren `allCases` dizisini otomatik üretir. Ayarlar ekranında seçenek listesi göstermek gibi durumlarda kullanılır.

Raw values

Her case'e aynı tipte sabit bir değer verilebilir. Raw value tipi `String`, `Character`, tam sayı ya da ondalıklı sayı olabilir. `String` raw value'da değer verilmezse case'in adı, `Int`'te ise sıfırdan başlayarak artan sayılar kullanılır.

Raw value'dan enum oluşturan `init?(rawValue:)` failable'dır, çünkü verilen değere karşılık gelen bir case olmayabilir.

```swift
enum OrderStatus: String, Codable {
    case pending
    case shipped
    case delivered
    case cancelled = "canceled"
}

let fromAPI = OrderStatus(rawValue: "shipped")    // .shipped
let unknown = OrderStatus(rawValue: "returned")   // nil

let json = #"{"status": "canceled"}"#.data(using: .utf8)!
struct OrderDTO: Codable {
    let status: OrderStatus
}
let order = try JSONDecoder().decode(OrderDTO.self, from: json)
print(order.status)    // cancelled
```

Tercih

API'den gelen string değerleri doğrudan `String` olarak saklamak yerine raw value'lu bir enum'a çevirmek yazım hatalarını ortadan kaldırır ve `switch` ile tüm durumların ele alınmasını sağlar. Case adı Swift isimlendirme kurallarına uyarken (`cancelled`), raw value API'nin beklediği değeri (`canceled`) taşıyabilir.

Dikkat

Sunucu, enum'da olmayan yeni bir değer göndermeye başlarsa decode işlemi hata fırlatır ve tüm cevap çözümlenemez. Kritik durumlarda bilinmeyen değerleri karşılayan bir case eklemek ya da decode'u özelleştirmek gerekir.

Associated values

Associated value, bir case'e o case'e özel veri iliştirir. Her case farklı tipte ve sayıda değer taşıyabilir. Bu özellik bir ekranın durumunu modellemek için idealdir.

```swift
struct Article {
    let title: String
}

enum LoadState {
    case idle
    case loading
    case loaded([Article])
    case failed(message: String, canRetry: Bool)
}

func render(_ state: LoadState) -> String {
    switch state {
    case .idle:
        "Pull to refresh"
    case .loading:
        "Loading"
    case .loaded(let articles) where articles.isEmpty:
        "No articles yet"
    case .loaded(let articles):
        "\(articles.count) articles"
    case .failed(let message, canRetry: true):
        "\(message). Tap to retry"
    case .failed(let message, canRetry: false):
        message
    }
}

print(render(.loaded([Article(title: "Swift 6"), Article(title: "SwiftData")])))   // 2 articles
print(render(.failed(message: "Offline", canRetry: true)))            // Offline. Tap to retry
```

> **Önemli**
>
> Bu yaklaşım, geçersiz durumların temsil edilememesini sağlar. Aynı ekran `isLoading: Bool`, `articles: [Article]`, `error: String?` property'leriyle modellenseydi "hem yükleniyor hem hata var" gibi anlamsız kombinasyonlar mümkün olurdu. Enum ile ekran aynı anda sadece tek bir durumda olabilir.

Sadece tek bir case'i kontrol etmek için `switch` yerine `if case` ya da `guard case` kullanılır.

```
let state = LoadState.failed(message: "Timeout", canRetry: true)

if case .failed(let message, _) = state {
    print("Log: \(message)")
}
```

> **Not**
>
> Bir enum'da raw value ve associated value birlikte kullanılamaz. Raw value derleme anında sabittir. Associated value ise her instance oluşturulduğunda verilir.

Metotlar ve mutating

Enum'lar metot ve computed property içerebilir. Enum'un kendi değerini değiştiren metotlar `mutating` olarak işaretlenir.

```
enum PlaybackState {
    case playing, paused, stopped

    mutating func togglePlayPause() {
        switch self {
        case .playing: self = .paused
        case .paused, .stopped: self = .playing
        }
    }
}

var playback = PlaybackState.stopped
playback.togglePlayPause()
print(playback)    // playing
```

Recursive enumerations

Bir case'in associated value'su enum'un kendi tipini içeriyorsa case `indirect` olarak işaretlenir. Swift bu durumda değeri dolaylı olarak, ayrı bir bellek alanında saklar. İç içe filtreler ya da ifade ağaçları gibi yapılar için kullanılır.

```
struct Product {
    let price: Int
    let tags: Set<String>
}

indirect enum Filter {
    case tag(String)
    case maxPrice(Int)
    case and(Filter, Filter)
    case or(Filter, Filter)

    func matches(_ product: Product) -> Bool {
        switch self {
        case .tag(let tag): product.tags.contains(tag)
        case .maxPrice(let limit): product.price <= limit
        case .and(let lhs, let rhs): lhs.matches(product) && rhs.matches(product)
        case .or(let lhs, let rhs): lhs.matches(product) || rhs.matches(product)
        }
    }
}

let filter = Filter.and(.tag("wireless"), .or(.maxPrice(500), .tag("sale")))
let headphones = Product(price: 900, tags: ["wireless", "sale"])
print(filter.matches(headphones))    // true
```

Case'siz enum ile namespace

Hiç case'i olmayan bir enum'dan instance oluşturulamaz. Bu özellik, sabitleri ve static yardımcıları bir isim altında toplamak için kullanılır. Aynı iş için struct kullanılsaydı `APIConstants()` şeklinde anlamsız bir instance oluşturmak mümkün olurdu.

```
enum APIConstants {
    static let baseURL = URL(string: "https://api.example.com")!
    static let timeout: TimeInterval = 30
    static let maxRetryCount = 3
}
```

Framework enum'ları ve @unknown default

Apple framework'lerindeki bazı enum'lar ileride yeni case'ler alabilir (non-frozen). Bunlar üzerinde `switch` yazarken bilinen case'lerin hepsi yazılır ve sona `@unknown default` eklenir. Yeni bir SDK ile derlendiğinde eklenen case için derleyici uyarı verir, eski bir SDK'daki uygulama ise yeni case

geldiğinde crash olmadan varsayılan dala düşer. `UNAuthorizationStatus` ve `AVAuthorizationStatus` gibi izin durumları tipik örneklerdir.

Mülakat soruları

S. Raw value ile associated value arasındaki fark ne?

C. Raw value her case'e derleme anında verilen sabit bir değerdir ve tüm case'lerde aynı tiptedir. API'den gelen string'leri enum'a çevirmek için kullanılır. Associated value ise instance oluşturulurken verilen ve case'e göre tipi değişebilen veridir. Bir ekranın `loaded([Item])` ya da `failed(Error)` gibi durumlarını modellemek için kullanılır. İkisi aynı enum'da birlikte kullanılamaz.

S. Bir ekranın durumunu neden ayrı Bool ve optional property'ler yerine enum ile modellersin?

C. Ayrı property'lerle "hem yükleniyor hem hata var" ya da "hata var ama veri de dolu" gibi geçersiz kombinasyonlar oluşabilir ve her yerde bunları kontrol etmek gerekir. Associated value'lu bir enum ile ekran aynı anda sadece tek bir durumda bulunabilir ve her durum kendi verisini taşır. `switch` ile tüm durumların ele alınması da derleyici tarafından garanti edilir.

S. @unknown default ne zaman kullanılır?

C. Apple framework'leri gibi ileride yeni case eklenebilecek enum'lar üzerinde `switch` yazarken kullanılır. Bilinen tüm case'ler yazılır, sona `@unknown default` eklenir. Yeni bir case eklendiğinde derleyici uyarı verir, ama uygulama çalışma anında crash olmadan varsayılan dala düşer.

S. indirect ne işe yarar?

C. Kendi tipini associated value olarak içeren recursive enum'larda kullanılır. Normalde enum'un boyutu derleme anında bilinmelidir. Kendini içeren bir case'in boyutu ise sonsuz olurdu. `indirect`, değeri ayrı bir bellek alanında saklayıp enum'da sadece ona referans tutarak bu sorunu çözer.

S. Sabitleri gruplamak için neden struct değil de case'siz enum kullanılır?

C. Case'i olmayan bir enum'dan instance oluşturulamaz, bu yüzden sadece bir namespace olarak kullanılabilir. Struct kullanılsaydı anlamsız bir instance oluşturmak mümkün olurdu. Bunu engellemek için `private init()` yazmak gerekirdi.

9. Structures and Classes

Struct ve class, property ve metotlarla kendi tiplerini tanımlamanın iki yoludur. İkisi de property, metot, subscript ve initializer tanımlayabilir, extension ile genişletilebilir ve protokollere uyabilir. Aralarındaki temel fark, struct'ın value type, class'ın reference type olmasıdır.

Sadece class'ta olanlar

- Inheritance: bir class başka bir class'tan türeyebilir.
- Type casting: bir instance'ın gerçek tipi çalışma anında kontrol edilebilir.
- Deinitializer: instance bellekten silinmeden önce `deinit` çalışır.
- Reference counting: bir instance'a birden fazla referans olabilir ve ARC bunları sayar.
- Identity: iki referansın aynı nesneyi gösterip göstermediği `===` ile kontrol edilebilir.

Value semantics ve reference semantics

Struct bir değişkene atandığında ya da fonksiyona verildiğinde kopyalanır. Kopyadaki değişiklik orijinali etkilemez. Class ise kopyalanmaz. Atanan şey aynı nesneye bir referanstır ve değişiklik o nesneyi gösteren herkese yansır.

```
struct DraftSettings {
    var fontSize: Int
}

final class SharedSettings {
    var fontSize: Int
    init(fontSize: Int) { self.fontSize = fontSize }
}

var draftA = DraftSettings(fontSize: 14)
var draftB = draftA
draftB.fontSize = 18
print(draftA.fontSize, draftB.fontSize)    // 14 18

let sharedA = SharedSettings(fontSize: 14)
let sharedB = sharedA
sharedB.fontSize = 18
print(sharedA.fontSize, sharedB.fontSize)  // 18 18
print(sharedA === sharedB)                 // true
```

Bu fark, uygulamadaki birçok hatanın kaynağıdır. Bir ekranda düzenlenen nesne class ise, kullanıcı "İptal" dediğinde bile değişiklik listedeki orijinal nesneye yansımış olur. Struct ile düzenleme ekranı kendi kopyası üzerinde çalışır ve sadece "Kaydet" dendiğinde değer geri yazılır.

> **Önemli**
>
> `let` struct'ta ve class'ta farklı davranır. `let` ile tanımlanan bir struct'ın hiçbir property'si değiştirilemez, çünkü struct değerin kendisidir. `let` ile tanımlanan bir class instance'ında ise sadece referans sabittir. `var` property'ler değiştirilebilir. Yukarıdaki `sharedB` bu yüzden `let` olmasına rağmen güncellenebiliyor.

Memberwise initializer

Struct'lar, tüm stored property'leri parametre olarak alan bir initializer'ı otomatik olarak alır. Varsayılan değeri olan property'ler bu initializer'da isteğe bağlı parametre olur. Class'larda böyle bir initializer yoktur, `init` elle yazılır.

```
struct APIRequest {
    let path: String
    var method = "GET"
    var headers: [String: String] = [:]
}

let profile = APIRequest(path: "/me")
let update = APIRequest(path: "/me", method: "PATCH", headers: ["Content-Type":
"application/json"])
```

> **Not**
>
> Struct'a kendi `init`'ini yazdığında memberwise initializer kaybolur. İkisini birlikte kullanmak istiyorsan özel `init`'i bir extension içine yazarsın (bkz. Initialization).

Struct mı class mı

Apple'ın önerisi varsayılan olarak struct kullanmaktır. Class'a ancak gerçekten class'a özel bir özellik gerektiğinde geçilir.

Durum	Tercih	Gerekçe
Veri modeli (`User` , `Order` , API DTO'su)	struct	Bağımsız kopyalar, beklenmedik paylaşım yok, otomatik `Equatable` / `Codable` /memberwise init
Kimliği önemli, paylaşılan ve değişen durum (oturum yöneticisi, cache)	class	Herkes aynı nesneyi görmeli
Yaşam döngüsü önemli (`deinit` ile kaynak bırakma)	class	Struct'ta `deinit` yoktur
UIKit'ten türeme (`UIViewController`) ya da Objective-C uyumluluğu	class	Inheritance ve `NSObject` gerekir
SwiftUI'da gözlemlenen model (`@Observable`)	class	Macro sadece class'lara uygulanır
SwiftUI view'ları	struct	Hafif, sık oluşturulup atılan değerler

> **Tercih**
>
> Bir class'tan türetme yapılmayacaksa `final` olarak işaretlenir. `final` , niyeti açıkça belirtir ve derleyicinin metot çağrılarını dinamik dispatch yerine doğrudan çağrıya çevirmesine (static dispatch) izin verir.

Değişikliğin yayılması

Bir struct, başka bir struct'ın ya da dizinin içindeyse ve bir property'si değiştirilirse, değişiklik dıştaki değere de yansır. Çünkü değiştirilen şey zaten dıştaki değerin bir parçasıdır.

```
struct CartItem {
    let sku: String
    var quantity: Int
}

var items = [CartItem(sku: "KB-01", quantity: 1)]
items[0].quantity += 1          // dizideki eleman yerinde güncellenir
print(items[0].quantity)        // 2

var copy = items[0]
copy.quantity = 10              // sadece kopya değişir
print(items[0].quantity)        // 2
```

> **Dikkat**
>
> `for item in items { item.quantity += 1 }` derlenmez, çünkü döngü değişkeni her elemanın sabit bir kopyasıdır. Dizideki struct'ları güncellemek için index üzerinden gidilir (`for index in items.indices { items[index].quantity += 1 }`) ya da `map` ile yeni dizi oluşturulur.

Mülakat soruları

S. Struct ve class arasındaki farklar neler?

C. Struct value type'tır, atandığında ya da fonksiyona verildiğinde kopyalanır. Class reference type'tır ve atandığında aynı nesneye yeni bir referans oluşturulur. Sadece class'larda olan özellikler: inheritance, type casting, `deinit` , ARC ile reference counting ve `===` ile identity kontrolü. Struct'lar otomatik memberwise initializer alır. `let` ile tanımlanan struct tamamen değişmezdir, class'ta ise sadece referans sabittir. Struct'ta değişiklik yapan metotlar `mutating` olarak işaretlenir.

S. Bir model için struct mı class mı seçersin, neden?

C. Varsayılan olarak struct seçerim. Value semantics sayesinde bir değeri değiştirdiğimde başka bir yerin beklenmedik şekilde etkilenmeyeceğini bilirim. Bu, özellikle birden fazla thread'in aynı veriyle çalıştığı durumlarda büyük avantajdır. Class'ı şu durumlarda seçerim: nesnenin kimliği önemliyse ve birden fazla yer aynı değişen durumu görmeliyse (oturum yöneticisi gibi), `deinit` ile yaşam döngüsünü yönetmem gerekiyorsa, UIKit'ten türetme ya da Objective-C uyumluluğu gerekiyorsa ya da SwiftUI'da `@Observable` kullanacaksam.

S. Struct her zaman stack'te, class her zaman heap'te mi tutulur?

C. Bu yaygın bir basitleştirmedir ama tam doğru değildir. Class instance'ları heap'te tutulur. Struct'lar ise genellikle stack'te tutulur ama bir closure tarafından yakalandığında, `any` ile bir existential kutusuna konduğunda ya da `Array` gibi heap'te depolama kullanan bir tipin içinde olduğunda heap'e taşınabilir. Asıl fark bellekte nerede durduklarından çok semantiktedir: value semantics ile reference semantics.

S. == ile === arasındaki fark ne?

C. `==` iki değerin eşit olup olmadığını kontrol eder ve tipin `Equatable` 'a uymasını gerektirir. Eşitliğin ne anlama geldiğini tip kendisi tanımlar. `===` sadece class instance'larında çalışır ve iki referansın bellekteki aynı nesneyi gösterip göstermediğini kontrol eder.

S. final anahtar kelimesi ne işe yarar?

C. Bir class'ın alt sınıfının oluşturulmasını ya da bir metodun override edilmesini engeller. Niyeti açıkça belirtir. Ayrıca derleyici, override edilemeyeceğini bildiği metotları dinamik dispatch yerine doğrudan çağırabilir, bu da küçük bir performans kazancı sağlar.

S. SwiftUI view'ları neden struct?

C. SwiftUI'da view'lar, ekranın o anki durumunun hafif bir tarifidir. Durum her değiştiğinde yeniden oluşturulurlar. Struct'lar ucuzdur, inheritance yükü taşımaz ve value semantics sayesinde SwiftUI eski ve yeni tarifi karşılaştırıp sadece değişen kısmı güncelleyebilir. Değişen durum ise view'da değil, `@State` ya da `@Observable` bir model gibi SwiftUI'ın yönettiği depolama alanında tutulur.

10. Properties

Property, bir değeri bir tipe bağlar. Stored property değeri instance'ın içinde saklar. Computed property ise değeri saklamaz, her erişildiğinde hesaplar. Bu ikisinin üzerine property observer'lar, lazy property'ler, property wrapper'lar ve type property'ler gelir.

Stored properties

Stored property'ler `let` ya da `var` ile tanımlanır. Struct'larda, instance `let` ile tanımlandıysa `var` property'ler bile değiştirilemez.

```
struct Coordinate {
    var latitude: Double
    var longitude: Double
}

let office = Coordinate(latitude: 41.04, longitude: 29.0)
// office.latitude = 40.0    error: cannot assign to property: 'office' is a 'let'
constant
```

Lazy stored properties

`lazy` property'nin değeri ilk erişildiğinde hesaplanır. Oluşturulması pahalı olan ya da her instance'ta gerekmeyen değerler için kullanılır. `lazy` sadece `var` olabilir, çünkü değeri init tamamlandıktan sonra atanır.

```swift
final class ReportExporter {
    lazy var formatter: DateFormatter = {
        print("Creating formatter")
        let formatter = DateFormatter()
        formatter.dateFormat = "yyyy-MM-dd"
        return formatter
    }()

    func filename(for date: Date) -> String {
        "report-\(formatter.string(from: date)).csv"
    }
}

let exporter = ReportExporter()
print("Exporter ready")
print(exporter.filename(for: Date(timeIntervalSince1970: 0)))
// Exporter ready
// Creating formatter
// report-1970-01-01.csv
```

Sondaki `()` önemlidir. Closure'ın kendisi değil, closure'ın çalıştırılmasıyla üretilen değer property'ye atanır. Lazy property'nin closure'ı içinde `self`'e erişilebilir, çünkü ilk erişim anında instance zaten tamamen oluşturulmuş olur.

> **Dikkat**
>
> `lazy` property thread safe değildir. Birden fazla thread ilk erişimi aynı anda yaparsa değer birden fazla kez oluşturulabilir.

Computed properties

Computed property bir değer saklamaz. Başka property'lerden türetilen değeri her erişimde hesaplar. Sadece getter'ı olan computed property read-only'dir ve `get` anahtar kelimesi yazılmadan tanımlanabilir.

```
struct Subscription {
    var monthlyPrice: Decimal
    var discountRate: Decimal

    var yearlyPrice: Decimal {
        monthlyPrice * 12 * (1 - discountRate)
    }
}

let plan = Subscription(monthlyPrice: 100, discountRate: 0.2)
print(plan.yearlyPrice)    // 960
```

Setter'ı olan computed property, kendisine atanan değeri başka bir stored property'ye çevirerek yazar. Setter içinde atanan değer `newValue` adıyla gelir.

```
struct TimeoutSetting {
    var milliseconds: Int

    var seconds: Double {
        get { Double(milliseconds) / 1000 }
        set { milliseconds = Int(newValue * 1000) }
    }
}

var timeout = TimeoutSetting(milliseconds: 1500)
timeout.seconds = 2.5
print(timeout.milliseconds)    // 2500
```

Önemli

Birbirinden türetilebilen iki değer varsa sadece biri stored olur, diğeri computed yapılır. Bunun asıl sebebi bellek değil, single source of truth'tur. İkisi de stored olsaydı biri güncellenip diğeri unutulduğunda `milliseconds = 1500` iken `seconds = 2.0` gibi tutarsız bir durum oluşabilirdi. Tek bir kaynak olduğunda bu tutarsızlık imkânsız hale gelir.

Dikkat

Computed property bir property'dir, metot değildir. `var summary() -> String` şeklinde bir tanım geçersizdir. Doğru yazım `var summary: String { ... }` şeklindedir ve kullanılırken parantez yazılmaz: `plan.summary`. Hesaplama pahalıysa, parametre alıyorsa ya da yan etkisi varsa computed property yerine metot tercih edilir.

Property observers

`willSet` ve `didSet`, bir stored property'ye değer atandığında çalışır. `willSet` atama öncesinde çalışır ve yeni değere `newValue` ile erişir. `didSet` atama sonrasında çalışır ve eski değere `oldValue` ile erişir. Değer aynı olsa bile atama yapıldığı için observer'lar çalışır.

```
final class UploadProgress {
    var onMilestone: ((Int) -> Void)?

    var percent: Int = 0 {
        didSet {
            percent = min(max(percent, 0), 100)
            let previousMilestone = oldValue / 25
            let currentMilestone = percent / 25
            if currentMilestone > previousMilestone {
                onMilestone?(currentMilestone * 25)
            }
        }
    }
}

let progress = UploadProgress()
progress.onMilestone = { print("Reached \($0)%") }
progress.percent = 30     // Reached 25%
progress.percent = 60     // Reached 50%
progress.percent = 140    // Reached 100%
print(progress.percent)   // 100
```

`didSet` içinde property'nin kendisine tekrar değer atamak observer'ı yeniden tetiklemez. Bu yüzden yukarıdaki sınırlama işlemi sonsuz döngüye girmez.

> **Dikkat**
>
> `didSet` içinde yeni değer `newValue` ile değil, property'nin kendi adıyla okunur. `newValue` sadece `willSet` içinde vardır, `oldValue` ise sadece `didSet` içinde. Bir eşiğin geçildiğini anlamak için `oldValue` ile yeni değer birlikte karşılaştırılır.

> **Not**
>
> Observer'lar initializer içindeki ilk atamada çalışmaz. Sadece instance oluşturulduktan sonraki atamalarda tetiklenir.

Property wrappers

Property wrapper, bir property'nin nasıl saklanacağını ya da hangi kurallara uyacağını tanımlayan mantığı tekrar kullanılabilir hale getirir. Tip `@propertyWrapper` ile işaretlenir ve `wrappedValue` adında bir property içermek zorundadır. Başlangıç değeri verilebilmesi için `init(wrappedValue:)` yazılır.

```
@propertyWrapper
struct Clamped {
    private var value: Int
    private let range: ClosedRange<Int>

    init(wrappedValue: Int, _ range: ClosedRange<Int>) {
        self.range = range
        self.value = min(max(wrappedValue, range.lowerBound), range.upperBound)
    }

    var wrappedValue: Int {
        get { value }
        set { value = min(max(newValue, range.lowerBound), range.upperBound) }
    }
}

struct VideoSettings {
    @Clamped(0...100) var volume = 50
    @Clamped(1...4) var playbackSpeed = 1
}

var settings = VideoSettings()
settings.volume = 150
print(settings.volume)    // 100
```

Bir wrapper `projectedValue` tanımlarsa, bu değere property adının başına `$` eklenerek erişilir. SwiftUI'da `@State var isOn` için `$isOn` yazıldığında elde edilen `Binding` bir projected value'dur.

> **Not**
>
> SwiftUI'daki `@State`, `@Binding`, `@Environment`, `@AppStorage` ve `@Query` property wrapper'dır. `@AppStorage("theme") var theme = "system"` gibi bir tanım, değeri `UserDefaults`'ta saklayan ve değiştiğinde view'ı güncelleyen bir wrapper kullanır. Kendi wrapper'ını yazmayı bilmek, bu araçların perde arkasını anlamayı sağlar.

Type properties

`static` ile tanımlanan property'ler instance'a değil tipin kendisine aittir. Tüm instance'lar için tek bir değer vardır. Stored type property'ler lazy olarak, ilk erişimde ve thread safe şekilde başlatılır.

```
@MainActor
struct FeatureFlags {
    static let defaultFlags: Set<String> = ["newOnboarding"]
    static var overrides: [String: Bool] = [:]

    static func isEnabled(_ flag: String) -> Bool {
        overrides[flag] ?? defaultFlags.contains(flag)
    }
}

print(FeatureFlags.isEnabled("newOnboarding"))    // true
```

Class'larda computed type property'ler `static` yerine `class` ile tanımlanırsa alt sınıflar tarafından override edilebilir.

> **Dikkat**
>
> Swift 6 dil modunda değişebilen bir `static var`, her thread'den erişilebilen paylaşılmış bir durum olduğu için derleyici hata verir: `static property 'overrides' is not concurrency-safe`. Çözüm, durumu tek bir actor'a bağlamak ya da değeri `let` yapmaktır. Yukarıdaki örnekte tip `@MainActor` ile işaretlendi, böylece `overrides`'a sadece main thread'den erişilebilir (bkz. Concurrency).

Mülakat soruları

S. Stored property ile computed property arasındaki fark ne, birbirinden türetilebilen iki değerde hangisini seçersin?

C. Stored property değeri instance'ın içinde saklar. Computed property ise hiçbir şey saklamaz, her erişimde değeri başka property'lerden hesaplar. Birbirinden türetilebilen iki değerde birini stored, diğerini computed yaparım. Sebebi single source of truth'tur. Tek bir stored değer olduğunda ikisinin birbiriyle tutarsız hale gelmesi imkânsız olur.

S. lazy property nedir, neden let olamaz?

C. Değeri ilk erişildiğinde hesaplanan stored property'dir. Oluşturulması pahalı ya da her zaman gerekmeyen değerler için kullanılır. `let` olamaz çünkü Swift'te `let` property'lerin initialization tamamlanmadan önce değer almış olması gerekir. Lazy property'nin değeri ise initialization'dan sonra, ilk erişimde atanır.

S. willSet ve didSet ne zaman çalışır, init içinde çalışırlar mı?

C. `willSet` değer atanmadan hemen önce çalışır ve yeni değere `newValue` ile erişir. `didSet` atamadan hemen sonra çalışır ve eski değere `oldValue` ile erişir. Initializer içindeki ilk atamalarda çalışmazlar, sadece instance oluşturulduktan sonraki atamalarda tetiklenirler.

S. Property wrapper nedir? SwiftUI'daki @State ile bağlantısı ne?

C. Property'nin nasıl saklanacağını ya da değerine hangi kuralların uygulanacağını tanımlayan mantığı tekrar kullanılabilir bir tipe taşır. Tip `@propertyWrapper` ile işaretlenir ve `wrappedValue` sağlar. `@State` da bir property wrapper'dır. Değeri view struct'ının içinde değil SwiftUI'ın yönettiği bir depolama alanında tutar, değer değiştiğinde view'ın yeniden çizilmesini tetikler. `$isOn` yazıldığında elde edilen `Binding`, wrapper'ın `projectedValue`'sudur.

S. static ile class anahtar kelimeleri type property ve metotlarda nasıl farklıdır?

C. İkisi de üyeyi instance'a değil tipe bağlar. `static` ile tanımlanan üye alt sınıflarda override edilemez. `class` ile tanımlanan üye ise override edilebilir. `class` sadece class'larda ve sadece computed property'ler ile metotlarda kullanılabilir.

11. Methods

Metot, belirli bir tipe bağlı fonksiyondur. Instance metotları bir instance üzerinde, type metotları tipin kendisi üzerinde çağrılır. Swift'te struct ve enum'lar da metot tanımlayabilir. Bu, Objective-C'den önemli bir farktır.

Instance methods ve self

Her instance'ın, instance'ın kendisini gösteren örtük bir `self` property'si vardır. Çoğu zaman yazılması gerekmez. Parametre adı bir property ile aynı olduğunda ayırt etmek için yazılır.

```
struct Pagination {
    var page = 1
    var pageSize = 20

    func offset() -> Int {
        (page - 1) * pageSize
    }

    func with(pageSize: Int) -> Pagination {
        var copy = self
        copy.pageSize = pageSize
        return copy
    }
}

let pagination = Pagination(page: 3)
print(pagination.offset())                      // 40
print(pagination.with(pageSize: 50).offset())   // 100
```

mutating

Value type'larda instance metotları varsayılan olarak property'leri değiştiremez. Değişiklik yapacak metot `mutating` olarak işaretlenir. `mutating` metot `self`'e tamamen yeni bir değer de atayabilir.

```
struct ShoppingCart {
    private(set) var items: [String: Int] = [:]

    mutating func add(_ sku: String, quantity: Int = 1) {
        items[sku, default: 0] += quantity
    }

    mutating func clear() {
        self = ShoppingCart()
    }
}

var cart = ShoppingCart()
cart.add("KB-01")
cart.add("KB-01", quantity: 2)
print(cart.items["KB-01"] ?? 0)    // 3
cart.clear()
print(cart.items.isEmpty)          // true
```

> **Dikkat**
>
> `mutating` bir metot `let` ile tanımlanmış bir struct üzerinde çağrılamaz. `let cart = ShoppingCart()` dedikten sonra `cart.add("KB-01")` yazmak derlenmez. Instance'ın değişmesi gerekiyorsa `var` ile tanımlanır.

> **Not**
>
> Class'larda `mutating` yoktur. Class metotları property'leri her zaman değiştirebilir, çünkü değiştirilen şey referansın gösterdiği nesnedir, referansın kendisi değil.

Type methods

`static func` ile tanımlanan metotlar tipin kendisi üzerinde çağrılır. Factory metotları, ortak yardımcılar ve önceden hazırlanmış örnekler için kullanılır. Type metodunun içinde `self`, instance'ı değil tipin kendisini gösterir.

```swift
struct Money {
    let amount: Decimal
    let currency: String

    static func zero(in currency: String) -> Money {
        Money(amount: 0, currency: currency)
    }

    static func parse(_ text: String) -> Money? {
        let parts = text.split(separator: " ")
        guard parts.count == 2, let amount = Decimal(string: String(parts[0])) else {
return nil }
        return Money(amount: amount, currency: String(parts[1]))
    }
}

print(Money.parse("49.90 TRY")?.amount ?? 0)    // 49.9
print(Money.zero(in: "EUR").currency)           // EUR
```

Mülakat soruları

S. mutating anahtar kelimesi neden var, class'larda neden yok?

C. Struct ve enum value type'tır ve instance'ın property'leri değerin kendisinin bir parçasıdır. Bir property'yi değiştirmek değerin kendisini değiştirmek demektir. Swift bunun açıkça belirtilmesini ister ve `let` ile tanımlanmış bir instance üzerinde `mutating` metot çağrılmasını engeller. Class'ta değiştirilen şey referansın gösterdiği nesnedir, referansın kendisi değişmez. Bu yüzden `mutating`'e gerek yoktur.

S. Bir mutating metot içinde self'e yeni değer atamak mümkün mü?

C. Evet. `mutating` metot içinde `self = ShoppingCart()` gibi bir atama yapılabilir ve instance tamamen yeni bir değerle değiştirilir. Enum'larda bu yöntem durum geçişlerini yazmak için sık kullanılır, örneğin `self = .paused`.

12. Subscripts

Subscript, bir tipin elemanlarına köşeli parantez ile erişilmesini sağlar. `array[0]` ve `dictionary["key"]` subscript kullanır. Kendi tiplerinde de subscript tanımlanabilir. Bir subscript birden fazla parametre alabilir, read-only olabilir ve farklı parametre tipleriyle overload edilebilir.

```
struct HTTPHeaders {
    private var storage: [String: String] = [:]

    subscript(name: String) -> String? {
        get { storage[name.lowercased()] }
        set { storage[name.lowercased()] = newValue }
    }
}

var headers = HTTPHeaders()
headers["Content-Type"] = "application/json"
print(headers["content-type"] ?? "missing")    // application/json
```

Bu örnekte subscript, başlık isimlerini büyük küçük harfe duyarsız hale getiriyor. Dışarıdan bakıldığında sıradan bir sözlük gibi kullanılıyor ama kuralı tip kendisi uyguluyor.

> **Dikkat**
>
> Subscript parametreleri, fonksiyonlardan farklı olarak varsayılan olarak argument label almaz. Yukarıdaki subscript `headers[name: "..."]` şeklinde değil `headers["..."]` şeklinde çağrılır. Label istenirse parametreye açıkça iki isim verilir: `subscript(row row: Int) -> ...`.

Birden fazla parametreli subscript, iki boyutlu yapılar için kullanılır.

```
struct SeatMap {
    let rows: Int
    let columns: Int
    private var reserved: [Bool]

    init(rows: Int, columns: Int) {
        self.rows = rows
        self.columns = columns
        reserved = Array(repeating: false, count: rows * columns)
    }

    subscript(row: Int, column: Int) -> Bool {
        get {
            precondition(row < rows && column < columns, "Seat out of range")
            return reserved[row * columns + column]
        }
        set {
            precondition(row < rows && column < columns, "Seat out of range")
            reserved[row * columns + column] = newValue
        }
    }
}

var hall = SeatMap(rows: 10, columns: 12)
hall[2, 5] = true
print(hall[2, 5])    // true
```

Dizilerde index dışına çıkmak crash'e yol açar. Güvenli erişim için yaygın bir yöntem, label'lı bir subscript'i extension ile eklemektir.

```
extension Collection {
    subscript(safe index: Index) -> Element? {
        indices.contains(index) ? self[index] : nil
    }
}

let tabs = ["Home", "Search", "Profile"]
print(tabs[safe: 1] ?? "none")    // Search
print(tabs[safe: 7] ?? "none")    // none
```

`static subscript` ile tipin kendisi üzerinde çalışan subscript'ler de tanımlanabilir.

Mülakat soruları

S. Subscript nedir, kendi tipinde ne zaman tanımlarsın?

C. Bir tipin elemanlarına köşeli parantez ile erişmeyi sağlayan özel bir üyedir. Tip doğal olarak bir koleksiyon ya da tablo gibi davranıyorsa tanımlarım. Örneğin büyük küçük harfe duyarsız bir header kabı ya da satır ve sütunla erişilen bir koltuk haritası. Erişim kuralını (normalize etme, sınır kontrolü) tipin içine gömmüş olurum.

S. Diziye güvenli index erişimi nasıl eklersin?

C. `Collection`'a extension ile `subscript(safe index: Index) -> Element?` eklerim. Index `indices` içindeyse elemanı, değilse `nil` döndürür. Label kullanmak hem normal subscript ile çakışmayı önler hem de çağrı noktasında güvenli erişim yapıldığını açıkça gösterir.

13. Inheritance

Bir class, başka bir class'ın property, metot ve subscript'lerini miras alabilir. Miras veren class superclass, alan class subclass olarak adlandırılır. Inheritance sadece class'larda vardır. Swift'te çoklu inheritance yoktur. Bir class sadece tek bir class'tan türeyebilir ama birden fazla protokole uyabilir.

Subclassing ve override

Subclass, miras aldığı bir üyeyi yeniden tanımlamak için `override` yazmak zorundadır. Bu zorunluluk, yanlışlıkla aynı isimde bir metot tanımlayıp üst sınıfınkini ezmeyi engeller. Üst sınıfın implementasyonuna `super` ile erişilir.

```swift
class APIClient {
    let baseURL: String

    init(baseURL: String) {
        self.baseURL = baseURL
    }

    func headers() -> [String: String] {
        ["Accept": "application/json"]
    }

    func describe() -> String {
        "Client for \(baseURL) with \(headers().count) headers"
    }
}

final class AuthenticatedAPIClient: APIClient {
    let token: String

    init(baseURL: String, token: String) {
        self.token = token
        super.init(baseURL: baseURL)
    }

    override func headers() -> [String: String] {
        var headers = super.headers()
        headers["Authorization"] = "Bearer \(token)"
        return headers
    }
}

let client: APIClient = AuthenticatedAPIClient(baseURL: "https://api.example.com", token:
"abc")
print(client.describe())    // Client for https://api.example.com with 2 headers
```

`client` değişkeninin tipi `APIClient` olmasına rağmen `describe()` içindeki `headers()` çağrısı subclass'ın implementasyonunu çalıştırır. Hangi implementasyonun çalışacağı çalışma anında, nesnenin gerçek tipine göre belirlenir (dynamic dispatch).

Property override

Subclass, miras aldığı bir property'ye getter ve setter ekleyebilir ya da property observer ekleyebilir. Stored property'nin kendisi override edilemez, ama computed bir property ile ezilebilir ya da üzerine `didSet` eklenebilir.

```
class Control {
    var isEnabled = true
}

final class SubmitButton: Control {
    private(set) var renderCount = 0

    override var isEnabled: Bool {
        didSet { renderCount += 1 }
    }
}

let button = SubmitButton()
button.isEnabled = false
print(button.renderCount)    // 1
```

final

`final` olarak işaretlenen class'tan türetilemez. `final` olarak işaretlenen metot, property ya da subscript override edilemez. Türetme için tasarlanmamış her class'ın `final` olarak işaretlenmesi iyi bir alışkanlıktır.

> **Tercih**
>
> Inheritance mı, protokol mü? Swift'te kod paylaşımı ve soyutlama için ilk tercih protokol ve protocol extension'dır. Struct'lar da kullanabilir, bir tip birden fazla protokole uyabilir ve sıkı bir hiyerarşi oluşmaz. Inheritance genellikle UIKit gibi class tabanlı framework'lerle çalışırken (`UIViewController`, `UIView` subclass'ları) ya da ortak durumu ve kimliği olan gerçek bir "is a" ilişkisi olduğunda kullanılır.

Mülakat soruları

S. Swift'te override neden zorunlu bir anahtar kelime?

C. Yanlışlıkla üst sınıftaki bir metotla aynı imzada bir metot yazıp onu ezmeyi engellemek için. Ayrıca `override` yazılıp üst sınıfta eşleşen bir üye yoksa derleyici hata verir. Böylece üst sınıftaki metodun adı değiştiğinde alt sınıftaki override sessizce bağımsız bir metoda dönüşmez.

S. Swift çoklu inheritance destekler mi? Desteklemiyorsa ne kullanırsın?

C. Class'lar için desteklemez, bir class sadece bir class'tan türeyebilir. Ancak bir tip birden fazla protokole uyabilir ve protocol extension'lar varsayılan implementasyon sağlayabilir. Bu yaklaşım çoklu inheritance'ın faydalarını, karmaşık hiyerarşi ve elmas problemi gibi sorunlar olmadan sağlar.

S. Dynamic dispatch nedir?

C. Çağrılacak metodun derleme anında değil, çalışma anında nesnenin gerçek tipine bakılarak seçilmesidir. Bir `APIClient` değişkeni aslında bir `AuthenticatedAPIClient` gösteriyorsa, override edilmiş metot çalışır. Class'larda bu işlem bir metot tablosu (vtable) üzerinden yapılır. `final` ya da struct kullanıldığında derleyici hangi metodun çalışacağını önceden bildiği için doğrudan çağrı (static dispatch) yapabilir.

14. Initialization

Initialization, bir instance'ı kullanıma hazırlama sürecidir. Swift'in temel kuralı şudur: initializer bittiğinde her stored property'nin bir değeri olmalıdır. Optional property'ler otomatik olarak `nil` ile başlar. Diğer tüm property'lere ya tanımda varsayılan değer verilir ya da initializer içinde değer atanır.

Varsayılan değerler ve özel initializer'lar

Bir property her zaman aynı başlangıç değerini alıyorsa bu değer tanımda verilir. Initializer'da sadece dışarıdan gelmesi gereken değerler atanır.

```
struct RetryPolicy {
    let maxAttempts: Int
    let baseDelay: TimeInterval
    var attemptsMade = 0

    init(maxAttempts: Int = 3, baseDelay: TimeInterval = 0.5) {
        self.maxAttempts = maxAttempts
        self.baseDelay = baseDelay
    }
}
```

`let` property'lere initializer içinde değer atanabilir. Initialization bittikten sonra değiştirilemezler.

Struct'larda memberwise initializer

Struct'lar, stored property'lerin hepsini parametre olarak alan memberwise initializer'ı otomatik alır. Struct'ın içinde özel bir `init` yazıldığında bu initializer kaybolur. İkisini birlikte tutmak için özel `init` bir extension içine yazılır.

```swift
struct Location {
    let latitude: Double
    let longitude: Double
}

extension Location {
    init?(text: String) {
        let parts = text.split(separator: ",").compactMap {
Double($0.trimmingCharacters(in: .whitespaces)) }
        guard parts.count == 2 else { return nil }
        self.init(latitude: parts[0], longitude: parts[1])
    }
}

let a = Location(latitude: 41.0, longitude: 29.0)    // memberwise init hâlâ
kullanılabilir
let b = Location(text: "41.0, 29.0")
print(b?.latitude ?? 0)    // 41.0
```

Initializer delegation

Bir initializer işin bir kısmını aynı tipin başka bir initializer'ına devredebilir. Value type'larda bu `self.init(...)` ile yapılır. Yukarıdaki `init?(text:)` memberwise initializer'a delegate ediyor.

Failable initializers

Geçersiz girdiyle instance oluşturmanın anlamsız olduğu durumlarda `init?` kullanılır. Başarısız olursa `return nil` yapılır ve sonuç optional olur. Raw value'lu enum'ların `init?(rawValue:)`'su ve `Int("abc")` failable initializer örnekleridir.

```swift
struct EmailAddress {
    let value: String

    init?(_ raw: String) {
        let trimmed = raw.trimmingCharacters(in: .whitespacesAndNewlines).lowercased()
        guard trimmed.contains("@"), !trimmed.hasPrefix("@"), !trimmed.hasSuffix("@")
else {
            return nil
        }
        value = trimmed
    }
}

print(EmailAddress("  Ada@Example.com ")?.value ?? "invalid")    // ada@example.com
print(EmailAddress("not-an-email")?.value ?? "invalid")          // invalid
```

> **Tercih**
>
> Failable init mi, throwing init mi? Başarısızlığın tek bir anlamı varsa ("geçersiz girdi") `init?` yeterlidir. Çağıranın başarısızlığın nedenini bilmesi gerekiyorsa (kullanıcıya hangi kuralın ihlal edildiğini göstermek gibi) `init(...) throws` kullanılır.

Bu tip, geçerli bir e-posta adresi taşıdığını tipin kendisiyle garanti eder. Uygulamanın geri kalanında `String` yerine `EmailAddress` kullanıldığında doğrulamanın tekrar tekrar yapılmasına gerek kalmaz.

Class'larda designated ve convenience initializer'lar

Class'larda inheritance olduğu için initialization daha katıdır. İki tür initializer vardır:

- **Designated initializer**: class'ın ana initializer'ıdır. Class'ın kendi property'lerinin hepsini başlatır, sonra superclass'ın bir designated initializer'ını çağırır.
- **Convenience initializer**: `convenience` ile işaretlenir. Mutlaka aynı class'ın başka bir initializer'ını çağırır. Sık kullanılan bir yapılandırmayı kısaltmak için kullanılır.

Kural üç cümleyle özetlenir. Designated initializer superclass'ın designated initializer'ını çağırır. Convenience initializer aynı class'ın başka bir initializer'ını çağırır. Convenience initializer'ın zinciri sonunda mutlaka bir designated initializer'a ulaşır.

```swift
class NetworkSession {
    let timeout: TimeInterval
    let maxConnections: Int

    init(timeout: TimeInterval, maxConnections: Int) {
        self.timeout = timeout
        self.maxConnections = maxConnections
    }

    convenience init() {
        self.init(timeout: 30, maxConnections: 6)
    }
}

final class UploadSession: NetworkSession {
    let chunkSize: Int

    init(chunkSize: Int) {
        self.chunkSize = chunkSize                          // 1. aşama: önce kendi property'leri
        super.init(timeout: 120, maxConnections: 2)         // sonra superclass
        print("Upload session ready: \(describe())")        // 2. aşama: artık self kullanılabilir
    }

    func describe() -> String {
        "\(chunkSize) bytes per chunk, timeout \(timeout)s"
    }
}

let upload = UploadSession(chunkSize: 512_000)
// Upload session ready: 512000 bytes per chunk, timeout 120.0s
```

Two phase initialization

Class initialization iki aşamada gerçekleşir. Birinci aşamada her class, hiyerarşide aşağıdan yukarıya doğru kendi stored property'lerine değer atar. Bu aşama bitmeden `self` üzerinde metot çağrılamaz ve property okunamaz. Tüm property'ler değer aldıktan sonra ikinci aşama başlar. Bu aşamada her class, yukarıdan aşağıya doğru instance'ı özelleştirebilir, metot çağırabilir ve `self`'i kullanabilir.

> **Önemli**
>
> Subclass initializer'ında kendi property'lerine `super.init` çağrısından önce değer atanır. Objective-C'de ve birçok dilde sıra terstir. Swift bu sırayı, superclass'ın initializer'ı override edilmiş bir metodu çağırdığında subclass'ın henüz başlatılmamış bir property'ye erişmesini engellemek için zorunlu tutar.

required initializers

`required` ile işaretlenen initializer'ı tüm subclass'lar implemente etmek zorundadır. UIKit'te `UIView` ya da `UIViewController` subclass'ı kendi designated initializer'ını tanımladığında `required init?(coder:)` yazmak bu yüzden zorunludur. Storyboard kullanılmıyorsa bu metot genellikle `fatalError("init(coder:) has not been implemented")` ile doldurulur.

Varsayılan değeri closure ile vermek

Bir property'nin başlangıç değeri birkaç adımda hazırlanıyorsa, değer bir closure'ın çalıştırılmasıyla verilebilir. Closure'ın sonundaki `()` onu hemen çalıştırır. Bu closure instance henüz hazır olmadan çalıştığı için içinde `self` kullanılamaz.

```
struct CheckoutScreen {
    let currencyFormatter: NumberFormatter = {
        let formatter = NumberFormatter()
        formatter.numberStyle = .currency
        formatter.currencyCode = "TRY"
        return formatter
    }()
}
```

Mülakat soruları

S. Designated ve convenience initializer arasındaki fark ne?

C. Designated initializer class'ın ana initializer'ıdır. Class'ın tüm stored property'lerini başlatır ve superclass'ın designated initializer'ını çağırır. Convenience initializer ise ikincil bir kısayoldur ve mutlaka aynı class'ın başka bir initializer'ını çağırır. Zincir sonunda her zaman bir designated initializer'a ulaşır. Böylece property'lerin başlatılması tek bir yerde toplanmış olur.

S. Two phase initialization nedir, neden var?

C. Birinci aşamada her class aşağıdan yukarıya doğru kendi stored property'lerine değer atar, bu aşamada `self` kullanılamaz. Tüm property'ler değer aldıktan sonra ikinci aşamada `self` kullanılabilir hale gelir ve metot çağrılabilir. Amaç, hiçbir kodun başlatılmamış bir property'ye erişmemesini garanti etmektir. Özellikle superclass initializer'ı override edilmiş bir metot çağırdığında, subclass'ın property'leri zaten hazır olmuş olur.

S. Struct'a özel bir init yazınca memberwise init kaybolur. Hem özel init hem de memberwise init nasıl tutulur?

C. Özel initializer'ı struct'ın gövdesine değil bir extension'a yazarak. Extension'daki initializer'lar memberwise initializer'ın oluşturulmasını engellemez.

S. Failable initializer ne zaman, throwing initializer ne zaman?

C. Başarısızlığın tek bir anlamı varsa ve nedeni önemli değilse failable (`init?`) kullanırım, örneğin geçersiz bir string'den `EmailAddress` oluşturmak. Çağıranın neden başarısız olduğunu bilmesi ve buna göre farklı davranması gerekiyorsa `throws` kullanırım. Örneğin bir formda hangi alanın hangi kuralı ihlal ettiğini göstermek.

S. UIViewController subclass'ında neden required init?(coder:) yazmak zorundayız?

C. `init?(coder:)` , `NSCoding` protokolünden gelen `required` bir initializer'dır ve storyboard ya da xib'den nesne oluşturulurken kullanılır. Subclass kendi designated initializer'ını tanımladığında superclass'ın initializer'larını otomatik miras almayı bırakır. `required` olan bu initializer'ın ise her subclass'ta bulunması zorunludur. Storyboard kullanılmıyorsa `fatalError` ile implemente edilir.

15. Deinitialization

Deinitializer, bir class instance'ı bellekten silinmeden hemen önce çalışır. Sadece class'larda vardır ve `deinit` anahtar kelimesiyle, parametresiz ve parantezsiz tanımlanır. Elle çağrılamaz, ARC tarafından otomatik çağrılır. Subclass'ın `deinit`'i bittikten sonra superclass'ınki otomatik olarak çalışır.

Swift belleği otomatik yönettiği için `deinit`'e çoğu zaman gerek yoktur. Kullanıldığı durumlar, ARC'ın bilmediği kaynakların bırakılmasıdır: bir observer kaydını kaldırmak, bir timer'ı durdurmak, açık bir dosyayı ya da bağlantıyı kapatmak.

```
final class LiveScoreSubscription {
    let matchID: Int

    init(matchID: Int) {
        self.matchID = matchID
        print("Subscribed to match \(matchID)")
    }

    deinit {
        print("Unsubscribed from match \(matchID)")
    }
}

func showMatchScreen() {
    let subscription = LiveScoreSubscription(matchID: 7)
    print("Showing match \(subscription.matchID)")
}

showMatchScreen()
print("Back to home")
// Subscribed to match 7
// Showing match 7
// Unsubscribed from match 7
// Back to home
```

`subscription` sadece fonksiyonun içinde tutulduğu için fonksiyon bittiğinde referans sayısı sıfıra düşer ve `deinit` hemen çalışır.

Tercih

`deinit` içine bir `print` koymak, bir ekranın kapandıktan sonra gerçekten bellekten silinip silinmediğini anlamanın en hızlı yoludur. Ekran kapandığı halde mesaj görünmüyorsa büyük ihtimalle bir retain cycle vardır (bkz. Automatic Reference Counting).

Not

Global ya da top level bir değişkende tutulan nesneler program bitene kadar yaşar ve `deinit` mesajları görünmeyebilir. `deinit`'i denerken nesneyi bir fonksiyonun içinde oluşturmak ya da optional bir değişkene atayıp sonra `nil` yapmak gerekir.

Mülakat soruları

S. deinit ne zaman çalışır? Struct'larda neden yok?

C. Bir class instance'ına olan güçlü referans sayısı sıfıra düştüğünde, nesne bellekten silinmeden hemen önce ARC tarafından çağrılır. Struct'lar value type'tır. Referans sayılmaz ve her kopya bağımsızdır. "Son kopya ne zaman yok oldu" gibi anlamlı bir an olmadığı için `deinit`'leri yoktur.

S. deinit'te ne tür işler yapılır?

C. ARC'ın otomatik olarak bırakamadığı kaynaklar temizlenir: `NotificationCenter` observer'larını kaldırmak, bir `Timer`'ı geçersiz kılmak, dosya ya da soket bağlantısını kapatmak, bir aboneliği iptal etmek. Ayrıca debug sırasında bir nesnenin bellekten silinip silinmediğini kontrol etmek için log yazılır.

16. Optional Chaining

Optional chaining, optional olabilecek bir değer üzerinden property, metot ya da subscript'e erişmeyi sağlar. Zincirdeki herhangi bir halka `nil` ise zincirin geri kalanı çalışmaz ve tüm ifade `nil` döner. Force unwrap'ten farkı, `nil` durumunda crash olmak yerine sessizce `nil` üretmesidir.

```
struct Address {
    var city: String
    var postalCode: String?
}

struct Customer {
    var name: String
    var address: Address?
}

struct Order {
    var customer: Customer?
}

let order = Order(customer: Customer(name: "Ada", address: nil))

let city = order.customer?.address?.city
print(city ?? "No city")    // No city
```

Sonuç her zaman optional'dır

Zincirin sonundaki değer optional olmasa bile sonuç optional olur, çünkü zincir herhangi bir noktada kesilebilir. `city` property'si `String` tipindedir ama `order.customer?.address?.city` ifadesinin tipi `String?`'dir. Zincir birden fazla optional'dan geçse de sonuç tek katmanlı optional olur, `String??` olmaz.

Metot çağrısı ve atama

Optional chaining ile metot çağrılabilir. Dönüş tipi `Void` olan bir metot zincir üzerinden çağrıldığında sonuç `Void?` olur. Bu değer, çağrının gerçekten yapılıp yapılmadığını anlamak için `nil` ile karşılaştırılabilir.

Zincir üzerinden atama da yapılabilir. Zincir `nil`'e takılırsa atama yapılmaz ve eşittir işaretinin sağ tarafı hiç değerlendirilmez.

```
final class SessionStore {
    var currentUser: UserProfile?
}

final class UserProfile {
    var displayName: String
    init(displayName: String) { self.displayName = displayName }

    func signOut() {
        print("\(displayName) signed out")
    }
}

let store = SessionStore()
store.currentUser?.displayName = "Guest"           // kullanıcı yok, atama yapılmaz

if store.currentUser?.signOut() == nil {
    print("Nobody to sign out")                    // Nobody to sign out
}

store.currentUser = UserProfile(displayName: "Ada")
store.currentUser?.signOut()                       // Ada signed out
```

> **Dikkat**
>
> Zincir üzerinden yapılan bir atamanın gerçekleşmemesi hiçbir uyarı üretmez. `store.currentUser?.displayName = "Guest"` satırı kullanıcı `nil` iken sessizce hiçbir şey yapmaz. Atamanın mutlaka gerçekleşmesi gerekiyorsa `guard let` ile değer açılır ve yokluk ayrıca ele alınır.

Subscript ve optional dönüş

Optional bir değerin subscript'ine `?` ile erişilir. Subscript'in kendisi optional döndürüyorsa (sözlükte olduğu gibi), `?` subscript'ten sonra yazılarak sonucun üzerinden zincire devam edilir.

```
let featureConfig: [String: [String]]? = ["beta": ["darkMode", "newSearch"]]

let firstBetaFeature = featureConfig?["beta"]?.first
print(firstBetaFeature ?? "none")    // darkMode
```

Mülakat soruları

S. Optional chaining ile force unwrap arasındaki fark ne?

C. Force unwrap (`!`) değer `nil` ise programı crash ettirir. Optional chaining (`?`) ise `nil` ile karşılaştığında zincirin geri kalanını çalıştırmaz ve tüm ifadenin sonucu olarak `nil` döndürür. Bu yüzden optional chaining'in sonucu, zincirin sonundaki değer optional olmasa bile her zaman optional'dır.

S. a?.b?.c ifadesinde c optional değilse sonucun tipi nedir?

C. `c` 'nin tipi `T` ise sonuç `T?` olur. Zincir kaç optional'dan geçerse geçsin sonuç tek katmanlı optional'dır. `c` 'nin kendisi `T?` ise sonuç yine `T?` olur, `T??` olmaz.

17. Error Handling

Swift'te hatalar, `Error` protokolüne uyan tiplerle temsil edilir. Bir fonksiyon hata fırlatabiliyorsa imzasında `throws` yazar. Çağıran taraf `try` yazmak zorundadır, bu sayede hata fırlatabilecek her çağrı kodda görünür. Swift'in hata mekanizması exception'lara benzer bir söz dizimi kullanır ama stack unwinding gibi pahalı işlemler yapmaz. Bir hata fırlatmak, performans açısından bir değer döndürmekle benzerdir.

Hata tipini tanımlamak

Hatalar genellikle enum ile modellenir. Her case bir hata durumunu, associated value'lar ise hatanın ayrıntılarını taşır.

```
enum CheckoutError: Error {
    case emptyCart
    case outOfStock(sku: String)
    case paymentDeclined(reason: String)
}
```

Kullanıcıya gösterilecek mesajlar için `LocalizedError` protokolüne uyulur. `localizedDescription` property'si bu durumda `errorDescription`'ı döndürür.

```
enum UploadError: LocalizedError {
    case fileTooLarge(limitMB: Int)
    case unsupportedFormat(String)

    var errorDescription: String? {
        switch self {
        case .fileTooLarge(let limit): "File must be smaller than \(limit) MB."
        case .unsupportedFormat(let ext): "\(ext.uppercased()) files are not supported."
        }
    }
}

let error: Error = UploadError.fileTooLarge(limitMB: 25)
print(error.localizedDescription)    // File must be smaller than 25 MB.
```

Hata fırlatmak ve iletmek

`throws` ile işaretlenen fonksiyon `throw` ile hata fırlatır. İçinde başka bir throwing fonksiyonu `try` ile çağırır ve hatayı yakalamazsa, hata otomatik olarak çağırana iletilir (propagation).

```
struct InventoryService {
    let stock: [String: Int]

    func reserve(sku: String, quantity: Int) throws {
        guard let available = stock[sku], available >= quantity else {
            throw CheckoutError.outOfStock(sku: sku)
        }
    }
}

struct CheckoutService {
    let inventory: InventoryService

    func checkout(cart: [String: Int]) throws -> String {
        guard !cart.isEmpty else { throw CheckoutError.emptyCart }
        for (sku, quantity) in cart {
            try inventory.reserve(sku: sku, quantity: quantity)    // hata yakalanmaz,
yukarı iletilir
        }
        return "ORD-1001"
    }
}
```

do-catch

Hata `do-catch` ile yakalanır. `catch` blokları yukarıdan aşağıya kontrol edilir ve pattern ile eşleşen ilk blok çalışır. Pattern'siz `catch` her hatayı yakalar ve hata `error` adıyla gelir.

```
let service = CheckoutService(inventory: InventoryService(stock: ["KB-01": 3, "MS-07":
0]))

do {
    let orderID = try service.checkout(cart: ["MS-07": 1])
    print("Order placed: \(orderID)")
} catch CheckoutError.outOfStock(let sku) {
    print("\(sku) is out of stock")             // MS-07 is out of stock
} catch CheckoutError.emptyCart {
    print("Your cart is empty")
} catch {
    print("Unexpected error: \(error)")
}
```

try, try? ve try!

Yazım	Hata olursa	Ne zaman
`try`	Hata `catch`'e ya da çağırana iletilir	Hatanın nedeni önemliyse, kullanıcıya gösterilecek ya da farklı ele alınacaksa
`try?`	Sonuç `nil` olur, hata bilgisi kaybolur	Başarısızlığın nedeni önemli değilse. Örnek: cache'ten okuma, okunamazsa ağdan çekilir
`try!`	Program crash olur	Sadece hata olmasının programcı hatası anlamına geldiği yerlerde. Örnek: uygulama paketindeki bir dosyayı ya da sabit bir regex'i yüklemek

```
func loadCachedProfile(from data: Data?) -> [String: String]? {
    guard let data else { return nil }
    return try? JSONDecoder().decode([String: String].self, from: data)
}

print(loadCachedProfile(from: Data("not json".utf8)) == nil)    // true
```

> **Tehlike**
>
> `try!` force unwrap ile aynı risk sınıfındadır. Ağdan, diskten ya da kullanıcıdan gelen verinin işlenmesinde asla kullanılmaz.

defer ile temizlik

Throwing bir fonksiyonda hangi satırda hata fırlatılırsa fırlatılsın yapılması gereken işler `defer` ile yazılır. Ayrıntılar Control Flow bölümündedir.

Typed throws

Swift 6 ile bir fonksiyonun hangi hata tipini fırlatacağı imzada belirtilebilir: `throws(CheckoutError)`. Bu durumda `catch` bloğundaki `error` değeri `any Error` değil, doğrudan `CheckoutError` tipindedir ve `switch` ile tüm case'ler eksiksiz ele alınabilir.

```
func validateCart(_ cart: [String: Int]) throws(CheckoutError) {
    if cart.isEmpty { throw .emptyCart }
}

do {
    try validateCart([:])
} catch {
    switch error {
    case .emptyCart: print("Add something first")
    case .outOfStock, .paymentDeclined: print("Other checkout error")
    }
}
```

Tercih

Typed throws, hata kümesinin kapalı ve sabit olduğu yerlerde kullanılır: bir modülün iç fonksiyonları ya da gömülü sistemler gibi. Public API'lerde genellikle düz `throws` tercih edilir. Hata tipini imzaya yazmak, ileride yeni bir hata türü eklemeyi API'yi kıran bir değişikliğe dönüştürür.

Result

`Result<Success, Failure>`, bir işlemin başarılı sonucunu ya da hatasını tek bir değer olarak taşır. async/await öncesinde completion handler'larda yaygın olarak kullanılırdı. Hâlâ bir sonucu saklamak ya da başka bir yere iletmek gerektiğinde kullanışlıdır. `get()` metodu ile tekrar `throws` dünyasına dönülür.

```
func parsePort(_ text: String) -> Result<Int, URLError> {
    guard let port = Int(text), (1...65_535).contains(port) else {
        return .failure(URLError(.badURL))
    }
    return .success(port)
}

switch parsePort("8080") {
case .success(let port): print("Port \(port)")    // Port 8080
case .failure(let error): print(error.code)
}

let port = try? parsePort("99999").get()
print(port ?? -1)    // -1
```

rethrows

Parametre olarak closure alan bir fonksiyon, sadece closure hata fırlattığında hata fırlatıyorsa `rethrows` olarak işaretlenir. `map` ve `filter` bu şekilde tanımlıdır. Throwing olmayan bir closure verildiğinde çağıranın `try` yazmasına gerek kalmaz.

Mülakat soruları

S. try, try? ve try! arasındaki farkı ve her birini ne zaman kullandığını anlatır mısın?

C. `try` ile hata ya `do-catch` ile yakalanır ya da fonksiyon `throws` olduğu için çağırana iletilir. Hatanın nedeni önemliyse, örneğin kullanıcıya bir mesaj gösterilecekse kullanılır. `try?` hata olursa `nil` döndürür ve hata bilgisini atar. Başarısızlığın nedeni önemli olmadığında kullanılır, örneğin cache'ten okuma başarısızsa ağdan çekilir. `try!` hata olursa crash olur. Sadece hata olmasının programcı hatası anlamına geldiği yerlerde, örneğin uygulama paketindeki bir dosyayı yüklerken kullanılır.

S. Optional döndürmek ile hata fırlatmak arasında nasıl seçim yaparsın?

C. Değerin yokluğu normal bir durumsa ve tek bir anlamı varsa optional döndürürüm. `Int("abc")` ya da bir sözlükte anahtar aramak buna örnektir. Başarısızlığın birden fazla nedeni olabiliyorsa ve çağıranın bu nedene göre farklı davranması gerekiyorsa hata fırlatırım. Ağ isteği, dosya okuma ve doğrulama işlemleri buna örnektir.

S. Result tipi ne işe yarar, async/await ile hâlâ gerekli mi?

C. Başarılı sonucu ya da hatayı tek bir değer olarak taşır. async/await öncesinde completion handler'larda, hem değer hem hata için iki ayrı optional parametre kullanmanın yarattığı belirsizliği ortadan kaldırmak için kullanılırdı. async/await ile çoğu durumda doğrudan `throws` kullanılır. Ancak bir sonucu saklamak, bir koleksiyonda tutmak ya da ileride işlemek üzere başka bir yere iletmek gerektiğinde `Result` hâlâ kullanışlıdır.

S. Typed throws nedir, her yerde kullanmalı mıyız?

C. Swift 6 ile gelen, fonksiyonun hangi hata tipini fırlatacağını imzada belirtme özelliğidir: `throws(MyError)`. `catch` içinde hata doğrudan o tipte gelir ve eksiksiz `switch` yazılabilir. Her yerde kullanılmamalıdır. Hata kümesinin kapalı olduğu iç kodlarda faydalıdır. Public API'lerde ise yeni bir hata türü eklemek API'yi kıran bir değişiklik haline gelir, bu yüzden düz `throws` tercih edilir.

S. rethrows ne demek?

C. Fonksiyonun kendisi hata üretmez, sadece parametre olarak aldığı closure hata fırlatırsa onu iletir anlamına gelir. `map` ve `filter` `rethrows` olarak tanımlıdır. Throwing olmayan bir closure verildiğinde çağrıda `try` yazmak gerekmez.

18. Concurrency

Swift'in yerleşik concurrency modeli asenkron kodu `async` ve `await` ile, paralel işleri `async let` ve task group'larla, paylaşılan durumu ise actor'larla yönetir. Swift 6 dil modunda derleyici, data race oluşturabilecek kodu derleme anında yakalar. Bu bölüm konunun özetidir. Pratikteki ayrıntılar networking ve SwiftData ile birlikte ele alınır.

async ve await

`async` ile işaretlenen bir fonksiyon çalışırken askıya alınabilir (suspend). Askıya alınma, thread'i bloke etmez. Fonksiyon beklerken thread başka işleri yürütür, sonuç hazır olduğunda fonksiyon kaldığı yerden devam eder. `await`, bir askıya alınma noktasını işaretler.

```
struct Article: Decodable {
    let id: Int
    let title: String
}

enum APIError: Error {
    case badStatus(Int)
}

func fetchArticles(from url: URL) async throws -> [Article] {
    let (data, response) = try await URLSession.shared.data(from: url)
    guard let http = response as? HTTPURLResponse, (200..<300).contains(http.statusCode)
else {
        throw APIError.badStatus((response as? HTTPURLResponse)?.statusCode ?? -1)
    }
    return try JSONDecoder().decode([Article].self, from: data)
}
```

> **Önemli**
>
> `await`, "burada thread bekletilir" anlamına gelmez. "Burada fonksiyon askıya alınabilir ve devam ettiğinde farklı bir thread'de olabilir" anlamına gelir. Bu yüzden bir `await` öncesinde okunan durum, `await` sonrasında değişmiş olabilir.

Task

Senkron bir bağlamdan async bir fonksiyon çağırmak için `Task` oluşturulur. SwiftUI'da bunun yerine genellikle `.task` modifier'ı kullanılır. Bu modifier view kaybolduğunda task'i otomatik olarak iptal eder.

```
func loadInitialContent() async -> String {
    "Welcome"
}

let task = Task {
    let message = await loadInitialContent()
    print(message)
}
await task.value
```

`Task { }` içinde başlatılan iş, oluşturulduğu actor bağlamını miras alır. Bağlamdan tamamen bağımsız bir iş için `Task.detached` vardır ama nadiren gerekir.

Paralel çalıştırma: async let ve task group

Sırayla yazılan `await`'ler işleri sırayla yürütür. Birbirinden bağımsız işler `async let` ile aynı anda başlatılabilir.

```
func fetchProfile() async -> String { "Ada" }
func fetchNotifications() async -> Int { 3 }

func loadDashboard() async -> String {
    async let profile = fetchProfile()
    async let unread = fetchNotifications()
    return "\(await profile), \(await unread) unread"    // ikisi de aynı anda çalışır
}

print(await loadDashboard())    // Ada, 3 unread
```

Sayısı çalışma anında belli olan işler için `withTaskGroup` ya da `withThrowingTaskGroup` kullanılır.

```swift
func fetchThumbnailSize(id: Int) async -> Int {
    id * 1_024
}

func totalThumbnailSize(ids: [Int]) async -> Int {
    await withTaskGroup(of: Int.self) { group in
        for id in ids {
            group.addTask { await fetchThumbnailSize(id: id) }
        }
        var total = 0
        for await size in group {
            total += size
        }
        return total
    }
}

print(await totalThumbnailSize(ids: [1, 2, 3]))    // 6144
```

`async let` ve task group ile oluşturulan task'ler structured concurrency'nin parçasıdır. Ebeveyn task iptal edilirse çocuklar da iptal edilir, ebeveyn bitmeden önce tüm çocukların bitmesi beklenir.

Cancellation

Swift'te iptal işlemi kooperatiftir. Bir task iptal edildiğinde zorla durdurulmaz, sadece iptal edildi olarak işaretlenir. Uzun süren işler bu durumu `Task.checkCancellation()` ya da `Task.isCancelled` ile kontrol etmelidir. `URLSession` gibi sistem API'leri iptali kendileri kontrol eder.

Actors

Actor, kendi durumuna aynı anda sadece tek bir işin erişmesini garanti eden bir reference type'tır. Paylaşılan ve değişen durumu korumak için kullanılır. Actor'un dışından property ve metotlarına erişim `await` ile yapılır, çünkü erişim sırasında başka bir işin bitmesini beklemek gerekebilir.

```swift
actor ImageCache {
    private var storage: [URL: Data] = [:]

    func image(for url: URL) -> Data? {
        storage[url]
    }

    func store(_ data: Data, for url: URL) {
        storage[url] = data
    }
}

let cache = ImageCache()
let avatarURL = URL(string: "https://cdn.example.com/a.png")!
await cache.store(Data([0x1]), for: avatarURL)
print(await cache.image(for: avatarURL)?.count ?? 0)    // 1
```

> **Dikkat**
>
> Actor reentrancy: bir actor metodu içinde `await` olduğunda metot askıya alınır ve bu sırada aynı actor'a gelen başka çağrılar çalışabilir. `await` öncesinde kontrol edilen bir koşul, `await` sonrasında artık geçerli olmayabilir. Örneğin cache'te değer yok diye başlatılan bir indirme, beklerken başka bir çağrı tarafından da başlatılmış olabilir.

MainActor

`@MainActor`, kodun main thread'de çalışmasını garanti eden global actor'dur. UI'ı güncelleyen her şey main actor'da çalışmalıdır. SwiftUI view'ları ve UIKit sınıfları zaten `@MainActor` olarak işaretlidir. ViewModel'ler de genellikle `@MainActor` ile işaretlenir.

```swift
@MainActor
final class ArticleListViewModel {
    private(set) var titles: [String] = []
    private(set) var isLoading = false

    func load() async {
        isLoading = true
        defer { isLoading = false }
        titles = await fetchTitles()
    }

    private nonisolated func fetchTitles() async -> [String] {
        ["Swift 6", "SwiftData"]
    }
}

let listViewModel = ArticleListViewModel()
await listViewModel.load()
print(listViewModel.titles)    // ["Swift 6", "SwiftData"]
```

Sendable

`Sendable`, bir değerin concurrency sınırları arasında (task'ler ve actor'lar arasında) güvenle geçirilebileceğini belirten protokoldür. Sadece `Sendable` property'leri olan struct ve enum'lar otomatik olarak `Sendable` olur. Actor'lar her zaman `Sendable`'dır. Değişebilen durumu olan class'lar ise genellikle `Sendable` değildir. Swift 6 dil modunda `Sendable` olmayan bir değeri başka bir concurrency alanına geçirmek derleme hatasıdır.

> **Tercih**
>
> Concurrency ile ilgili derleyici hatalarının çoğu üç kalıptan biriyle çözülür: veriyi value type (struct) yapmak, paylaşılan değişen durumu bir actor'a taşımak ya da UI ile ilgili kodu `@MainActor`'a bağlamak. `@unchecked Sendable` ve `nonisolated(unsafe)` derleyicinin kontrolünü kapatır ve sadece güvenliği başka bir yolla (örneğin bir kilitle) sağlanmış kodda kullanılır.

Mülakat soruları

S. async/await, completion handler'lara göre ne kazandırır?

C. Asenkron kod yukarıdan aşağıya, senkron kod gibi okunur. İç içe callback'ler (callback hell) oluşmaz. Hata yönetimi `throws` ve `do-catch` ile doğal şekilde yapılır. Completion handler'da bir yolda handler'ı çağırmayı unutmak mümkündü, async fonksiyon ise her yolda ya değer döndürmek ya da hata fırlatmak zorundadır. Ayrıca iptal ve öncelik bilgisi structured concurrency ile otomatik olarak yayılır.

S. await yazılan satırda thread bloke olur mu?

C. Hayır. Fonksiyon askıya alınır ve thread başka işleri yürütmek için serbest kalır. Sonuç hazır olduğunda fonksiyon kaldığı yerden, muhtemelen farklı bir thread'de devam eder. Bu yüzden `await` öncesinde okunan durum, sonrasında değişmiş olabilir.

S. Actor nedir, class'tan farkı ne?

C. Actor da class gibi bir reference type'tır ama kendi değişen durumuna aynı anda sadece bir işin erişmesini garanti eder. Böylece data race derleme anında engellenir. Dışarıdan erişim `await` ile yapılır. Actor'larda inheritance yoktur. Dikkat edilmesi gereken nokta reentrancy'dir: actor metodu `await` ile askıya alındığında başka çağrılar araya girebilir.

S. @MainActor ne işe yarar?

C. İşaretlenen kodun main thread'de çalışmasını garanti eder. UI güncellemeleri main thread'de yapılmak zorunda olduğu için ViewModel'ler ve UI ile etkileşen kodlar `@MainActor` ile işaretlenir. Derleyici, başka bir bağlamdan bu koda erişirken `await` yazılmasını zorunlu tutar.

S. async let ile Task arasındaki fark ne?

C. `async let` structured concurrency'nin parçasıdır. Oluşturulan iş, bulunduğu kapsama bağlıdır, kapsam bitmeden beklenir ve ebeveyn iptal edilirse o da iptal edilir. `Task { }` ise unstructured bir task oluşturur. Kendi yaşam döngüsü vardır ve iptal edilmesi gerekiyorsa referansı saklanıp elle `cancel()` çağrılır. Birbirinden bağımsız birkaç işi aynı anda başlatıp sonuçlarını birleştirmek için `async let`, senkron bir bağlamdan async iş başlatmak için `Task` kullanılır.

S. Sendable nedir?

C. Bir değerin farklı task'ler ya da actor'lar arasında güvenle paylaşılabileceğini belirten protokoldür. `Sendable` property'leri olan struct ve enum'lar otomatik olarak `Sendable` olur. Actor'lar her zaman `Sendable`'dır. Değişebilen durumu olan class'lar genellikle değildir. Swift 6'da `Sendable` olmayan bir değeri concurrency sınırının öbür tarafına geçirmek derleme hatasıdır.

19. Macros

Macro, derleme sırasında kaynak koda yeni kod ekleyen bir yapıdır. Tekrar eden kalıpları elle yazmak yerine derleyicinin üretmesini sağlar. Macro'lar sadece kod ekler, mevcut kodu silmez ya da değiştirmez. Üretilen kod da normal kod gibi tip kontrolünden geçer.

İki tür macro vardır:

Tür	Yazım	Ne yapar	Örnek
Freestanding	`#` ile başlar	Kendi başına bir ifade ya da bildirim üretir	`#Preview`, `#warning`, `#Predicate`, `#expect`
Attached	`@` ile başlar	Bağlandığı bildirime üye, extension ya da protokol uyumu ekler	`@Observable`, `@Model`, `@Test`

```
func legacyCheckout() {
    #warning("Remove after the new payment flow ships")
}
```

Sık karşılaşılan macro'lar

- `@Observable` (Observation): Class'ın property'lerini gözlemlenebilir yapar. SwiftUI, bir view'ın hangi property'leri okuduğunu takip eder ve sadece onlar değiştiğinde view'ı günceller. `ObservableObject` ve `@Published` kullanımının yerini almıştır.
- `@Model` (SwiftData): Bir class'ı kalıcı olarak saklanabilen bir modele dönüştürür.
- `#Preview`: Xcode'da SwiftUI ya da UIKit önizlemesi tanımlar.
- `@Test` ve `#expect` (Swift Testing): Test fonksiyonu tanımlar ve beklenti kontrol eder.

> **Not**
>
> Xcode'da bir macro'nun üzerine sağ tıklayıp "Expand Macro" seçildiğinde ürettiği kod görülebilir. `@Observable`'ın property'leri nasıl değiştirdiğini görmek, SwiftUI'ın gözlem mekanizmasını anlamanın en hızlı yoludur.

Kendi macro'nu yazmak ayrı bir Swift package ve SwiftSyntax kütüphanesi gerektirir. Günlük uygulama geliştirmede genellikle hazır macro'lar kullanılır.

> **Dikkat**
>
> Property wrapper ile macro karıştırılmamalıdır. Property wrapper (`@State`, `@Published`) çalışma anında bir sarmalayıcı tip üzerinden çalışır. Attached macro (`@Observable`) ise derleme anında kod üretir. Her ikisi de `@` ile yazıldığı için ilk bakışta benzer görünürler. `@escaping` gibi attribute'lar ise ikisinden de farklıdır, derleyiciye sadece bilgi verir ve kod üretmez.

Mülakat soruları

S. Macro nedir, property wrapper'dan farkı ne?

C. Macro derleme anında kaynak koda yeni kod ekler. Üretilen kod tip kontrolünden geçer ve Xcode'da açılıp görülebilir. Property wrapper ise çalışma anında, property'nin değerini bir sarmalayıcı tip üzerinden saklayarak çalışır. `@Observable` bir macro'dur ve class'a gözlem kodunu derleme anında ekler. `@Published` ise bir property wrapper'dır.

S. @Observable, ObservableObject'e göre ne kazandırır?

C. `ObservableObject` ile her `@Published` property değiştiğinde, o nesneyi gözlemleyen tüm view'lar güncellenir. `@Observable` ile SwiftUI, her view'ın hangi property'leri gerçekten okuduğunu takip eder ve sadece o property'ler değiştiğinde view'ı günceller. Ayrıca `@Published` yazmaya gerek kalmaz ve view tarafında `@ObservedObject` yerine düz property ya da `@State` kullanılır.

20. Type Casting

Type casting, bir instance'ın tipini çalışma anında kontrol etmek ya da onu hiyerarşideki başka bir tip olarak ele almak için kullanılır. Class hiyerarşilerinde, protokol tiplerinde ve `Any` ile çalışırken gerekir.

Operatör	Ne yapar	Sonuç
`is`	Tip kontrolü yapar	`Bool`
`as?`	Koşullu dönüşüm yapar	Optional, başarısızsa `nil`
`as!`	Zorla dönüşüm yapar	Başarısızsa crash
`as`	Derleme anında kesin olan dönüşüm (upcast, bridging)	Doğrudan değer

```swift
class InboxItem {
    let title: String
    init(title: String) { self.title = title }
}

final class MessageItem: InboxItem {
    let sender: String
    init(title: String, sender: String) {
        self.sender = sender
        super.init(title: title)
    }
}

final class SystemItem: InboxItem {}

let inbox: [InboxItem] = [
    MessageItem(title: "New message", sender: "Ada"),
    SystemItem(title: "Update available"),
    MessageItem(title: "Reply", sender: "Linus")
]

let messageCount = inbox.filter { $0 is MessageItem }.count
print(messageCount)    // 2

for notification in inbox {
    if let message = notification as? MessageItem {
        print("\(message.sender): \(message.title)")
    }
}
```

Koşullu dönüşümde dönüştürülen değere bir isim verilmesi gerekir. `if let message = notification as? MessageItem` doğru yazımdır.

> **Dikkat**
>
> `if let x is MessageItem` ve `if let x as? MessageItem` geçersiz yazımlardır. `is` sadece `Bool` üretir ve `if notification is MessageItem` şeklinde kullanılır. Dönüştürülmüş değere ihtiyaç varsa `if let message = notification as? MessageItem` yazılır.

Birden fazla tipe göre farklı davranmak için `switch` içinde `case let x as Type` kalıbı kullanılır. Daha özel tipler daha genel olanlardan önce yazılır.

```swift
func icon(for notification: InboxItem) -> String {
    switch notification {
    case let message as MessageItem where message.sender == "Ada":
        "star"
    case is MessageItem:
        "bubble"
    case is SystemItem:
        "gear"
    default:
        "bell"
    }
}

print(inbox.map(icon))    // ["star", "gear", "bubble"]
```

Any ve AnyObject

`Any` fonksiyonlar dahil her tipten bir değeri, `AnyObject` ise sadece class instance'larını temsil eder. Tip bilgisi kaybolduğu için kullanmadan önce cast etmek gerekir. JSON'u `[String: Any]` olarak işlemek ya da Objective-C API'leriyle çalışmak gibi durumlarda karşılaşılır.

> **Tercih**
>
> `Any` yerine her zaman daha belirli bir tip tercih edilir. JSON için `Codable`, farklı tiplerdeki değerler için associated value'lu bir enum ya da bir protokol kullanmak, cast işlemlerini ve çalışma anı hatalarını ortadan kaldırır. `as!` ise sadece başarısızlığın programcı hatası olduğu yerlerde, örneğin `tableView.dequeueReusableCell(...) as! ProductCell` gibi kayıtlı bir hücreyi alırken kullanılır.

Mülakat soruları

S. as, as? ve as! arasındaki fark ne?

C. `as` derleme anında başarılı olacağı kesin olan dönüşümler içindir, örneğin bir subclass'ı superclass olarak ele almak (upcast). `as?` çalışma anında denenen koşullu dönüşümdür ve başarısızsa `nil` döner. `as!` zorla dönüşümdür ve başarısızsa crash olur. Sadece dönüşümün başarısızlığının programcı hatası olduğu yerlerde kullanılır.

S. Switch'te type casting yaparken case'leri hangi sırayla yazarsın?

C. Özelden genele doğru. `switch` eşleşen ilk case'i çalıştırır. Genel bir tip ya da koşulsuz bir case önce yazılırsa, daha özel case'lere hiç ulaşılmaz. Örneğin `where` koşullu bir case, aynı tipin koşulsuz case'inden önce gelmelidir.

S. Any ile AnyObject arasındaki fark ne?

C. `Any` struct, enum, class ve fonksiyonlar dahil her tipten değeri temsil edebilir. `AnyObject` sadece class instance'larını temsil eder. Protokollerde `AnyObject` kısıtı, protokolün sadece class'lar tarafından benimsenebileceğini belirtmek için kullanılır. Bu, delegate'lerin `weak` olarak tutulabilmesi için gereklidir.

21. Nested Types

Bir tip başka bir tipin içinde tanımlanabilir. İç tip, sadece dış tip bağlamında anlamlı olan kavramları gruplar ve isim çakışmalarını önler. Dışarıdan `DışTip.İçTip` şeklinde erişilir.

```
struct Payment {
    enum Method: String {
        case card, transfer, wallet
    }

    enum Status {
        case pending
        case completed(transactionID: String)
        case failed(Failure)
    }

    struct Failure: Error {
        let code: Int
        let message: String
    }

    let amount: Decimal
    let method: Method
    var status: Status = .pending
}

var payment = Payment(amount: 250, method: .card)
payment.status = .failed(Payment.Failure(code: 51, message: "Insufficient funds"))

let preferred = Payment.Method(rawValue: "wallet")
print(preferred == .wallet)    // true
```

`Payment.Status` adı, başka bir yerde tanımlanan `Order.Status` ya da `Shipment.Status` ile çakışmaz. Her biri kendi dış tipinin bağlamında anlam kazanır.

> **Tercih**
>
> Bir tip sadece başka bir tipin içinde kullanılıyorsa ve onun kavramsal bir parçasıysa iç içe tanımlanır. Birden fazla tip tarafından paylaşılıyorsa üst seviyede tanımlanır. Swift'in kendi API'lerinde de bu kalıp sık görülür: `String.Index`, `Dictionary.Keys`, `URLRequest.CachePolicy`.

Bir tipin `CodingKeys` enum'u iç tip kullanımının en yaygın örneğidir.

```swift
struct Product: Decodable {
    let id: Int
    let displayName: String
    let unitPrice: Decimal

    enum CodingKeys: String, CodingKey {
        case id
        case displayName = "display_name"
        case unitPrice = "unit_price"
    }
}

let data = Data(#"{"id": 7, "display_name": "Keyboard", "unit_price": 1200}"#.utf8)
let product = try JSONDecoder().decode(Product.self, from: data)
print(product.displayName)    // Keyboard
```

Mülakat soruları

S. Nested type'ı ne zaman kullanırsın?

C. Bir tip sadece başka bir tipin bağlamında anlamlıysa. Örneğin bir `Payment`'ın `Status`'u ya da bir modelin `CodingKeys`'i. Bu yaklaşım isim çakışmalarını önler (`Payment.Status` ile `Order.Status` ayrı kalır), ilişkiyi kodda açıkça gösterir ve global alanı kirletmez.

22. Extensions

Extension, var olan bir tipe yeni işlevsellik ekler. Kaynak koduna erişimin olmadığı tiplere, örneğin `String` ya da `Date` gibi standart kütüphane tiplerine de uygulanabilir.

Extension ile eklenebilenler: computed property'ler, instance ve type metotları, yeni initializer'lar, subscript'ler, nested type'lar ve protokol uyumu.

Extension ile eklenemeyenler: stored property'ler, property observer'lar, mevcut bir metodu override etmek ve class'lara designated initializer.

Mevcut tiplere işlev eklemek

```
extension Date {
    func isSameDay(as other: Date, calendar: Calendar = .current) -> Bool {
        calendar.isDate(self, inSameDayAs: other)
    }
}

extension String {
    var isValidEmail: Bool {
        let parts = split(separator: "@")
        return parts.count == 2 && parts[1].contains(".")
    }
}

print("ada@example.com".isValidEmail)    // true
print(Date().isSameDay(as: Date()))      // true
```

Kodu organize etmek

Extension'ların en yaygın kullanımlarından biri bir tipin kodunu mantıksal parçalara ayırmaktır. Her protokol uyumu ayrı bir extension'da yazılır. Böylece hangi metodun hangi protokole ait olduğu açıkça görülür.

```
struct Invoice {
    let number: String
    let total: Decimal
}

extension Invoice: CustomStringConvertible {
    var description: String {
        "Invoice \(number): \(total)"
    }
}

extension Invoice: Equatable {}

print(Invoice(number: "INV-7", total: 320))    // Invoice INV-7: 320
```

UIKit projelerinde `UITableViewDataSource` ve `UITableViewDelegate` metotlarının ayrı extension'larda toplanması bu kalıbın bilinen bir örneğidir.

Initializer eklemek

Struct'a extension ile initializer eklemek memberwise initializer'ı korur (bkz. Initialization). Class'lara extension ile sadece convenience initializer eklenebilir.

Constrained extensions

`where` ile sadece belirli koşulları sağlayan tiplere uygulanan extension'lar yazılabilir.

```
extension Array where Element == Decimal {
    var total: Decimal {
        reduce(0, +)
    }
}

let lineTotals: [Decimal] = [120, 80.5, 45]
print(lineTotals.total)    // 245.5
```

> **Dikkat**
>
> Extension'lar stored property ekleyemez. Bunun sebebi, bir tipin bellekteki boyutunun derleme anında sabit olması gerektiğidir. Başka bir modülde tanımlanmış bir tipe sonradan stored property eklemek bu boyutu değiştirirdi. Ek bir değer saklamak gerekiyorsa, tipi saran yeni bir tip yazılır.

Mülakat soruları

S. Extension ile neler eklenebilir, neler eklenemez?

C. Computed property, metot, initializer, subscript, nested type ve protokol uyumu eklenebilir. Stored property, property observer ve class'lar için designated initializer eklenemez. Mevcut bir metot da extension ile override edilemez.

S. Extension'lar neden stored property ekleyemez?

C. Bir tipin bellekteki yerleşimi ve boyutu derleme anında belirlenir. Extension'lar başka modüllerde, tipin tanımından bağımsız olarak yazılabildiği için stored property eklemek, tipin bu yerleşimini sonradan değiştirmek anlamına gelirdi. Instance'ları zaten oluşturulmuş kodlar bu değişiklikten haberdar olamazdı.

S. Protokol uyumlarını neden ayrı extension'larda yazarsın?

C. Kod organizasyonu için. Her extension tek bir protokolün gereksinimlerini içerir. Böylece hangi metodun hangi protokol için yazıldığı açıkça görülür, dosyada gezinmek kolaylaşır ve bir protokol uyumu kaldırılmak istendiğinde tek bir blok silinir.

23. Protocols

Protokol, bir tipin sahip olması gereken property, metot ve diğer gereksinimlerin tanımıdır. Implementasyon içermez, sadece sözleşmeyi belirler. Class, struct ve enum'lar protokollere uyabilir (conform). Swift'te soyutlama, test edilebilirlik ve kod paylaşımı büyük ölçüde protokoller üzerine kuruludur. Apple bu yaklaşımı protocol oriented programming olarak adlandırır.

Property ve metot gereksinimleri

Protokolde property'ler her zaman `var` ile yazılır ve sadece okunabilir mi (`{ get }`) yoksa yazılabilir mi (`{ get set }`) olduğu belirtilir. Stored mı computed mı olacağına uyan tip karar verir. Değer tipinin kendisini değiştiren metotlar `mutating` olarak yazılır. Böylece hem struct'lar hem class'lar protokole uyabilir.

```
protocol Trackable {
    var eventName: String { get }
    var parameters: [String: String] { get }
}

struct ScreenView: Trackable {
    let screen: String

    var eventName: String { "screen_view" }
    var parameters: [String: String] { ["screen": screen] }
}

struct Purchase: Trackable {
    let sku: String
    let amount: Decimal

    let eventName = "purchase"
    var parameters: [String: String] { ["sku": sku, "amount": "\(amount)"] }
}

let events: [any Trackable] = [ScreenView(screen: "Home"), Purchase(sku: "KB-01", amount:
1200)]
for event in events {
    print(event.eventName, event.parameters.count)
}
// screen_view 1
// purchase 2
```

`ScreenView` gereksinimi computed property ile, `Purchase` ise `eventName`'i stored property ile karşılıyor. Protokol için ikisi de geçerlidir.

> **Dikkat**
>
> Protokolde property olarak tanımlanan bir gereksinim, uyan tipte metot ile karşılanamaz. `var summary: String { get }` gereksinimine `func summary() -> String` yazmak protokol uyumunu sağlamaz ve derleyici "does not conform" hatası verir.

Protokolü tip olarak kullanmak

Protokol bir tip olarak kullanılabilir: parametre, dönüş tipi, property ya da koleksiyon elemanı. Bu durumda Swift 5.6'dan itibaren başına `any` yazılır: `[any Trackable]`. `any`, "bu protokole uyan herhangi bir tip" anlamına gelen bir kutudur ve farklı tiplerin aynı koleksiyonda tutulmasını sağlar. Ayrıntılar Opaque and Boxed Protocol Types bölümündedir.

Delegation

Delegation, bir nesnenin bazı kararları ya da olay bildirimlerini başka bir nesneye devretmesidir. Devredilecek işler bir protokolle tanımlanır. UIKit'in neredeyse tamamı bu kalıp üzerine kuruludur: `UITableViewDelegate`, `UITextFieldDelegate`, `URLSessionDelegate`.

```
protocol DownloadManagerDelegate: AnyObject {
    func downloadManager(_ manager: DownloadManager, didUpdateProgress progress: Double)
    func downloadManager(_ manager: DownloadManager, didFinishWith data: Data)
}

final class DownloadManager {
    weak var delegate: DownloadManagerDelegate?

    func start() {
        for step in 1...2 {
            delegate?.downloadManager(self, didUpdateProgress: Double(step) / 2)
        }
        delegate?.downloadManager(self, didFinishWith: Data([0xCA, 0xFE]))
    }
}

final class DownloadScreen: DownloadManagerDelegate {
    let manager = DownloadManager()

    init() {
        manager.delegate = self
    }

    func downloadManager(_ manager: DownloadManager, didUpdateProgress progress: Double)
{
        print("Progress: \(Int(progress * 100))%")
    }

    func downloadManager(_ manager: DownloadManager, didFinishWith data: Data) {
        print("Finished, \(data.count) bytes")
    }
}

let screen = DownloadScreen()
screen.manager.start()
// Progress: 50%
// Progress: 100%
// Finished, 2 bytes
```

> **Önemli**
>
> Delegate property'si her zaman `weak` olarak tanımlanır. Ekran manager'ı güçlü şekilde tutar. Manager da ekranı güçlü şekilde tutsaydı ikisi birbirini bellekte tutardı ve hiçbiri silinmezdi (retain cycle). `weak` sadece class instance'larına uygulanabildiği için delegate protokolü `AnyObject` ile kısıtlanır. Delegate metotlarının ilk parametresi olarak gönderen nesnenin verilmesi de Apple'ın standart kalıbıdır. Bu sayede bir delegate birden fazla nesneyi dinleyebilir.

Protocol extensions ve varsayılan implementasyon

Protokole extension ile implementasyon eklenebilir. Protokole uyan her tip bu implementasyonu otomatik olarak kazanır. Gereksinim listesindeki bir metoda extension'da implementasyon verilirse, bu varsayılan implementasyon olur. Uyan tip isterse kendi implementasyonunu yazar.

```swift
protocol Exportable {
    var exportFields: [String] { get }
    func csvLine() -> String
}

extension Exportable {
    func csvLine() -> String {
        exportFields.joined(separator: ",")
    }
}

struct Customer: Exportable {
    let name: String
    let email: String
    var exportFields: [String] { [name, email] }
}

struct Note: Exportable {
    let text: String
    var exportFields: [String] { [text] }

    func csvLine() -> String {
        "\"\(text.replacingOccurrences(of: "\"", with: "\"\""))\""
    }
}

print(Customer(name: "Ada", email: "ada@example.com").csvLine())    //
Ada,ada@example.com
print(Note(text: #"Say "hi""#).csvLine())                           // "Say ""hi"""
```

Tehlike

Sadece extension'da tanımlanan, protokolün gereksinim listesinde olmayan bir metot dinamik dispatch kullanmaz. Değişkenin tipi protokol ise, uyan tip aynı metodu kendisi yazmış olsa bile extension'daki implementasyon çalışır. Bu fark sessiz ve zor bulunan hatalara yol açar.

```
protocol Greeter {
    func greet() -> String
}

extension Greeter {
    func greet() -> String { "Hello" }
    func farewell() -> String { "Bye" }    // gereksinim listesinde yok
}

struct FormalGreeter: Greeter {
    func greet() -> String { "Good morning" }
    func farewell() -> String { "Farewell" }
}

let greeter: any Greeter = FormalGreeter()
print(greeter.greet())       // Good morning, gereksinim olduğu için dinamik
print(greeter.farewell())    // Bye, sadece extension'da olduğu için statik
```

Protokol bileşimi ve inheritance

Bir protokol başka protokollerden türeyebilir. Bir değerin birden fazla protokole aynı anda uyması gerekiyorsa `&` ile birleştirilir.

```swift
protocol Named {
    var name: String { get }
}

protocol Priced {
    var price: Decimal { get }
}

struct Course: Named, Priced {
    let name: String
    let price: Decimal
}

func label(for item: some Named & Priced) -> String {
    "\(item.name) — \(item.price)"
}

print(label(for: Course(name: "Swift Concurrency", price: 499)))
```

Synthesized conformance

`Equatable`, `Hashable`, `Comparable` (raw value'su olmayan enum'larda) ve `Codable` için Swift, uyumu otomatik olarak üretebilir. Koşul, tüm stored property'lerin (ya da associated value'ların) da aynı protokole uymasıdır.

```swift
struct Tag: Hashable, Codable {
    let id: Int
    let title: String
}

let uniqueTags: Set<Tag> = [Tag(id: 1, title: "swift"), Tag(id: 1, title: "swift")]
print(uniqueTags.count)    // 1

enum Severity: Comparable {
    case low, medium, high
}

print([Severity.high, .low, .medium].sorted().map { "\($0)" })    // ["low", "medium",
"high"]
```

Sıralama kuralı otomatik üretilemeyen tiplerde `Comparable` elle yazılır. Sadece `<` operatörünü yazmak yeterlidir, diğer karşılaştırma operatörleri protokolün varsayılan implementasyonlarından gelir.

```
struct AppVersion: Comparable {
    let major: Int
    let minor: Int

    static func < (lhs: AppVersion, rhs: AppVersion) -> Bool {
        (lhs.major, lhs.minor) < (rhs.major, rhs.minor)
    }
}

let installed = AppVersion(major: 2, minor: 9)
let required = AppVersion(major: 2, minor: 10)
print(installed < required)    // true, string olarak karşılaştırılsaydı "2.9" büyük çıkardı
```

Tercih

Bir tipi `Comparable` yapmanın kazancı, `sorted()`, `min()`, `max()` ve range gibi tüm standart kütüphane API'lerini ek kod yazmadan kullanabilmektir. Maliyeti, tip için tek bir "doğal" sıralama seçmek zorunda kalmaktır. Tip farklı ekranlarda farklı ölçütlerle sıralanıyorsa (tarihe göre, fiyata göre) `Comparable` yerine `sorted(by:)` ya da key path ile sıralama tercih edilir.

Protokol ile test edilebilirlik

Bir bileşenin bağımlılığını somut bir tip yerine protokol olarak almak, testte gerçek bağımlılık yerine sahte bir implementasyon vermeyi sağlar. Bu yaklaşım dependency injection olarak adlandırılır.

```swift
protocol ArticleFetching {
    func fetchTitles() throws -> [String]
}

struct StubArticleService: ArticleFetching {
    let result: Result<[String], Error>
    func fetchTitles() throws -> [String] { try result.get() }
}

struct ArticleListModel {
    let service: any ArticleFetching

    func headline() -> String {
        guard let titles = try? service.fetchTitles(), let first = titles.first else {
            return "No articles"
        }
        return first
    }
}

let model = ArticleListModel(service: StubArticleService(result: .success(["Swift 6 is here"])))
print(model.headline())    // Swift 6 is here
```

Mülakat soruları

S. Protocol oriented programming nedir, class inheritance'a göre avantajı ne?

C. Davranışları ve sözleşmeleri class hiyerarşisi yerine protokollerle tanımlama ve paylaşılan kodu protocol extension'larla sağlama yaklaşımıdır. Protokollere struct ve enum'lar da uyabilir, bu sayede value semantics korunur. Bir tip birden fazla protokole uyabilir, oysa tek bir superclass'tan türeyebilir. Sıkı ve derin hiyerarşiler oluşmaz. Her tip sadece ihtiyacı olan yetenekleri alır.

S. Delegate neden weak tutulur, protokol neden AnyObject ile kısıtlanır?

C. Delegate genellikle kendisine delegate olan nesneyi zaten güçlü şekilde tutar. Örneğin bir view controller kendi table view'ını tutar. Delegate de güçlü tutulsaydı iki nesne birbirini tutar ve retain cycle oluşurdu. `weak` sadece reference type'lara uygulanabilir. Protokolü `AnyObject` ile kısıtlamak, sadece class'ların uyabileceğini derleyiciye söyler ve `weak var delegate` tanımını mümkün kılar.

S. Protocol extension'da tanımlanan ama protokol gereksinimlerinde olmayan bir metodu uyan tip de yazarsa hangisi çalışır?

C. Değişkenin derleme anındaki tipine bağlıdır. Değişkenin tipi somut tipse, tipin kendi implementasyonu çalışır. Tipi protokol ise (`any P` ya da generic `T: P`), extension'daki implementasyon çalışır. Çünkü gereksinim listesinde olmayan metotlar dinamik dispatch kullanmaz, statik olarak çözülür. Metodun uyan tipe göre değişmesi isteniyorsa protokolün gereksinim listesine eklenmesi gerekir.

S. Bir tipi Comparable yapmanın maliyeti ve kazancı ne?

C. Kazancı, `<` operatörünü bir kez yazarak `sorted()` , `min()` , `max()` , `>` , `<=` ve range gibi tüm standart API'leri kullanabilmektir. Maliyeti, tip için tek ve kalıcı bir doğal sıralama seçmektir. Uygulama versiyonu gibi tek bir doğal sırası olan tiplerde mantıklıdır. Farklı ekranlarda farklı ölçütlere göre sıralanan bir ürün listesinde ise `Comparable` yerine `sorted(by:)` kullanılır.

S. Protokoller test edilebilirliğe nasıl katkı sağlar?

C. Bir bileşen bağımlılığını somut bir tip yerine protokol olarak alırsa, testte o protokole uyan sahte bir implementasyon verilebilir. Böylece ağ ya da veritabanı gibi yavaş ve kontrol edilemeyen bağımlılıklar olmadan, istenen her senaryo (başarılı cevap, hata, boş liste) kolayca test edilebilir.

24. Generics

Generic kod, herhangi bir tiple çalışabilen ama tip güvenliğini kaybetmeyen fonksiyon ve tipler yazmayı sağlar. `Array`, `Dictionary`, `Optional` ve `Result` generic tiplerdir. `Array<Element>` her eleman tipi için ayrı ayrı yazılmamıştır, ama `[Int]` içine `String` koymak yine de derleme hatasıdır.

Generic fonksiyonlar

Fonksiyon adından sonra köşeli parantez içinde bir tip parametresi tanımlanır. Bu parametre, fonksiyon her çağrıldığında gerçek bir tiple yer değiştirir.

```
func decode<T: Decodable>(_ type: T.Type, from json: String) throws -> T {
    try JSONDecoder().decode(T.self, from: Data(json.utf8))
}

struct Profile: Decodable {
    let name: String
}

let profile = try decode(Profile.self, from: #"{"name": "Ada"}"#)
let numbers = try decode([Int].self, from: "[1, 2, 3]")
print(profile.name, numbers.count)    // Ada 3
```

Generic tipler

```
struct Page<Item> {
    let items: [Item]
    let pageNumber: Int
    let hasMore: Bool

    func map<NewItem>(_ transform: (Item) -> NewItem) -> Page<NewItem> {
        Page<NewItem>(items: items.map(transform), pageNumber: pageNumber, hasMore:
hasMore)
    }
}

let page = Page(items: [3, 1, 2], pageNumber: 1, hasMore: true)
let labels = page.map { "Item \($0)" }
print(labels.items)    // ["Item 3", "Item 1", "Item 2"]
```

`Page<Int>` ile `Page<String>` farklı tiplerdir. Derleyici her birini ayrı ayrı kontrol eder.

Type constraints

Tip parametresine kısıt eklenerek sadece belirli bir protokole uyan ya da belirli bir class'tan türeyen tiplerin kabul edilmesi sağlanır. Kısıt, fonksiyonun içinde o tip üzerinde hangi işlemlerin yapılabileceğini belirler.

```swift
func mostFrequent<T: Hashable>(in values: [T]) -> T? {
    var counts: [T: Int] = [:]
    for value in values {
        counts[value, default: 0] += 1
    }
    return counts.max { $0.value < $1.value }?.key
}

print(mostFrequent(in: ["ios", "swift", "ios"]) ?? "none")    // ios
```

`T: Hashable` kısıtı olmasaydı `T` sözlük anahtarı olarak kullanılamazdı.

Associated types

Protokoller generic parametre almaz. Bunun yerine `associatedtype` ile, protokole uyan tipin belirleyeceği bir yer tutucu tip tanımlar. Swift'in `Sequence` ve `Collection` protokolleri `Element` associated type'ı ile çalışır.

```swift
protocol Repository {
    associatedtype Entity: Identifiable

    func save(_ entity: Entity)
    func find(id: Entity.ID) -> Entity?
    var all: [Entity] { get }
}

struct Book: Identifiable {
    let id: Int
    let title: String
}

final class InMemoryRepository<Entity: Identifiable>: Repository {
    private var storage: [Entity.ID: Entity] = [:]

    func save(_ entity: Entity) {
        storage[entity.id] = entity
    }

    func find(id: Entity.ID) -> Entity? {
        storage[id]
    }

    var all: [Entity] {
        Array(storage.values)
    }
}

let books = InMemoryRepository<Book>()
books.save(Book(id: 1, title: "The Swift Programming Language"))
print(books.find(id: 1)?.title ?? "missing")
```

Burada kullanılan `Identifiable`, standart kütüphanedeki protokoldür. Kendi `ID` associated type'ı vardır ve `Hashable` olması gerekir. Bu yüzden `Entity.ID` sözlük anahtarı olarak kullanılabiliyor.

Generic where clauses

`where` ile tip parametreleri ve associated type'lar üzerine daha ayrıntılı koşullar yazılır. Koşullar fonksiyonlara, extension'lara ve tek tek metotlara uygulanabilir.

```swift
func haveSameItems<A: Collection, B: Collection>(_ a: A, _ b: B) -> Bool
where A.Element == B.Element, A.Element: Hashable {
    Set(a) == Set(b)
}

print(haveSameItems(["a", "b"], Set(["b", "a"])))    // true

extension Page where Item: Identifiable {
    var ids: [Item.ID] {
        items.map(\.id)
    }
}

let bookPage = Page(items: [Book(id: 7, title: "SwiftUI")], pageNumber: 1, hasMore:
false)
print(bookPage.ids)    // [7]
```

`Page<Book>` üzerinde `ids` property'si vardır, `Page<Int>` üzerinde yoktur. Extension sadece koşulu sağlayan tiplere uygulanır.

> **Tercih**
>
> Generic mi, `any` protokol mü? Tek tip ile çalışan ve performansın önemli olduğu kodda generic (`<T: P>` ya da parametrede `some P`) tercih edilir. Derleyici gerçek tipi bilir ve her tip için optimize edilmiş kod üretebilir. Farklı tiplerin aynı koleksiyonda karıştırılması gerekiyorsa `any P` kullanılır.

Mülakat soruları

S. Generics ne problemi çözer?

C. Aynı mantığı farklı tipler için tekrar tekrar yazmadan, tip güvenliğini kaybetmeden yazmayı sağlar. `Any` kullanmak da kodu tekrar kullanılabilir yapar ama tip bilgisini kaybettirir ve çalışma anında cast gerektirir. Generic kodda tip bilgisi korunur, hatalar derleme anında yakalanır ve derleyici her tip için optimize edilmiş kod üretebilir.

S. associatedtype nedir, protokoller neden generic parametre yerine bunu kullanır?

C. Protokolde, gerçek tipi protokole uyan tipin belirleyeceği bir yer tutucudur. Örneğin `Collection`'ın `Element`'i ya da bir `Repository`'nin `Entity`'si. Generic parametre, tipi kullanan tarafın seçtiği bir şeydir (`Array<Int>`). Associated type ise uyan tipin kendi doğasının bir parçasıdır. Bir `[String]` dizisinin elemanı her zaman `String`'dir, kullanan taraf bunu ayrıca seçmez.

S. Generic bir tipte sadece belirli eleman tipleri için çalışan bir metot nasıl eklersin?

C. `where` koşullu bir extension yazarım: `extension Page where Item: Identifiable { ... }`. Bu extension'daki metot ve property'ler sadece koşulu sağlayan tiplerde görünür. Standart kütüphanede `Array`'in `Equatable` elemanlarda `contains(_:)` metoduna sahip olması aynı yöntemle yapılmıştır.

25. Opaque and Boxed Protocol Types

Bir protokolü tip olarak kullanmanın iki yolu vardır: opaque type (`some P`) ve boxed protocol type (`any P`). İkisi de "P'ye uyan bir tip" der ama çok farklı garantiler verir.

some: opaque type

`some P`, derleyicinin bildiği ama çağırandan gizlenen tek bir somut tiptir. Fonksiyon her zaman aynı somut tipi döndürmek zorundadır. Tip bilgisi korunduğu için derleyici optimizasyon yapabilir ve associated type'ı olan protokoller de kullanılabilir.

```
protocol Shape {
    func area() -> Double
}

struct Circle: Shape {
    let radius: Double
    func area() -> Double { .pi * radius * radius }
}

struct Square: Shape {
    let side: Double
    func area() -> Double { side * side }
}

func makeBadge() -> some Shape {
    Circle(radius: 1)
}

// func makeShape(rounded: Bool) -> some Shape {
//     if rounded { return Circle(radius: 1) }
//     return Square(side: 2)
// }
// error: function declares an opaque return type 'some Shape', but the return statements
in its body do not have matching underlying types
```

SwiftUI'daki `var body: some View` bu mekanizmayı kullanır. Body'nin gerçek tipi `VStack<TupleView<(Text, Button<Text>)>>` gibi uzun ve iç içe bir tiptir. `some View` bu tipi yazma zorunluluğunu ortadan kaldırırken SwiftUI'ın gerçek tipi bilmesini sağlar.

Parametre konumunda `some P` yazmak generic yazımın kısa halidir. `func render(_ shape: some Shape)` ile `func render<S: Shape>(_ shape: S)` aynı anlama gelir.

any: boxed protocol type

`any P`, P'ye uyan herhangi bir tipi tutabilen bir kutudur (existential). Somut tip derleme anında bilinmez ve çalışma anında değişebilir. Bu sayede farklı tipler aynı koleksiyonda tutulabilir.

```
let shapes: [any Shape] = [Circle(radius: 1), Square(side: 2)]
let totalArea = shapes.reduce(0) { $0 + $1.area() }
print(Int(totalArea))    // 7
```

Bu esnekliğin bir maliyeti vardır. Değer bir kutuya konur, büyük değerler için heap'te yer ayrılabilir ve metot çağrıları dolaylı olarak yapılır. Ayrıca tip bilgisi silindiği için (type erasure) bazı işlemler yapılamaz.

Kutu protokole uymaz

`any Shape` kutusunun kendisi `Shape` protokolüne uymaz. Protokollere sadece somut tipler uyabilir. Bu durum generic kodda karşımıza çıkar.

```
func describe<S: Shape>(_ shape: S) -> String {
    "Area: \(Int(shape.area()))"
}

func totalArea<S: Shape>(_ shapes: [S]) -> Double {
    shapes.reduce(0) { $0 + $1.area() }
}

let single: any Shape = Square(side: 3)
print(describe(single))    // Area: 9

let mixed: [any Shape] = [Circle(radius: 1), Square(side: 2)]
// totalArea(mixed)
// error: type 'any Shape' cannot conform to 'Shape'
```

Tek bir `any Shape` değeri generic fonksiyona verilebilir. Swift 5.7'den itibaren derleyici kutuyu otomatik olarak açar ve içindeki gerçek tipi `S` olarak kullanır (implicitly opened existential). Dizi ise açılamaz. `[S]` tüm elemanların aynı tipte olmasını ister, `[any Shape]` ise farklı tipler içerir. `S` yerine `any Shape` konmak zorunda kalınır ve kutu `Shape`'e uymadığı için kod derlenmez.

Karşılaştırma

	`some P`	`any P`
Somut tip	Tek, sabit, derleyici biliyor	Herhangi biri, çalışma anında değişebilir
Farklı tipleri karıştırmak	Mümkün değil	Mümkün
Performans	Doğrudan çağrı, optimizasyon mümkün	Kutu ve dolaylı çağrı maliyeti
Tip bilgisi	Korunur	Silinir (type erasure)
Tipik kullanım	SwiftUI `body`, generic parametre, gizlenmiş dönüş tipi	Heterojen koleksiyonlar, saklanan bağımlılıklar (`let service: any APIClient`)

> **Tercih**
>
> Varsayılan tercih `some`'dır. Farklı tiplerin aynı yerde tutulması gerçekten gerekiyorsa `any`'ye geçilir.

Mülakat soruları

S. some ile any arasındaki fark ne?

C. `some P` tek ve sabit bir somut tiptir. Derleyici bu tipi bilir ama çağırandan gizler. Tip bilgisi korunduğu için optimizasyon yapılabilir. `any P` ise P'ye uyan herhangi bir tipi tutabilen bir kutudur. Tip bilgisi silinir ve çalışma anında farklı tipler tutulabilir. Bu esnekliğin bedeli kutulama ve dolaylı çağrı maliyetidir. Varsayılan olarak `some` kullanırım. Farklı tipleri bir koleksiyonda karıştırmam gerekiyorsa `any` kullanırım.

S. SwiftUI'da body neden some View?

C. Body'nin gerçek tipi, kullanılan view'lara ve modifier'lara göre oluşan uzun ve iç içe bir generic tiptir. Bunu elle yazmak pratik değildir. `some View` bu tipi gizler ama derleyiciye ve SwiftUI'a gerçek tipi bildirir. SwiftUI bu tip bilgisini, view hiyerarşisinde neyin değiştiğini verimli şekilde tespit etmek için kullanır. `any View` kullanılsaydı bu bilgi kaybolurdu.

S. [any Shape] dizisi neden (_ shapes: [S]) fonksiyonuna verilemez?

C. `[S]` tüm elemanların aynı somut tipte olmasını ister. `[any Shape]` ise farklı tipler içerebilir. Bu yüzden `S` yerine ancak `any Shape` konabilir. Ancak `any Shape` bir kutudur ve kutunun kendisi `Shape` protokolüne uymaz, çünkü protokollere sadece somut tipler uyabilir. Tek bir `any Shape` değeri ise Swift 5.7'den itibaren otomatik olarak açıldığı için generic fonksiyona verilebilir.

S. Type erasure ne demek?

C. Bir değerin somut tip bilgisini gizleyip sadece protokol arayüzü üzerinden kullanılmasıdır. `any P` dilin yerleşik type erasure mekanizmasıdır. Swift 5.7 öncesinde associated type'ı olan protokoller `any` ile kullanılamadığı için `AnyPublisher` ya da `AnyHashable` gibi elle yazılmış sarmalayıcı tipler kullanılırdı. Bu tipler hâlâ framework'lerde karşımıza çıkar.

26. Automatic Reference Counting

Swift, class instance'larının belleğini ARC ile yönetir. Her class instance'ı için o nesneyi gösteren güçlü (strong) referansların sayısı tutulur. Sayı sıfıra düştüğünde nesne bellekten silinir ve `deinit` çalışır. ARC sadece reference type'lar (class, actor, closure) için geçerlidir. Struct ve enum'lar referans sayılmaz.

ARC, Java ya da C#'taki garbage collector'dan farklıdır. Arka planda periyodik olarak çalışan bir toplayıcı yoktur. Sayma işlemleri derleyicinin eklediği kodla anında yapılır ve nesne referansı biter bitmez silinir. Bunun karşılığında ARC, birbirini tutan nesneleri (retain cycle) kendi başına çözemez.

Strong reference cycle

İki nesne birbirini güçlü referansla tutarsa referans sayıları hiçbir zaman sıfıra düşmez. Dışarıdan bu nesnelere ulaşan son referans da kaldırılsa bile ikisi bellekte kalır. Bu duruma memory leak denir.

```swift
final class ChatRoom {
    let name: String
    var moderator: Moderator?

    init(name: String) { self.name = name }
    deinit { print("ChatRoom \(name) deallocated") }
}

final class Moderator {
    let name: String
    weak var room: ChatRoom?    // weak olmasaydı retain cycle oluşurdu

    init(name: String) { self.name = name }
    deinit { print("Moderator \(name) deallocated") }
}

func openRoom() {
    let room = ChatRoom(name: "ios-dev")
    let moderator = Moderator(name: "Ada")
    room.moderator = moderator
    moderator.room = room
}

openRoom()
// ChatRoom ios-dev deallocated
// Moderator Ada deallocated
```

`room` property'si `weak` olmasaydı, fonksiyon bittiğinde iki nesne birbirini tutmaya devam eder ve hiçbir `deinit` mesajı görünmezdi.

weak ve unowned

	weak	**unowned**
Referans sayısını artırır mı	Hayır	Hayır
Tip	Her zaman optional `var`	Optional olmayabilir, `let` olabilir
Nesne silinince	Otomatik olarak `nil` olur	Erişilirse crash olur
Ne zaman	Diğer nesne daha kısa yaşayabilir ya da yaşam süresi belirsiz	Diğer nesne en az bu nesne kadar uzun yaşar, garanti

```swift
final class Account {
    let email: String
    var card: CreditCard?

    init(email: String) { self.email = email }
    deinit { print("Account deallocated") }
}

final class CreditCard {
    let lastFour: String
    unowned let owner: Account    // kart, hesabı olmadan var olamaz

    init(lastFour: String, owner: Account) {
        self.lastFour = lastFour
        self.owner = owner
    }
    deinit { print("Card \(lastFour) deallocated") }
}

func createAccount() {
    let account = Account(email: "ada@example.com")
    account.card = CreditCard(lastFour: "4242", owner: account)
    print(account.card?.owner.email ?? "")
}

createAccount()
// ada@example.com
// Account deallocated
// Card 4242 deallocated
```

> **Önemli**
>
> `weak` ile `unowned` arasındaki seçim tamamen yaşam süresine bağlıdır. Referans verilen nesnenin, referansı tutan nesneden önce silinme ihtimali varsa `weak`. Silinme ihtimali hiç yoksa, yani iki nesne birlikte yaşayıp birlikte ölüyorsa ya da referans verilen daha uzun yaşıyorsa `unowned`. Emin değilsen `weak` güvenli olan seçimdir.

Closure'larda retain cycle

Closure'lar da reference type'tır ve kullandıkları nesneleri güçlü şekilde yakalar. Bir class bir closure'ı property olarak saklıyorsa ve closure içinde `self` kullanılıyorsa, nesne closure'ı, closure da nesneyi tutar.

```swift
final class SearchViewModel {
    var query = ""
    var onResultsChange: (([String]) -> Void)?
    private var searchHandler: (() -> Void)?

    func configure() {
        searchHandler = { [weak self] in
            guard let self else { return }
            self.onResultsChange?(["Result for \(self.query)"])
        }
    }

    func search(_ text: String) {
        query = text
        searchHandler?()
    }

    deinit { print("SearchViewModel deallocated") }
}

func runSearch() {
    let viewModel = SearchViewModel()
    viewModel.onResultsChange = { print($0) }
    viewModel.configure()
    viewModel.search("swift")
}

runSearch()
// ["Result for swift"]
// SearchViewModel deallocated
```

`[weak self]` yazılmasaydı `searchHandler` closure'ı `self`'i güçlü tutardı. `self` de `searchHandler`'ı tuttuğu için `deinit` hiçbir zaman çalışmazdı.

Tercih

Her closure'da `[weak self]` gerekmez. Ölçüt, closure'ın saklanıp saklanmadığıdır. `map`, `filter` gibi non-escaping closure'lar fonksiyon bitince yok olur, retain cycle oluşturamaz. `UIView.animate` ya da tek seferlik bir ağ isteği gibi kısa süreli escaping closure'lar nesneyi geçici olarak uzatır ama cycle oluşturmaz. Asıl risk, closure'ın `self` tarafından doğrudan ya da dolaylı olarak saklanmasıdır: property olarak tutulan callback'ler, `Timer`, `NotificationCenter` observer'ları ve Combine abonelikleri.

> **Not**
>
> `guard let self else { return }` closure'ın başında `self`'i bir kez güçlü referansa çevirir. Böylece closure çalıştığı sürece nesne silinmez ve gövdede tekrar tekrar `self?` yazmaya gerek kalmaz. Nesne zaten silinmişse closure hiçbir şey yapmadan çıkar.

Retain cycle'ı tespit etmek

- Şüphelenilen class'a `deinit` içinde bir `print` eklenir. Ekran kapatıldığında mesaj görünmüyorsa nesne bellekte kalmıştır.
- Xcode'daki Debug Memory Graph, bellekteki nesneleri ve aralarındaki referansları gösterir. Retain cycle'lar mor ünlem işaretiyle işaretlenir.
- Instruments'taki Leaks ve Allocations araçları, uygulama çalışırken sızıntıları ve bellek büyümesini gösterir.

Mülakat soruları

S. ARC nasıl çalışır, garbage collection'dan farkı ne?

C. Her class instance'ı için güçlü referans sayısı tutulur. Derleyici, referans oluşturulan ve bırakılan yerlere sayacı artıran ve azaltan kodu derleme anında ekler. Sayı sıfıra düştüğünde nesne hemen silinir. Garbage collector ise arka planda periyodik olarak çalışır ve ulaşılamayan nesneleri toplar. ARC'ın avantajı, silinme anının belirli olması ve uygulamayı duraksatan toplama işlemleri olmamasıdır. Dezavantajı, birbirini tutan nesneleri kendi başına çözememesidir.

S. Retain cycle nedir, nasıl oluşur, nasıl çözülür?

C. İki ya da daha fazla nesnenin birbirini güçlü referansla tutmasıdır. Referans sayıları hiçbir zaman sıfıra düşmez ve nesneler bellekte kalır. En sık görüldüğü yerler: delegate'in güçlü tutulması, bir nesnenin property olarak sakladığı closure içinde `self`'i güçlü yakalaması ve parent ile child nesnelerin birbirini tutması. Çözüm, ilişkinin bir tarafını `weak` ya da `unowned` yapmaktır. Hangisinin seçileceği nesnelerin yaşam süresine bağlıdır.

S. weak ile unowned arasında nasıl seçim yaparsın?

C. Yaşam süresine bakarım. Referans verilen nesne, referansı tutandan önce silinebiliyorsa `weak` kullanırım. `weak` optional'dır ve nesne silinince otomatik olarak `nil` olur. Referans verilen nesnenin en az referansı tutan kadar yaşayacağı garantiyse `unowned` kullanırım, örneğin bir kredi kartı ile ait olduğu hesap. `unowned` silinmiş bir nesneye erişilirse crash olur. Emin değilsem `weak` seçerim.

S. Her closure'da [weak self] yazmak gerekir mi?

C. Hayır. Ölçüt, closure'ın `self` tarafından doğrudan ya da dolaylı olarak saklanıp saklanmadığıdır. Non-escaping closure'lar (`map` , `filter`) cycle oluşturamaz. Tek seferlik çalışıp biten escaping closure'lar nesnenin ömrünü geçici olarak uzatır ama cycle oluşturmaz. Property olarak saklanan callback'lerde, `Timer` 'larda, observer'larda ve aboneliklerde ise `[weak self]` gerekir.

S. Bir ekranın kapandıktan sonra bellekten silinmediğini nasıl anlarsın?

C. İlk adım olarak `deinit` içine bir log eklerim ve ekranı kapattığımda mesajın görünüp görünmediğine bakarım. Görünmüyorsa Xcode'un Debug Memory Graph aracıyla nesneyi kimin tuttuğunu incelerim. Retain cycle'lar burada işaretli görünür. Daha kapsamlı analiz için Instruments'taki Leaks ve Allocations araçlarını kullanırım.

27. Memory Safety

Swift, aynı bellek alanına aynı anda çakışan erişimleri engeller. İki erişimin çakışması için üç koşulun birlikte sağlanması gerekir: en az biri yazma işlemidir, ikisi de aynı bellek alanına erişir ve süreleri örtüşür. Swift bu kuralı exclusivity olarak adlandırır. Çakışmaların çoğu derleme anında yakalanır, geri kalanı çalışma anında crash ile sonuçlanır.

Erişimlerin çoğu anlıktır ve çakışma oluşturmaz. Uzun süreli erişimler ise `inout` parametreler ve `mutating` metotlar sırasında oluşur. Bu erişimler fonksiyon boyunca sürer.

inout parametrelerde çakışma

```
func applyStep() {
    var stepSize = 1

    func increment(_ number: inout Int) {
        number += stepSize
    }

    // increment(&stepSize)
    // Simultaneous accesses to 0x..., but modification requires exclusive access.

    var copyOfStepSize = stepSize
    increment(&copyOfStepSize)
    stepSize = copyOfStepSize
    print(stepSize)    // 2
}

applyStep()
```

Yorum satırındaki çağrı çalışma anında crash olur. `increment` fonksiyonu `stepSize` için fonksiyon boyunca süren bir yazma erişimi tutarken, gövdedeki `stepSize` okuması aynı bellek alanına ikinci bir erişimdir. Çözüm, örnekte olduğu gibi değerin önce bir kopyasını almak, işlemi kopya üzerinde yapmak ve sonucu geri yazmaktır.

Aynı değişkeni iki `inout` parametreye birden vermek derleme anında hatadır.

```swift
func balance(_ x: inout Int, _ y: inout Int) {
    let sum = x + y
    x = sum / 2
    y = sum - x
}

var playerOneScore = 42
var playerTwoScore = 30
balance(&playerOneScore, &playerTwoScore)    // geçerli
// balance(&playerOneScore, &playerOneScore)
// error: overlapping accesses to 'playerOneScore', but modification requires exclusive access; consider copying to a local variable
print(playerOneScore, playerTwoScore)        // 36 36
```

Struct property'lerinde çakışma

Struct, enum ve tuple gibi value type'larda bir property'ye yazmak, değerin tamamına yazmak anlamına gelir. Bu yüzden global bir struct değişkeninin iki farklı property'sini aynı anda iki `inout` parametreye vermek çakışma sayılır. Yerel değişkenlerde ise derleyici iki property'nin örtüşmediğini kanıtlayabildiği için buna izin verir.

> **Not**
>
> Exclusivity kuralı, Swift'in değerler üzerinde güvenle optimizasyon yapabilmesini ve value semantics'in tutarlı kalmasını sağlar. Bu kural tek thread'li kodla ilgilidir. Birden fazla thread arasındaki erişimler ise Concurrency bölümündeki actor ve `Sendable` mekanizmalarıyla korunur.

Mülakat soruları

S. Swift'te memory safety ve exclusive access ne demek?

C. Aynı bellek alanına, en az biri yazma olan ve süreleri örtüşen iki erişim yapılamaz. Bu kurala exclusivity denir. Uzun süreli erişimler `inout` parametreler ve `mutating` metotlar sırasında oluşur. Örneğin bir değişkeni `inout` olarak alan fonksiyonun içinde aynı değişkeni okumak çakışmadır. Swift bu durumların çoğunu derleme anında, geri kalanını çalışma anında yakalar.

S. Exclusivity kuralı data race ile aynı şey mi?

C. Hayır. Exclusivity kuralı tek bir thread içindeki örtüşen erişimlerle ilgilidir, örneğin `inout` ile başlayan bir yazma devam ederken aynı değişkenin okunması. Data race ise farklı thread'lerin aynı veriye eş zamanlı erişmesidir. Data race'ler Swift 6'da actor'lar, `Sendable` ve strict concurrency kontrolleriyle engellenir.

28. Access Control

Access control, kodun hangi kısımlarının başka dosyalardan ve modüllerden görülebileceğini belirler. Amacı, bir tipin dışarıya açtığı arayüzü (API) iç implementasyon ayrıntılarından ayırmaktır. Böylece iç ayrıntılar, dışarıdaki kodu bozmadan değiştirilebilir.

Swift'te erişim, modül ve kaynak dosya düzeyinde tanımlanır. Modül, tek başına derlenen bir kod birimidir: bir uygulama hedefi, bir framework ya da bir Swift package.

Erişim seviyeleri

Seviye	Nereden erişilebilir
`open`	Her yerden. Başka modüllerde subclass'ı oluşturulabilir ve override edilebilir. Sadece class ve class üyeleri için
`public`	Her yerden. Başka modüllerde subclass'ı oluşturulamaz ve override edilemez
`package`	Aynı Swift package içindeki tüm modüllerden
`internal`	Aynı modül içinden. Varsayılan seviye
`fileprivate`	Sadece aynı kaynak dosyadan
`private`	Sadece tanımlandığı kapsamdan ve aynı dosyadaki o tipin extension'larından

> **Önemli**
>
> Varsayılan seviye `internal`'dır. Tek modüllü bir uygulamada hiçbir şey yazmadan her şey uygulamanın her yerinden erişilebilir. Bu yüzden erişim kontrolü özellikle framework, Swift package ve modüler uygulama mimarilerinde önem kazanır. Yine de uygulama kodunda `private` kullanmak, bir tipin iç durumunu yanlışlıkla dışarıdan değiştirmeyi engeller.

private(set)

Bir property'nin okunmasına izin verip dışarıdan değiştirilmesini engellemek için setter'ın erişimi ayrıca kısıtlanır. Bu, bir tipin kendi durumunu kontrol altında tutmasının en yaygın yoludur.

```swift
struct Wallet {
    private(set) var balance: Decimal = 0
    private var transactions: [Decimal] = []

    mutating func deposit(_ amount: Decimal) {
        precondition(amount > 0, "Deposit must be positive")
        balance += amount
        transactions.append(amount)
    }
}

var wallet = Wallet()
wallet.deposit(150)
print(wallet.balance)       // 150
// wallet.balance = 1_000_000
// error: cannot assign to property: 'balance' setter is inaccessible
```

`balance` okunabilir ama sadece `deposit` üzerinden değişebilir. Böylece bakiyenin her zaman işlem kayıtlarıyla tutarlı olması garanti edilir.

Public bir API'de property'nin dışarıdan okunup sadece modül içinde değiştirilmesi için `public private(set) var` ya da `public internal(set) var` yazılır.

Kurallar

- Bir üye, içinde bulunduğu tipten daha açık olamaz. `private` bir class'ın üyeleri pratikte en fazla `private` erişime sahiptir.
- Bir fonksiyon, parametre ve dönüş tiplerinden daha açık olamaz. `internal` bir tipi döndüren bir fonksiyon `public` yapılamaz.
- Public bir tipin üyeleri varsayılan olarak `internal`'dır. Dışarıya açılması istenen her üye ayrıca `public` olarak işaretlenir.
- Struct'ın memberwise initializer'ı, property'lerden biri `private` ise `private` olur. Public bir struct'ın memberwise initializer'ı ise `internal`'dır. Başka modüllerden kullanılabilmesi için `public init` elle yazılır.

> **Tercih**
>
> `private` ile `fileprivate` arasındaki seçimde varsayılan `private`'tır. Aynı dosyadaki extension'lar `private` üyelere zaten erişebildiği için `fileprivate` sadece aynı dosyada tanımlı farklı tiplerin birbirinin iç ayrıntılarına erişmesi gerektiğinde kullanılır.

Mülakat soruları

S. open ile public arasındaki fark ne?

C. İkisi de modül dışından erişime izin verir. `open` ayrıca başka modüllerde subclass oluşturmaya ve override etmeye izin verir. `public` bir class başka modüllerde kullanılabilir ama türetilemez. Bu ayrım, bir framework'ün hangi class'larının bilinçli olarak genişletilmeye açık olduğunu belirtmesini sağlar.

S. private(set) ne işe yarar, örnek verir misin?

C. Property'nin okunmasını serbest bırakıp yazılmasını tipin içiyle sınırlar. Örneğin bir cüzdan tipinde `private(set) var balance` yazarım. Bakiye her yerden okunabilir ama sadece `deposit` ve `withdraw` metotları üzerinden değişir. Böylece doğrulama kuralları atlanamaz ve tipin kendi durumu üzerindeki kontrolü korunur.

S. Varsayılan erişim seviyesi ne, tek modüllü bir uygulamada access control neden yine de önemli?

C. Varsayılan `internal`'dır, yani modül içindeki her yerden erişilebilir. Tek modüllü bir uygulamada bile `private` ve `private(set)` kullanmak, bir tipin iç durumunun başka yerlerden yanlışlıkla değiştirilmesini engeller ve tipin gerçekte hangi arayüzü sunduğunu açıkça gösterir. Ayrıca derleyici, `private` ve `final` bilgilerini optimizasyon için kullanabilir.

29. Advanced Operators

Bu bölüm bit düzeyindeki operatörleri, taşmaya izin veren operatörleri ve kendi tiplerin için operatör tanımlamayı kapsar.

Bitwise operators

`~` (NOT), `&` (AND), `|` (OR), `^` (XOR), `<<` ve `>>` (shift) operatörleri tam sayıların bitleri üzerinde çalışır. Uygulama kodunda en sık bayrak (flag) kümeleri, renk kodlarını ayrıştırma ve ikili protokol verileri ile çalışırken karşılaşılır.

```
let hexColor: UInt32 = 0x34C759
let red = (hexColor >> 16) & 0xFF
let green = (hexColor >> 8) & 0xFF
let blue = hexColor & 0xFF
print(red, green, blue)    // 52 199 89
```

Bayrak kümeleri için Swift'te `OptionSet` protokolü kullanılır. Her seçenek bir bittir ve set söz dizimi ile birleştirilir.

```
struct NotificationOptions: OptionSet {
    let rawValue: Int

    static let alert = NotificationOptions(rawValue: 1 << 0)
    static let badge = NotificationOptions(rawValue: 1 << 1)
    static let sound = NotificationOptions(rawValue: 1 << 2)
}

let options: NotificationOptions = [.alert, .sound]
print(options.contains(.sound))    // true
print(options.contains(.badge))    // false
print(options.rawValue)            // 5
```

`UNAuthorizationOptions` ve `UIView.AutoresizingMask` gibi Apple tipleri de `OptionSet`'tir.

Overflow operators

Swift'te tam sayı taşması varsayılan olarak çalışma anında crash ile sonuçlanır. Taşmanın bilinçli olarak istendiği durumlarda `&+`, `&-` ve `&*` kullanılır. Bu operatörler taşan bitleri atar ve değer başa sarar (wrap around). Hash hesaplama ve bazı kriptografik işlemler tipik kullanım alanlarıdır.

```swift
var hash: UInt8 = 250
hash = hash &+ 10
print(hash)    // 4
```

Operator overloading

Kendi tiplerin için var olan operatörlerin implementasyonu `static func` olarak yazılır. Bileşik atama operatörleri (`+=`) sol tarafı değiştirdiği için `inout` parametre alır. Tek operandlı operatörler `prefix` ya da `postfix` ile işaretlenir.

```swift
struct Money: Equatable {
    var amount: Decimal
    let currency: String

    static func + (lhs: Money, rhs: Money) -> Money {
        precondition(lhs.currency == rhs.currency, "Currency mismatch")
        return Money(amount: lhs.amount + rhs.amount, currency: lhs.currency)
    }

    static func += (lhs: inout Money, rhs: Money) {
        lhs = lhs + rhs
    }

    static prefix func - (value: Money) -> Money {
        Money(amount: -value.amount, currency: value.currency)
    }
}

var balance = Money(amount: 100, currency: "TRY")
let refund = Money(amount: 25, currency: "TRY")
balance += refund
print(balance.amount)           // 125
print((-refund).amount)         // -25
print(balance == Money(amount: 125, currency: "TRY"))    // true
```

> **Dikkat**
>
> `+=` gibi bileşik atama operatörleri değer döndürmez. `print(balance += refund)` yazmak `()` yazdırır. Önce işlem yapılır, sonra değer ayrıca yazdırılır.

> **Tercih**
>
> Operatör overloading sadece anlamı tartışmasız olduğunda kullanılır: para, vektör ve ölçü birimleri gibi matematiksel değerler. Bir operatörün ne yaptığını anlamak için tanımına bakmak gerekiyorsa, açık isimli bir metot daha iyi bir seçimdir.

Swift yeni operatör tanımlamaya da izin verir (`prefix operator`, `infix operator` ve precedence group ile). Ancak uygulama kodunda nadiren kullanılır, çünkü ekipteki herkesin yeni bir sembolü öğrenmesi gerekir.

Mülakat soruları

S. OptionSet nedir, nerede karşına çıkar?

C. Bir bit kümesini, her bitin bir seçeneği temsil ettiği tip güvenli bir set olarak modelleyen protokoldür. Seçenekler `[.alert, .sound]` şeklinde set söz dizimiyle birleştirilir ve `contains` ile kontrol edilir. Bildirim izinlerinde (`UNAuthorizationOptions`) ve view yeniden boyutlandırma maskelerinde karşımıza çıkar.

S. Overflow operatörleri ne zaman kullanılır?

C. Taşmanın hata değil beklenen davranış olduğu yerlerde. Örneğin hash hesaplamada değerin başa sarması istenir. `&+`, `&-`, `&*` taşan bitleri atar ve crash oluşturmaz. Normal operatörlerle taşma ise çalışma anında crash'e yol açar. Bu tasarım, istenmeyen taşmaların sessizce yanlış sonuç üretmesini engeller.

S. Kendi tipin için operatör tanımlamak ne zaman doğru bir karardır?

C. Operatörün anlamı herkes için açık olduğunda. Para, vektör ve ölçü birimleri gibi matematiksel değerlerde `+` ya da `==` doğal bir seçimdir. Anlamı tartışmalı olan durumlarda, örneğin iki kullanıcı nesnesini `+` ile birleştirmek, açık isimli bir metot daha okunaklıdır.